



AI-Based Sinhala Assistant for Personalized A/L and O/L Learning

An undergraduate project report submitted to the

Department of Electrical and Information Engineering
Faculty of Engineering
University of Ruhuna
Sri Lanka

in partial fulfillment of the requirements for the

Degree of the Bachelor of the Science of Engineering Honours

by

Akarshana A.N	-	EG/2021/4392
Gunasekara K.K.S.	-	EG/2021/4527
Sachinthana K.	-	EG/2021/4770
Wijesinghe D.M.A.B.	-	EG/2021/4875

.....
Dr. Nadeesha Sandamali
(Supervisor)

Abstract

Sri Lanka faces a shortage of qualified teachers, particularly in rural regions, resulting in unequal learning opportunities for General Certificate of Education Ordinary Level (GCE O/L) and Advanced Level (GCE A/L) students. Existing digital learning platforms provide static, prerecorded content and lack adaptive personalization, while general-purpose Large Language Models (LLMs) are not aligned with the Sri Lankan school curriculum. This project presents an Artificial Intelligence (AI)-driven Sinhala teaching assistant that integrates four main components: (i) a curriculum-aligned Sinhala LLM (SinLlama-8B) for educational content generation, (ii) a Hybrid Bayesian Knowledge Tracing (Hybrid-BKT) framework combining Personalized Clustered Bayesian Knowledge Tracing with Long Short-Term Memory (LSTM)-based temporal modeling for learner mastery estimation, (iii) a Retrieval-Augmented Generation (RAG) pipeline using ChromaDB-indexed syllabus content, and (iv) a LangGraph-orchestrated multi-agent architecture for instructional support. SinLlama-8B outperformed Qwen-14B, Llama-3.1-8B, DeepSeek-14B, and Mistral-7B in curriculum alignment and semantic fidelity. The Hybrid-BKT framework continuously estimates learner knowledge states, enabling adaptive content delivery based on proficiency levels. The production system uses a dual-LLM architecture in which the fine-tuned SinLlama-8B model handles curriculum-grounded content generation through the RAG pipeline, while Llama-3.3-70B performs rapid agent orchestration. Experimental results demonstrate the effectiveness of integrating curriculum-aligned LLMs, knowledge tracing, and RAG to support personalized secondary education in Sri Lanka. In addition, an interactive AI Avatar powered by Gemini Text-to-Speech delivers Sinhala lesson explanations, providing students with a more engaging and personalized learning experience.

Acknowledgments

First and foremost, we would like to express our deepest appreciation to our project supervisor, Dr. Nadeesha Sandamali, for her invaluable guidance, continuous support, and immense knowledge. Her insightful feedback and constant encouragement were instrumental in shaping the direction and success of this project.

We would also like to extend our sincere gratitude to the academic and non-academic staff of the Department of Electrical and Information Engineering, Faculty of Engineering, University of Ruhuna. Their assistance and the resources provided by the department played a vital role in completing this project smoothly.

Our heartfelt thanks go out to our families and friends for their unwavering moral support, patience, and understanding throughout this challenging journey.

Finally, as a team, we appreciate the dedication, cooperation, and hard work of each member, which made this collaborative effort a successful and rewarding experience.

Contents

Abstract	ii
Acknowledgements	iii
Contents	vii
List of Figures	ix
List of Tables	x
Acronyms	xii
1 Introduction	xiii
1.1 Background	xiv
1.2 Problem Statement	xiv
1.3 Objectives and Scope	xv
1.3.1 Objectives	xv
1.3.2 Scope	xv
2 Literature Review	2
2.1 Previous Work	2
2.1.1 Intelligent Tutoring Systems (ITS)	2
2.1.2 Adaptive Learning and Student Modeling	3
2.1.3 AI-Based Personalized Learning Systems	5
2.1.4 Educational Teaching Avatars	7
2.1.5 Sinhala Speech Technologies for Educational Systems	8
2.1.6 Generative AI and Large Language Models (LLMs) in Educational Applications	9
2.1.7 Retrieval-Augmented Generation (RAG) for Curriculum Based Learning	10
2.1.8 Multimodal Learning Systems	11
2.1.9 Multi-Agent Systems for Personalized Tutoring	12
2.1.10 Learning Analytics and Teacher Dashboards	13
2.2 Gaps in Literature	15
3 Methodology	17
3.1 Survey Overview and Key Findings	17
3.2 System Overview	19
3.3 System Workflow	20
3.4 Multi-Agent System Design	22

3.4.1	State Management and Orchestration	23
3.4.2	Agents Managed by the Supervisor (LangGraph)	24
3.4.3	Independent Agents (Directly Invoked)	27
3.5	Retrieval-Augmented Generation (RAG) Pipeline	29
3.5.1	Knowledge Base Construction	31
3.5.2	Vector Encoding and Indexing	31
3.5.3	Contextual Retrieval and Content Generation	31
3.5.4	Retrieval Modes	32
3.5.5	Integration with Multimodal Learning System	32
3.6	Real-Time Avatar Speech Delivery	33
3.6.1	Overall Avatar Speech Delivery Architecture	33
3.6.2	AVTR-1 Streaming Avatar	34
3.6.3	WebRTC Communication Process	35
3.6.4	Real-Time Playback Control	36
3.6.5	AVTR-1 Generative Model	37
3.6.6	Current AVTR-1 Integration	38
3.6.7	Full Body Avatar Mode	39
3.6.8	Comparison of Avatar Delivery Modes	40
3.6.9	Deprecated Avatar Pipeline	41
3.7	Teacher Dashboard and Learning Analytics	41
3.8	Evaluation Strategy	43
3.8.1	Learning Gain Analysis	43
3.8.2	Adaptivity and Knowledge Estimation Accuracy	43
3.8.3	Avatar and Speech Interaction Quality	43
3.8.4	User Satisfaction and Usability	44
3.9	Ethical Considerations	44
3.9.1	Data Privacy and Security	44
3.9.2	Fairness and Bias Mitigation	44
3.9.3	Transparency and Explainability	44
3.9.4	AI as an Educational Aid	45
3.10	Personalization and Adaptive Learning	45
3.10.1	System Architecture for Personalized Learning	45
3.10.2	Research Foundation	47
3.10.3	Personalization Mechanisms	48
3.10.4	Adaptive Assessment and Content Delivery	53
3.11	Data Collection	54
3.11.1	National Curriculum Data Collection and OCR	54
3.11.2	Digitization and Analysis of Sinhala Subject Content	54
3.11.3	Student Performance and Interaction Data Collection	55
3.11.4	Creation of Personalized Learning Plans	55
3.12	Frontend Development and Implementation	55
3.12.1	Framework and Approach	55
3.12.2	Design and Prototyping	56
3.12.3	UI Styling and Responsiveness	57
3.12.4	Component Structure and Reusability	57
3.12.5	State Management	58
3.12.6	Backend Communication	58
3.12.7	Testing and Debugging	58

3.13	Engagement Detection System	59
3.13.1	Feature Extraction and Metrics	59
3.13.2	Weighted Fusion Mechanism	60
3.13.3	Engagement Classification	61
3.14	Model Fine-Tuning: Task-Specific Fine-Tuning Iterations	61
3.14.1	Selection of the Base Language Model	61
3.14.2	Base Fine-Tuning for Educational Question Answering	64
3.14.3	Parameter-Efficient Fine-Tuning Using LoRA	66
3.14.4	Training Configuration	68
3.14.5	Selection of Hyperparameters	70
3.14.6	Training Dataset and Evaluation Strategy	71
3.14.7	Need for Task-Specific Fine-Tuning	72
3.14.8	Quiz Generation Fine-Tuning	73
3.14.9	Content Explanation Fine-Tuning	74
3.14.10	Level-Adaptive Content Rewriting	76
3.14.11	Iterative Fine-Tuning Strategy	77
3.14.12	Single Fine-Tuned Model for Multiple Subjects	78
3.14.13	Continuous System Optimization	79
3.15	Digitization and Analysis of Sinhala Subject Content	80
3.15.1	Page Type Detection	82
3.15.2	Font-Aware Text Extraction and Conversion	82
3.15.3	Heading and Topic Detection	82
3.15.4	Thirteen-Stage Text Cleaning Pipeline	83
3.15.5	Human Review and Content Validation	84
3.15.6	Non-Destructive Content Ingestion	84
3.15.7	Contribution of the Developed Pipeline	84
3.15.8	Limitations	85
3.16	System Deployment	85
3.16.1	Cloud-Hosted Components	85
3.16.2	Persistent Storage Strategy	86
3.16.3	Continuous Integration and Deployment	86
3.16.4	GPU-Hosted AI Services	87
3.16.5	Overall Deployment Architecture	88
4	Results	90
4.1	Overview	90
4.2	Model Fine-Tuning	90
4.2.1	ROUGE Score Evaluation	91
4.2.2	BERTScore Evaluation	92
4.2.3	Token Generation Speed Evaluation	93
4.2.4	LLM-as-a-Judge Evaluation	94
4.2.5	Selected Model for the Proposed System	95
4.3	Personalization Performance Evaluation	96
4.4	Engagement Detection System Performance Evaluation	97
4.4.1	YOLOv8 Performance Comparison	97
4.4.2	EmotionCNN Performance	98
4.5	System Implementation Outcomes	99
4.5.1	Retrieval-Augmented Generation (RAG) Pipeline Outcomes	99

4.5.2	Multi-Agent System Orchestration	99
4.5.3	Real-Time Avatar Speech Delivery	99
4.6	User Interfaces in Web Application	100
4.6.1	Public Landing	100
4.6.2	Account Registration and Login	100
4.6.3	Student Interface	103
4.6.4	Teacher Interface	109
4.7	User Interfaces in Mobile App	113
4.8	Achievements	114
5	User Feedback	116
6	Conclusion	119
6.1	Summary of the Project	119
6.2	Novel and Important Aspects of the Methodology	119
6.3	Results and Final Observations	120
6.4	Limitations and Recommendations for Future Work	121
	APPENDIX	122
	Bibliography	122

List of Figures

3.1	Different tuition classes students attempted	18
3.2	Students' status about their personalized learning	18
3.3	Overall System Workflow	21
3.4	Multi-Agent System Architecture	23
3.5	RAG Architecture	30
3.6	System Architecture for Personalized Learning	46
3.7	Personalization Mechanisms in the Proposed System	49
3.8	Bayesian Knowledge Tracing Update Process in the Proposed System	52
3.9	Engagement Detection System Architecture	59
3.10	Model Fine-Tuning Process	62
3.11	Digitization and Analysis Pipeline	81
3.12	System Deployment Architecture	89
4.1	Landing page	100
4.2	Student Login page	101
4.3	Student Signup page	101
4.4	Admin Login page	102
4.5	Admin Signup page	102
4.6	Student Dashboard page	103
4.7	Available Lessons page	104
4.8	Quiz Preparation page	104
4.9	Pre Lesson Quiz page	105
4.10	Student Level Analyzing page	106
4.11	Pre Quiz Result page	106
4.12	Lesson Delivering page	108
4.13	Youtube Feature page	108
4.14	Engagement Detection	109
4.15	Teacher Dashboard page	110
4.16	Lesson Adding page	111
4.17	PDF Ingestion - Select Images	111
4.18	PDF Ingestion - Review Topics	112
4.19	PDF Ingestion - Save and Ingest	112
4.20	PDF Ingestion - Success	113
4.21	Signup page	113
4.22	Login page	113
4.23	Landing page	114
4.24	Student Overall Progress page	114
4.25	A Snapshot of the First Page of the Submitted Paper for ICEEET 2026	115
5.1	Student feedback highlighting accelerated learning and value	117

5.2	Student feedback on the clarity and engaging nature of the AI avatar	117
5.3	Student feedback on the supportive avatar and encouraging quiz experience	118
5.4	Student feedback on personalized difficulty and the human-like teaching experience	118

List of Tables

2.1	Summary of Intelligent Tutoring Systems (ITS) Literature	3
2.2	Summary of Adaptive Learning and Student Modeling	5
2.3	Summary of AI-Based Personalized Learning Systems	6
2.4	Summary of Educational Teaching Avatars in Learning Systems	7
2.5	Summary of Sinhala Speech Technologies for Educational Systems . .	9
2.6	Summary of Generative AI and LLMs in Educational Applications . .	10
2.7	Summary of Retrieval-Augmented Generation (RAG) for Curriculum- Based Learning	11
2.8	Summary of Multimodal Learning Approaches in Education	12
2.9	Multi-Agent Systems for Personalized Tutoring	14
2.10	Learning Analytics and Teacher Dashboard Support in Adaptive Learn- ing Systems	15
3.1	Comparison of Avatar Delivery Modes	40
3.2	Mastery levels based on hybrid P(Know) score	53
4.1	Tested Large Language Models	91
4.2	ROUGE Score Comparison	92
4.3	BERTScore Comparison	93
4.4	Token Generation Speed Comparison	94
4.5	LLM-as-a-Judge Evaluation	95
4.6	Performance Comparison of BKT Variants	96
4.7	YOLOv8 Performance Comparison	98
4.8	EmotionCNN Performance Evaluation	98

Acronyms

AI Artificial Intelligence

API Application Programming Interface

ASR Automatic Speech Recognition

AUC Area Under the Curve

AWS Amazon Web Services

BERT Bidirectional Encoder Representations from Transformers

CFA Correct First Attempt

CPU Central Processing Unit

FSM Finite State Machine

G.C.E A/L General Certificate of Education Advanced Level

G.C.E O/L General Certificate of Education Ordinary Level

GPT Generative Pre-trained Transformer

GPU Graphics Processing Unit

HMM Hidden Markov Model

Hybrid-BKT Hybrid Bayesian Knowledge Tracing

ICEEET International Conference on Electrical Engineering and Emerging Technologies

ICIS International Conference on Information Systems

ICT Information and Communication Technology

IEEE Institute of Electrical and Electronics Engineers

ITS Intelligent Tutoring Systems

IVR Interactive Voice Response

LAD Learning Analytics Dashboard

LLM Large Language Model

LSTM Long Short-Term Memory

MAS Multi-Agent System

ML Machine Learning

NLP Natural Language Processing

OCR Optical Character Recognition

OIDC OpenID Connect

PC-BKT Personalized Clustered BKT

PEFT Parameter-Efficient Fine-Tuning

PII Personally Identifiable Information

RAG Retrieval-Augmented Generation

RAM Random Access Memory

RBAC Role-Based Access Control

RL Reinforcement Learning

RMSE Root Mean Square Error

SDP Session Description Protocol

SER Word Error Rate

STT Speech-to-Text

T5 Text-to-Text Transfer Transformer

TTS Text-to-Speech

WER Word Error Rate

Chapter 1

Introduction

One of the best investments made in the use of human resources is in the area of education. However, students belonging to secondary-level educational institutions, those who appear for General Certificate of Education Advanced Level (G.C.E.O/L) and General Certificate of Education Ordinary Level (G.C.E O/L), face grave challenges. One of the major problems they face is the lack of over 30,000 teachers in the country. This has led to a condition where a single teacher is left with the task of handling over 50 students. This problem is not only a grave concern but also worsens the scenario in rural areas. Not only is the quality impacted, but it also worsens the difference between the educational standards in rural areas compared to those in urban areas [1]. These influences have both stern and dire implications. Just 63.3% of O/L candidates manage to meet the required marks to qualify for A/L studies, thereby blocking more than 130,000 candidates every year from pursuing tertiary educations. Not only that, but they are also faced with a teaching mechanism that adopts a "one size fits all" policy, incapable of supporting varying levels of learning speed and understanding that will automatically be present within a typical classroom setting. Finally, in the tertiary educational phase, the situation becomes more acute as only 15% of candidates who qualify for A/L qualify for admission into universities. These figures, in effect, reveal that G.C.E. O/L and A/L are more than your average examinations in the Sri Lankan educational circuit, as they play a very crucial role in determining the fate of the candidates appearing for them [2].

However, to achieve in this competitive setting, students cannot be employed arbitrarily in tutoring activities. This is because they require learning processes characterized by strict obedience to the national curriculum to remove their misconceptions instantly. Although general applications of Artificial Intelligence(AI) are being improved to support the Sinhala language, the applications were not tailored to the Sri Lankan curriculum. This makes them incapable of preparing students for these exams [2, 3]. What if there were an intelligent system which could develop better learning for the students in a more meaningful way? What if there were a system which would harness the strengths of artificial intelligence and the strict adherence to the national syllabus, engaging with the students repeatedly and adapting itself to their needs? This would be able to teach the basic subjects in Sinhala, going through the national syllabus bit by bit, while interacting with the students by answering their queries. This would also be able to identify mistakes in learning in real-time, while adapting itself to the learning ability of each student, according to its performance history [4].

1.1 Background

There is a growing divide between what presently exists in digital solutions in Sri Lanka and what actually is needed to succeed in O/L and A/L examinations. While the online materials on platforms like e-Thaksalawa and DP Education are invaluable, they tend towards static content delivery on a subject-wise basis. Fully interactive AI-based tutoring systems that can effectively teach more than one subject in Sinhala are clearly absent for O/L and A/L learners[5, 6]. However, at the same time, large Large Language Models(LLMs) are getting better at learning the Sinhala language, and recent research like SinLlama has shown that LLMs can, in fact, learn to read and write the Sinhala language with accuracy. Yet, it is to be noted that LLMs are mostly trained through large web and text datasets and aren't at all structured around the school syllabus of the national schools of Sri Lanka. Nonetheless, as of the current implementation, LLMs are largely unaffiliated with the environment of the class or the learning outcomes of the respective subjects of either the O/L or the A/L level of schooling in the respective fields of study offered by the schools of the concerned regions of the mentioned country. They can, in fact, write explanations or instances in the Sinhala language but aren't at all oriented around the learning outcomes of the respective school grades and the learning outcomes of[7, 8].

The use of artificial intelligence in the educational process has great potential in changing the learning process experience dramatically when implemented along with traditional teaching methods[8]. This project presents an intelligent learning system which makes use of multi-agent artificial intelligence models in designing complicated learning strategies. Such strategies include learning skill evaluation, interaction analysis, and real-time feedback analysis that are quite crucial for personalized learning experiences. The learning system has been designed in such a way that it adheres very closely to the curriculum guidelines of the country and accordingly has been designed such that it follows all required educational guidelines while fitting the learning requirements of every individual child uniquely. This project makes use of recent developments in the area of intelligent tutoring systems and educational dashboards designed through multi-agent learning models specifically tailored for the exclusive situation of learning the Sinhala language.

1.2 Problem Statement

The education system in Sri Lanka also depends to a large extent on tuition classes conducted on a mass scale to prepare A/L and O/L education, in which a teacher has to deal with a very large number of pupils, leaving them no scope to provide proper assistance or remedial classes to a student facing a particular topic once the whole class moves on to the next lesson [9]. In such a context, when a student is struggling with a particular subject, he or she is left behind as a whole class moves to their next lesson [9]. In contrast, Sinhala medium students have a major drawback in the context of digital technology. As most of the current developing and available technologies in the world, like LLMs, are designed and generated from global and English-dominant sources, there is a major drawback in that context, as most of the technological applications developed in the world, including LLMs, fail to contain the needed context of Sri Lankan education, in such a context, when such applications like LLMs are used to prepare A/L or O/L studies, most of them in turn generate

answers in a completely generic manner[10, 11].

This gives rise to a pressing requirement for an AI system that must extend beyond standard proficiency with the Sinhala language and comprehensively educate school subjects with an emphasis on lacking knowledge of the Sinhala language and strict adherence to the Sri Lankan school curriculum. Additionally, there is a need for an AI teaching system for school students [12]. An effective mechanism to satisfy both the stationary knowledge of school subjects and the lack of qualified educators is an interactive teaching avatar that is able to act like an ordinary school teacher during the everyday educational process. An AI teaching avatar with the ability to educate school subjects with the Sinhala Language and responsive to the student's knowledge proficiency with information on student performance for teachers would play an important role and is proposed as the project objective.

1.3 Objectives and Scope

1.3.1 Objectives

- To develop an AI-powered Sinhala teaching avatar that delivers curriculum-aligned lessons and explanations using multimodal interactions, including speech, visuals, animations, and text.
- To analyze individual student performance, learning pace and interaction patterns in order to generate personalized learning paths and dynamically adapt lesson difficulty and content delivery.
- To incorporate assessment-based evaluations and interaction analysis for continuous evaluation of student understanding, enabling real-time feedback.
- To allow students to interact naturally with the teaching avatar by asking questions in Sinhala and receiving accurate, context-aware spoken explanations through AI-driven dialogue mechanisms.
- To implement a retrieval-augmented generation (RAG) framework that ensures the teaching avatar delivers accurate, up-to-date, and syllabus-based content aligned with the latest G.C.E. O/L and G.C.E. A/L national curriculum.
- To provide teachers with learning analytics and progress insights through a monitoring dashboard.

1.3.2 Scope

This project focuses on the following areas:

- The system will support selected subjects and lessons from the G.C.E. O/L and G.C.E. A/L Sinhala-medium national curriculum to demonstrate curriculum-aligned adaptive learning.
- The project will implement personalization and adaptivity by adjusting lesson content and difficulty based on student quiz performance, and mastery levels (Beginner, Intermediate, and Advanced).

- The project will include an AI teaching avatar that delivers lessons and explanations in the Sinhala language using multimedia elements such as text, speech, images, visuals, animations, and summarized text notes.
- The system will conduct assessments and track learning progress, providing performance insights and engagement analytics through a teacher dashboard.
- The solution will be developed as a web-based application and evaluated using predefined metrics such as learning improvement, user feedback, and system usability.

Chapter 2

Literature Review

2.1 Previous Work

2.1.1 Intelligent Tutoring Systems (ITS)

Intelligent Tutoring Systems (ITS) are an important focus of contemporary educational technology and have received considerable attention from the educational technology research community. Recent studies indicate that there is an increasing tendency for researchers to utilize artificial intelligence techniques in the modelling of student behavior, to study the analysis of the patterns of student learning, and to dynamically change and adjust instructional content based on a student's performance. See [13],[14].

In this study, an AI-driven framework was developed to create an individualized instructional experience using real-time learner data. The Adaptive Learning Framework uses a set of machine learning techniques including Predictive Modelling and Learning Analytics to assess a student's progress and modify the sequencing and difficulty of instructional material based on a learner's current performance. The Adaptive Learning Framework is continuously updated based on the student's interaction data with a tutor/teacher, the results of assessments, and learning histories to maintain an updated learner model for continued engagement with instructional material. Experimental results from this study demonstrate that learners were more engaged, had a higher assessment score, and retained more knowledge than learners exposed to non-adaptive instructional systems (e.g., a non-adaptive learning system). See [13].

Another study details the development of an intelligent tutoring model that is focused on providing learners with interactive learning opportunities and instant automated feedback [14]. This system provides the ability to analyze student responses and give student course feedback by utilizing supervised machine learning techniques, rule-based diagnostic models, and natural language processing techniques. This system uses adaptive testing methods to assess student learning needs and provide instruction targeted toward these gaps in understanding. The results of this evaluation show that intelligent tutoring systems using adaptive assessment methods and intelligent feedback mechanisms produced significantly higher levels of student mastery of course concepts, decreased cognitive load, and increased student satisfaction [14].

Technologies utilized in these studies include educational data mining, machine learning algorithms, student modeling procedures, and adaptive content delivery systems. These technologies allow intelligent tutoring systems to go from the traditional one-time delivery of material to an ongoing, learner-focused model of education. In

each of the studies, it has been shown that using adaptive methods with intelligent tutoring systems will lead to improved student learning outcomes by personalizing the teaching methodology to each student’s specific needs and providing them with timely and personalized feedback [13], [14]. The literature reviewed shows strong evidence indicating clear advantages of using AI-assisted Intelligent Tutoring Systems. I.e., They improve efficiency of learning, improve engagement and improve performance. However, most current implementations have been designed to serve the needs of learners who communicate in English through text input. Therefore, there remains a need for additional research on creating Intelligent Tutoring Systems capable of supporting multiple languages through speech input and delivering content that aligns with local curricula. In particular, this is an area where significant research needs exist due to the limited availability of resources available to many approximate low-resource learners.

Table 2.1: Summary of Intelligent Tutoring Systems (ITS) Literature

Ref.	Technologies Used	AI Models / Techniques	Methodology	Key Results	Limitations
[13]	Adaptive learning platform, learning analytics, educational data mining	Machine learning (predictive modeling, clustering)	Analysis of student interaction data and assessment results to personalize learning paths	Improved engagement, higher assessment scores, better knowledge retention	English-language focus; no speech or multimodal interaction; limited localization
[14]	Intelligent tutoring framework, adaptive testing, interactive feedback system	Supervised ML, rule-based reasoning, Bayesian knowledge tracing, NLP	Analysis of learner responses to generate adaptive feedback and targeted remediation	Increased concept mastery, reduced cognitive load, higher learner satisfaction	Text-based interaction only; no low-resource language support; no avatar-based tutoring

2.1.2 Adaptive Learning and Student Modeling

Adaptive learning systems have been designed to offer personalized learning opportunities to students through continuous modification of teaching materials according to individual student attributes. One of the basic elements of adaptive learning systems has been student modeling via personalization, which has been described as the process of capturing a student’s knowledge level and learning style through data-driven methodologies. Besides, this process has been shown to help adaptive learning systems to offer teaching according to a student’s understanding level.

However, some current studies are also pointing towards the value of learning analytics and student modeling using artificial intelligence in achieving adaptation. In a study conducted by Wang et al.[15], adaptive learning analytics strategies have been proposed. They suggest that student interaction data such as quiz scores, response time, and learning behavior could be used to make predictions regarding learner

mastery. They have been able to prove that a blend of statistical analysis and machine learning methods could improve learner state prediction accuracy, achieving real-time adaptation of lesson difficulties.

Another important aspect of adaptive learning systems is found in the realm of user-centered personalization, particularly targeting people with varying levels of learner needs. In this vein, Johnson and Lee discuss the application of AI-powered adaptive learning systems that can benefit students with special learning needs via personalized digital offerings. They emphasize that effective models of learning should take into account not only the academic performance of the student but also their preference, accessibility, and engagement criteria altogether. This is because adaptive learning systems result in greater success and success rates being associated with personalized complete student profiles rather than merely test results.

[16]

Aside from approaches grounded in analytics, reinforcement learning (RL) also provides an advanced method for adaptive instruction and student modeling. Kim et al. [17] offer a framework based upon a reinforcement learning process whereby the learning system interprets lesson sequencing as a decision-making process in a problem case. When selecting what instructional actions to take with students, the adaptive agent evaluates the response of the student due to the learning action taken, then evaluates the student's feedback based on the student's gain from learning through that specific course. RL-based adaptive systems have demonstrated through experimental data the ability to outperform traditional rule-based methods by providing a more rapid path to mastery of concepts within the context of teaching and learning.

The body of research available shows that adaptive learning has become dependent on maintaining a current, highly accurate information database about the students to personalize the instructional experience effectively. Learning analytics, machine learning and reinforcement learning will assist adaptive learning systems to assess the learner's areas of strength and weakness through their quiz results and the interaction data collected. The vast majority of existing adaptive learning research has been conducted in environments that focus on English-language instruction, and there is very limited research supporting speech-based interaction and local curricula.

Table 2.2: Summary of Adaptive Learning and Student Modeling

Aspect / Focus	Methods / Techniques	Key Findings / Outcomes	Reference
Student Modeling	Representing learner knowledge, skills, pace, and preferences using data-driven techniques	Enables targeted instruction matching learner's current understanding; improves learning efficiency	[15]
Learning Analytics	Collecting interaction data (quiz scores, response times, behavior); statistical analysis + machine learning	Accurate prediction of learner state; allows real-time adaptation of lesson difficulty; improves engagement	[15]
User-Centric Personalization	AI-driven adaptation considering preferences, accessibility, and engagement	Tailors content delivery for diverse learners including those with special needs; improves educational outcomes	[16]
Reinforcement Learning (RL)	Lesson sequencing as decision-making; adaptive agent selects instructional actions based on feedback	RL-based adaptive systems accelerate mastery achievement and outperform rule-based systems	[17]
Continuous Student Model Updates	Real-time analysis of learner interactions, quiz results, performance history	Helps identify strengths and weaknesses; ensures effective personalization of instruction	[15]-[17]

2.1.3 AI-Based Personalized Learning Systems

AI-Based Personalized Learning Technologies create customized instructional content, assessments, feedback, and learning paths for every learner. Unlike traditional one-size-fits-all e-learning solutions, AI-Based Personalized Learning Technologies design instructional content, assessments, feedback, and learning paths based on analysis of actual student data (i.e., quiz answers, study habits, performance history), while also dynamically adjusting the instructional material as necessary based on current student performance. The primary objective of these AI-Based Personalized Learning Systems is to provide maximum learning efficiency and flexibility to meet diverse learner needs, pacing, and preferences.

Advancements in this field show increasing interest in the ability of AI to drive personalized learning through data-driven approaches. Research by [1] on AI motivated adaptive personalization for educational platforms demonstrates using machine learning and predictive analytics to create a customized range of learning content at scale. Using personalized recommendation algorithms, the researchers assign learning resources based on a learner's prior performance, as well as preferences, and found that students who were provided AI-personalized content had higher levels of engagement and improved topic mastery compared with those in control groups that received learning resources through non-adaptive systems.

Deep learning will be another area of directed research that will utilize user modeling to create a highly personalized experience for learners. In [18], they describe systems that combine deep neural networks with a cognitive profile of each student and utilize AI for predicting student learning needs, in addition to providing the ability

to dynamically change the sequence of the lessons that students see in real-time. By creating a model that understands the user's unique learning behavior, the system can make recommendations on not only what students should learn next, but also how to present the information using a format that best fits the student's individual learning style, such as video versus text versus interactive exercises. Research indicates that using multimodal (visual, auditory, and kinesthetic) AI personalization significantly enhances knowledge and increases knowledge acquisition speed by reducing the amount of time needed to learn new information.

In addition to support for content recommendations and adaptive sequencing AI systems also provide automated forms of feedback and error analysis on student responses. According to [19] AI algorithms are able to analyse the responses of students as they occur and provide corrective feedback to learners that is personalised to those learners' needs. The findings of their study provide evidence that systems which generate detailed information and provide recommendations based on the interaction data of students do improve learner self-efficacy and accuracy. The researchers further note that AI personalisation fosters increased motivation and self-directed learning through self-paced progress.

The available studies reveal that AI-enhanced learning using Machine Learning, Deep Learning, Cognitive modelling, etc., facilitates greater learning by continually adapting and personalising content and feedback for each learner based on their individual needs. Such systems provide not only predictions of what each learner will require next but also, in most instances, instructional strategies tailored to individual learners' unique strengths and weaknesses. While many recent studies focus on learning platforms that are primarily English-based as well as traditional text/audio interaction, there has been much less research on how these technologies can be effectively applied across different languages.

Table 2.3: Summary of AI-Based Personalized Learning Systems

Aspect / Focus	Methods / Techniques	Key Findings / Outcomes	Reference
Personalization	Analysis of quiz responses, study patterns, performance history	Tailors content, feedback, and learning paths to individual learners; improves learning efficiency	[20]
Adaptive Recommendation	Machine learning, predictive analytics	Assigns learning resources based on past performance and preferences; higher engagement and better topic mastery	[20]
Deep Learning & User Modeling	Deep neural networks, AI-based cognitive profiling	Predicts learning requirements; recommends what and how to study (video, text, interactive exercises); improves retention and reduces learning time	[18]
Automated Feedback & Error Analysis	AI algorithms for real-time response evaluation and explanation generation	Provides corrective feedback targeting misconceptions; improves learner confidence, accuracy, motivation, and self-paced progression	[19]
Multimodal Personalization	Combining text, video, and interactive exercises	Supports different learning styles; enhances knowledge retention	[18]

2.1.4 Educational Teaching Avatars

Teaching avatars are a sort of virtual agent that uses speech, graphics, gestures, and interactive dialogues to simulate teaching behaviors in the context of learning. These avatars increasingly cooperate with AI to enable personalized explanations, adaptive feedback, and enjoyable learning. The key functions of teaching avatars are to improve motivation, understanding, and interaction by students compared to prior, classic text-based e-learning systems.

Current studies also focus on the growing use of artificial intelligence avatars in modern education. In a study offered in [21], the development and use of intelligent educational avatars for teaching that use artificial intelligence, multimodal interaction, and adaptive instruction strategies have been discussed. According to the study, intelligent educational avatars would use verbal communication, facial expressions, animations, and visual communication to teach students. This study also stated that avatar-mediated learning is very effective in self-paced learning environments, as well as for students who belong to the younger generation.

Moreover, the article goes into the depth of integrating the concept of AI-based personalisation in the teaching avatars. According to the study, the interaction of the learners is taken into account, and the information regarding the quiz performance, the manner of answering, as well as the interaction level, is always tracked to adapt the learning process. The avatar accordingly changes the level of explanation, the rate of instruction, and the manner of feedback based upon the learning level of the learner. According to those researchers, adaptive learning avatars lead to increased learner satisfaction and recall as compared to other digital learning tools.[21].

This study also gives valuable insight into how multimodal communication enhances teaching with avatars. The combination of spoken word with visual and textual information allows for a deeper understanding of concepts through increased comprehension. The avatars are not only a means of delivering the lesson but can function like a tutor by responding to learner questions and providing further clarification. Learners interact with the avatar much the same as when interacting with any other person. Having a human-like interaction means that the anxiety levels of the learners are low, as well as feeling confident in their ability to learn.

Table 2.4: Summary of Educational Teaching Avatars in Learning Systems

Focus / Component	Key Features / Methods	Benefits / Advantages	Limitations / Challenges	Reference
AI-powered Teaching Avatars	Virtual agents delivering lessons via speech, visuals, gestures, and animations	Enhances learner engagement, comprehension, and motivation	Requires high-quality language resources, emotion modeling	[21]
Adaptive Personalization	Learner data analysis (quiz performance, responses, engagement) to adapt lesson delivery	Adjusts explanation depth, teaching speed, and feedback style; improves knowledge retention	Needs robust data collection and analysis	[21]
Multimodal Communication	Combines spoken explanations, visual animations, and text summaries	Improves conceptual understanding, reduces anxiety, increases confidence	Mostly developed for English; adaptation to local languages needed	[21]

2.1.5 Sinhala Speech Technologies for Educational Systems

Speech technologies such as Text-to-Speech (TTS) and Automatic Speech Recognition (ASR) play a critical role in modern educational systems by enabling voice-based content delivery and interactive learning. These technologies are especially important for Sinhala-medium education, as Sinhala is considered a low-resource language with limited availability of large-scale digital speech datasets. Several studies have focused on developing Sinhala TTS systems. Nanayakkara et al. proposed a Sinhala TTS system using the MaryTTS framework with a unit selection synthesis approach [22]. Their system was trained using professionally recorded Sinhala speech and evaluated for intelligibility and naturalness. The results showed approximately 70% intelligibility and 60% overall speech quality, demonstrating that Sinhala TTS systems can effectively generate understandable speech. However, it also shed light on some of the issues such as limited pronunciation, unsatisfactory handling of prosody, and naturalness issues between machine-generated pronunciation and human pronunciation that might affect learner engagement.

Earlier work by Weerasinghe et al. introduced Festival-SI, one of the first Sinhala TTS systems, primarily designed to assist visually impaired users [23]. Although the system was effective for basic text reading tasks, it lacked adaptive capabilities and was not designed for interactive or personalized learning environments. Such early systems were largely static and focused on accessibility rather than pedagogical support.

In addition to speech synthesis, Sinhala Automatic Speech Recognition (ASR) has been explored for interactive systems. Manamperi et al. developed a Sinhala ASR system for Interactive Voice Response (IVR) applications using Hidden Markov Models (HMMs) [24]. Their system achieved a Word Error Rate (WER) of 11.2% for digit recognition and a Sentence Error Rate (SER) of 5.7% for recognizing Sinhala song names under controlled conditions. These results demonstrate that Sinhala speech recognition is feasible and can achieve reasonable accuracy when trained with appropriate acoustic models and phoneme dictionaries.

Despite these advancements, the ASR system proposed by Manamperi et al. was limited to small vocabularies and task-specific domains [24]. Performance significantly degraded in noisy environments and real-world usage scenarios, which are common in educational contexts such as classrooms and homes. Furthermore, the system did not support natural language questioning or open-ended student interaction.

Overall, existing Sinhala speech technology research mainly focuses on accessibility tools such as screen readers and IVR systems rather than educational intelligence [22–24]. Most systems are standalone, lack integration with adaptive learning models, and do not support curriculum-aligned educational dialogue. Additionally, emotional expression, pronunciation variation, and child-oriented speech remain underexplored.

Therefore, there is a clear research gap in developing a Sinhala speech-enabled educational system that integrates ASR, TTS, and AI-based personalization. This project addresses this gap by incorporating Sinhala speech technologies into an AI-powered teaching avatar capable of delivering curriculum-aligned lessons, responding to student questions, and supporting adaptive learning for A/L and O/L students.

Table 2.5: Summary of Sinhala Speech Technologies for Educational Systems

AI Models	Accuracy / Performance	Techniques / Approach	Method / Application	Reference
MaryTTS-based Sinhala TTS	70% intelligibility, 60% speech quality	MaryTTS framework, professional recordings	Speech synthesis for curriculum content delivery	[22]
Festival-SI Sinhala TTS	Basic text reading; non-adaptive	Festival TTS system	Assist visually impaired users, static reading tasks	[23]
HMM-based Sinhala ASR	WER: 11.2% (digits), SER: 5.7% (song names)	Hidden Markov Models (HMMs)	Interactive Voice Response (IVR), limited vocabulary recognition	[24]

2.1.6 Generative AI and Large Language Models (LLMs) in Educational Applications

Generative AI & Large Language Models (LLMs) such as Generative Pre-trained Models Transformer (GPT), Bidirectional Encoder Representations from Transformers (BERTs), Text-to-Text Transfer Transformer (T5) models have recently garnered significant attention in educational applications. These models are capable of understanding natural language, generating explanations, and answering questions, making them suitable for intelligent tutoring systems and educational chatbots.

A widely cited systematic review by Zawacki-Richter et al. examined artificial intelligence applications in education and identified personalized learning, adaptive feedback, and intelligent tutoring as key benefits of AI-based systems [25]. It is further emphasized by the report that a large number of AI-enabled learning systems tend to be experimental in nature with inadequate linkage with a curriculum.

Kasneci et al. examined in particular the employment of ChatGPT and large-scale language models in education [26]. The results clearly demonstrate that LLMs can be employed in a supportive role for students in terms of explaining ideas, responding to queries, and supplementing the education process in other ways. Yet at the same time, they identify some critical issues like a tendency to give wrong responses, non-transparent decision-making processes, and ethical implications.

Holmes et al. discussed AI-driven personalized learning and emphasized that generative AI systems can adapt learning materials based on student progress, learning pace, and engagement levels [27]. Despite these advantages, the authors highlight challenges such as language bias, data privacy risks, and unequal access to AI technologies, which may increase educational inequality.

Overall, existing research confirms that LLMs have strong potential to improve learning experiences through personalization and interaction. However, most current systems are English-focused, not curriculum-aligned, and not designed for local education systems. This project addresses these gaps by developing a Sinhala-language, curriculum-aligned AI teaching avatar specifically designed for A/L and O/L students

in Sri Lanka.

Table 2.6: Summary of Generative AI and LLMs in Educational Applications

Model / AI System	Application	Advantages	Limitations	Reference
GPT, BERT, T5	Intelligent tutoring, chatbots, explanation generation	Understands natural language, generates explanations, answers questions	English-focused, not curriculum-aligned, may hallucinate	[25]
ChatGPT	Student support: concept explanation, Q&A, learning assistance	Provides adaptive explanations, interactive support	Incorrect answers, ethical concerns, transparency issues	[26]
AI-driven Personalized Learning	Adaptive learning based on student progress and engagement	Material adapts to learning pace and engagement, personalization	Language bias, data privacy risks, unequal access	[27]

2.1.7 Retrieval-Augmented Generation (RAG) for Curriculum Based Learning

Retrieval-Augmented Generation (RAG) is a method in artificial intelligence that uses information retrieval and large language models in order to generate correct and contextually informed responses. Unlike traditional language models that exploit the knowledge embedded in them, in RAG, the model looks for the required information in external sources such as documents, textbooks, or databases, and then uses that information to generate a response. The application of RAG in educational settings, where curriculum alignment, accuracy, and factual correctness are paramount, can be very useful.

Recently, research has also been unveiled to prove the rising use of RAG in the education sector. [28] have performed a detailed survey study on RAG-powered chatbots used in the education sector. Their result includes proving how RAG reduces hallucination in answers given by AI to questions. According to the study, RAG-powered chatbots used in education offer skill-related answers to questions since they relate to topic studies, hence their use in learning environments such as schools. Another improvement made possible through RAG technology is the boost in trust among students because their queries are responded to using reliable sources for education.

The next major contribution is presented in reference [29], where the use of machine assistant based on RAG is advocated in order to enhance learning among students in particular and individuals in general for the efficient retrieval of correct knowledge facilitated by such assistive systems. The study clearly proves that employing a RAG system in particular is beneficial in learning content retrieval pertinent to the question asked before composing an answer, and hence, it is possible for the computer-based machine assistant to provide explanations in accordance with subjects covered during learning sessions. The experimental results clearly indicate that machine assistant-based RAG results in understanding subjects clearly and reducing confusion among learners in comparison to machine assistant-based pure generative models.

[30] commence a thorough review of algorithms associated with the retrieval-augmented generation in educational contexts. The paper proposes the categories of

RAG systems based upon the type of learning processes and data sources available for the construction of the models. The article explains the need for RAG in curriculum-based learning, where it helps the learning models of artificial intelligence to use the textbooks, slides, and test papers while answering questions they are given. The paper further explains the role of RAG in increasing the explainability of learning models of artificial intelligence, which helps the students test their answers through the learning materials retrieved by them.

Table 2.7: Summary of Retrieval-Augmented Generation (RAG) for Curriculum-Based Learning

Focus / Study Area	Key Idea	Educational Benefits	Key Findings	Reference
RAG-based Educational Chatbots	Combines information retrieval with LLMs to generate grounded responses	Reduces hallucinations, improves syllabus alignment	Answers are based on trusted learning materials, increasing learner trust	[28]
RAG-based Learning Assistants	Retrieves instructional content before response generation	Improves understanding and reduces student confusion	Students showed better comprehension compared to generative-only systems	[29]
RAG Techniques for Education	Categorizes RAG by retrieval methods and knowledge sources	Enhances transparency and explainability	Allows answers to be traced back to textbooks and lecture notes	[30]

2.1.8 Multimodal Learning Systems

Multimodal learning systems current educational content using more than one mode, such as text, images, audio, video, animations, and speech. The main idea behind multimodal learning is that students understand and remember lessons better when information is delivered through multiple sensory channels instead of using text alone. Because of this, multimodal approaches are widely used in e-learning platforms and modern educational systems.

Multimodal teaching and learning combine different communication modes such as written text, pictures, diagrams, sound, and visual symbols to support learning [31]. This study highlights that multimodal teaching helps students better understand complex ideas by linking abstract concepts with visual and auditory explanations. According to the authors, using multiple modes increases student attention and participation in the learning process, especially in classroom and digital learning environments.

Multimodal learning systems can exploit various types of data available for the learner, such as interaction and performance, in adapting content depending on the learner's need [32]. Apart from theoretical analysis, this was also an implementation phase for Multimodal Intelligent Adaptive Learning System. This helps in enhancing learner engagement by basing teaching approaches on the learner's response. However, there are various challenges that the study discusses, especially in terms of system complexity and merging different sources into one single adaptive platform for enhanced learner performance. [32].

From a technological perspective, [33] discusses the techniques, applications, and future work in multimodal learning. The researchers clarified that multimodal learning systems integrate knowledge obtained from text, audio, and images in order to enhance understanding and decision-making. The survey indicated that by employing artificial intelligence, multimodal learning approaches enhance the efficiency of learning feedback, interaction, and personalization. However, as indicated by the survey, some drawbacks include the elevated processing cost, asynchrony of modalities, and lack of real-world applications in educational settings. Even though these papers indicated that multimodal learning systems are accurate, some gaps remain. Most multimodal learning systems were built for English medium educational settings and are resource-rich. There has been limited work done in multi-modal learning systems that target local languages or national educational curriculum settings. Additionally, most solutions were built for laboratory or experimental settings, as opposed to real classroom settings [33].

In summary, existing research confirms that multimodal learning systems improve student engagement, understanding, and learning effectiveness. However, there is a clear research gap in developing Sinhala-based, curriculum-aligned multimodal educational systems. This project aims to address this gap by developing a multimodal learning system that supports Sinhala-medium education and aligns with the Sri Lankan school curriculum.

Table 2.8: Summary of Multimodal Learning Approaches in Education

Type / Focus	Key Features / Methods	Advantages	Limitations	Reference
Multimodal teaching theory	Combines text, images, diagrams, sound, visual symbols	Improves understanding of complex ideas, increases attention and participation	Mostly theoretical, not implemented in real classrooms	[31]
Adaptive multi-modal learning system	Uses learner interaction and performance to adapt content	Improves engagement and personalized learning	System complexity, integrating multiple data sources is challenging	[32]
AI-supported multi-modal learning	Integrates text, audio, visual data with AI for personalization	Enhances learning feedback, interaction quality, personalization	High computational cost, synchronization of modalities, limited real-world deployment	[33]

2.1.9 Multi-Agent Systems for Personalized Tutoring

Although traditional Intelligent Tutoring Systems (ITS) were initially based on basic rule-based logic, current research and development indicate that in intricate and difficult learning tasks it may be better to have complementary models using different AI approaches. An important breakthrough in this domain has been offered by IntelliCode, which is a multi-agent learning environment released toward the end of 2025 [34]. In contrast to conventional chatting assistants commonly used for learning interfaces but normally plagued by transitory memory loss issues, IntelliCode has been designed to have a learner state protected by a StateGraph Orchestrator featuring a continuous and versioned student progress flow. This is an important requirement for a Sinhala A/L learning assistant.

Unlike IntelliCode, which adopts a general-purpose model, this design utilizes an intricate structure composed of six specialized agents; consequently, it is very relevant

to this project. Skill Assessment Agent, Learner Profile, Pedagogical Feedback Agent, Engagement Orchestrator; agents such as these are used [34]. The Engagement Orchestrator is particularly relevant to this project aimed at determining inattentive students using computer vision methods by adapting teaching according to it. The IntelliCode system additionally adopts a policy titled Content Curator adhering to 40/50/10, whereby 40% is for improvement, 50% for teaching novel concepts, while 10% is for problematic material. The same applies to Sinhala O/L math/science to maintain optimal cognitive overload [34].

In order to create an AI assistant for interacting rather than for informational content, it is necessary to look into what dynamics are involved when educational AI assistants make their decisions. One of the most relevant studies conducted for this purpose and presented at the International Conference on Information Systems (ICIS) 2024, and titled Dynamic Tutor-Tutee Pairing System, offers a relevant framework for this purpose [35]. Despite this research focusing on peer-to-peer matching in virtual learning environments, the mechanisms for behavioral modeling/sequencing of exercises are straightaway applicable to this research on Sinhala AI teaching avatars. The research brings forth Learner Agents simulated by Finite State Machines for smooth transitions between interaction states [35]. The mechanism of Finite State Machines is apt for this research on the monitoring of attention. For instance, an AI teaching avatar can perform in multiple states.

Even though oriented for peer-to-peer matching in virtual learning environments, the above study has very clear mechanisms regarding behavioral modeling and exercise sequencing, which are applicable in this proposal for the development of Sinhala AI Teaching Avatar systems. The proposal introduces Finite State Machine-based Learner Agents.

Orchestration of Specialized Agents in Mathematics Education A general-purpose chatbot may respond appropriately to queries but may fail to provide the concrete educational strategies a student needs. To fill that gap, [36] introduced a Multi-Agent Mathematics Learning Framework that utilized GPT-4o and the AutoGen library. This work is significant to the proposed project as it explains how the diverse nature of a classroom teacher can be articulated as multiple specialized sub-agents [36]. This design pools varied tasks among specific agents such as the Teacher Agent, Tutor Agent, Solution Verifier Agent, Level Adapter Agent, Motivator Agent [36].

The Level Adapter and Motivator Agent understands and promotes its goals in relation to personalized education and engagement of students in great alignment with the proposed Sinhala teaching avatar. The results revealed that a state-driven approach in a structured manner is more effective in addressing curriculum-based education. The proposed approach of directing Sinhala O/L and A/L students through a national structured syllabus along with AI-driven personalization is relevant in this context.

With the implementation of the same approach of multi-agent orchestration, the suggested system is able to support curriculum consistency while ensuring flexible and engaging personalized learning.

2.1.10 Learning Analytics and Teacher Dashboards

Although the proposed project focuses on an AI-based student-facing learning interface, the literature recognizes that the human teacher's role continues to play a vital part

Table 2.9: Multi-Agent Systems for Personalized Tutoring

Focus Area	Multi-Agent Components / Methods	Key Contributions	Relevance to Proposed System	Ref.
IntelliCode Tutoring System	StateGraph Orchestrator, Skill Assessment Agent, Learner Profiler (BKT), Pedagogical Feedback Agent, Engagement Orchestrator, Content Curator	Centralized learner memory, adaptive feedback, engagement-aware tutoring	Enables long-term tracking of Sinhala A/L and O/L students and adaptive lesson planning	[34]
Behavioral Modeling of Agents	Learner Agents using Finite State Machines (FSM), utility-based sequencing	Structured interaction states and behavior-driven transitions	Supports attention-aware state changes such as lecturing, waiting, and re-engagement	[35]
Multi-Agent Mathematics Learning Framework	Teacher, Tutor, Solution Verifier, Level Adapter, Motivator agents (GPT-4o, AutoGen)	Decomposition of teaching roles, structured curriculum delivery	Supports personalized difficulty adjustment and student motivation in Sinhala education	[36]

in adaptive learning environments. Fully automated AI teaching systems, when introduced to formal school settings without teachers’ involvement, are often opposed and poorly adopted. Learning Analytics Dashboards have thus evolved to become a necessary connecting tool between artificial intelligence-driven instruction and classroom teaching.

Learning Analytics Dashboards play a vital role in supporting collaboration between intelligent learning systems and educators by transforming raw student data into meaningful insights. Recent research in educational technology shows that modern dashboards have moved beyond simple grade-reporting tools and now function as comprehensive decision-support systems [37]. Instead of only presenting assessment scores, these dashboards integrate diverse data such as student interaction patterns, time spent on learning activities, and performance trends, and present them through clear and actionable visualizations. In the proposed system, this means the dashboard will highlight not only final learning outcomes but also the underlying learning processes, enabling educators to better understand student behavior and provide timely, personalized support.

A very important element highlighted in the literature is real-time progress tracking. Unlike a normal system whereby feedback on progress is only displayed at the end of an academic term, an AI-driven tracking system is continuously monitoring student engagement patterns [38]. Research further accentuates how a good system is able to use these patterns to point out misconceptions before the end term and identify students who might not perform well [39].

This capability is directly relevant to the proposed Sinhala AI Teaching Avatar. By integrating a teacher-facing analytics dashboard, educators can be provided with granular, interpretable data, including:

- **Modality Preferences:** Visualization of whether individual students learn more effectively through voice-based instruction, video animations, or text-based notes.
- **Engagement Metrics:** Insights derived from the attention-tracking module, highlighting when and where students lose focus during lessons.

This dual-layer architecture—AI-driven personalized instruction for students combined with analytics-driven insights for teachers—ensures that the system continuously

validates and improves its instructional effectiveness through measurable learning outcomes, rather than delivering content in an unsupervised manner.

Table 2.10: Learning Analytics and Teacher Dashboard Support in Adaptive Learning Systems

Focus / Component	Key Features / Methods	Benefits / Advantages	Reference
Learning Analytics Dashboards (LADs)	Consolidates interaction logs, time-on-task data, and performance trends into visual representations	Provides actionable insights into student learning processes and enhances teacher collaboration	[37]
Real-Time Monitoring	Continuous analysis of student engagement and interaction patterns	Early detection of misconceptions, prediction of at-risk students, and timely intervention	[38]
Predictive Intervention	Uses attention tracking and engagement metrics to identify learning gaps	Supports data-driven instructional decisions and improves learning outcomes	[39]
Dual-layer Architecture	Combines AI-driven personalized instruction for students with teacher-facing analytics	Validates and improves instructional effectiveness while ensuring measurable learning progress	[37]–[39]

2.2 Gaps in Literature

A review of existing literature on Intelligent Tutoring Systems, Adaptive Learning, AI-Based Personalized Learning, and Educational Teaching Avatars reveals several critical gaps that motivate this research. Most current Intelligent Tutoring Systems and adaptive learning systems are developed for English-language contexts and rely mainly on text-based interaction, with limited support for low-resource languages such as Sinhala. Speech-based interaction and multimodal content delivery, including voice, visuals, and animations, are not integrated, reducing accessibility and learner engagement.

Although adaptive learning systems employ student modeling techniques such as learning analytics, machine learning, and reinforcement learning, existing research largely focuses on generic curricula and English-medium education. There is a lack of systems that combine Sinhala language support, speech-based assessment, and curriculum-aligned adaptive learning tailored to Sri Lankan G.C.E. O/L and A/L syllabus.

AI-based personalized learning research demonstrates strong potential for dynamic content adaptation, yet most systems depend on traditional text or limited audio interfaces and underutilize multimodal interaction. In addition, localized AI-driven personalization for Sinhala-medium education remains underexplored.

Research on educational teaching avatars highlights challenges related to language resources, speech synthesis, and emotion-aware interaction. Most existing avatar-based learning systems are designed for English-speaking learners and show limited integration with adaptive tutoring, student modeling, and curriculum-based assessment, which constrains their applicability in personalized learning environments.

Across these studies, the literature consistently shows that RAG enhances educational

AI by improving accuracy, reducing misinformation, and enabling curriculum-aligned instruction. RAG-based systems are particularly suitable for adaptive learning environments, where student queries and quiz results determine which content should be retrieved and explained. However, most existing RAG-based educational systems focus on English-language resources and higher-education contexts. There is limited research on applying RAG to Sinhala-medium school education, particularly for G.C.E. O/L and A/L syllabus.

Most existing dashboards still focus on grades and simple engagement, with limited support for visualizing the deeper learning process over an entire O/L or A/L curriculum. There is also very little work on dashboards specifically designed for Sinhala-medium school contexts, as most studies are in English and higher education settings. Finally, many systems only show analytics but are not tightly integrated with AI tutors or multi-agent architectures, thus teachers cannot directly adjust AI-driven instruction through a unified interface.

Chapter 3

Methodology

3.1 Survey Overview and Key Findings

In this study, we conducted a survey with randomly selected groups of Advanced Level (A/L) and Ordinary Level (O/L) students, to learn about the experiences of students during their education and to glean useful insights into different aspects of their learning practices. An online survey was distributed through Google Forms. The form was published to students in various areas and at different schools. About 80 of both A/L and O/L students shared their education levels, whether they have access to personalized teaching, and how effectively they use AI. Additionally, we collected their suggestions to help guide the development of our project and to align with their learning needs. These findings are quite informative and provide us with a perspective on how these students engage with their learning as well as what impacts their experiences have. We can also identify what sort of system really students want.

The most comparative insights from this survey are summarized below:

- There is a shift in the pattern of migration towards smaller group and individual classes, favored by O/L students, to large group classes, which are preferred by A/L students. This could be due to a change in their priorities ranging from personalized attention to accessing specialized, high demand teachers.
- O/L students are more likely to feel they receive personal attention from teachers (70.3%). But A/L students not getting personal attention from teachers (57.1%).
- A/L students have a strong preference for an interactive teaching style, of which the most preferred option is “The teacher asking questions during the lesson” (38.9%). The most preferred choice for O/L students is a “Q&A discussion after the lesson” (42.1%).
- 94.4% of the students stated that they apply ChatGPT and Gemini for their study or activities. It indicates a great tendency of students towards using AI-powered tools to fulfill their study purposes.
- Among the 94.4% of students who use AI tools, the majority of their purpose is to get answers to questions related to their studies. The second major purpose of AI tool use, as reported by a significant amount of students (56.8%), is to deepen their understanding of what they are learning.

- However, one keen insight from the survey is that 26.3% of students said these AI tools do not align answers with their syllabus. That is a main issue our project needs to address: to ensure that AI-driven tools, like our Sinhala assistant, provide content directly aligned with the local educational curriculum and support personalized learning among A/L and O/L students.

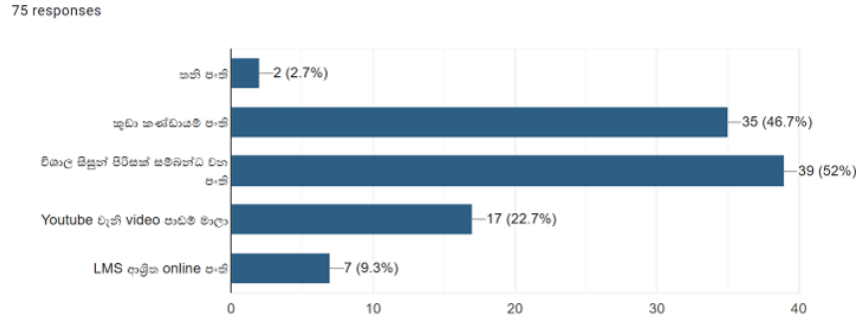


Figure 3.1: Different tuition classes students attempted

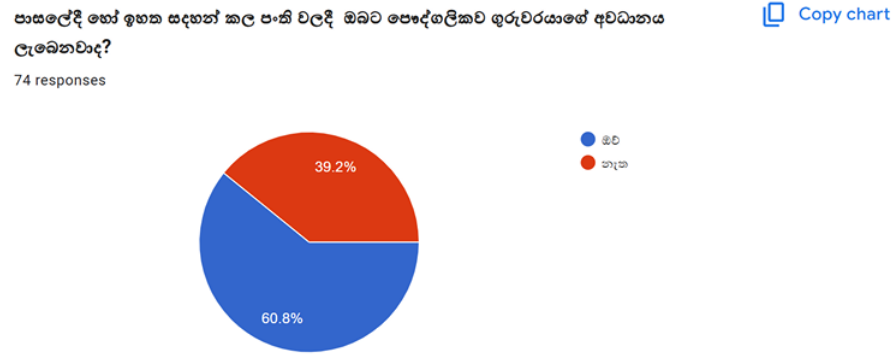


Figure 3.2: Students' status about their personalized learning

Most students emphasized that our AI learning system must adjust according to their knowledge level. In other words, the system should cater to a personal or relevant learning experience. Notably, they drew emphasis on the fact that the system should add lessons or answers according to the curriculum. Finally, students also emphasized the importance of the system's ability to detect their weaknesses in a particular area or provide their special focus according to those weaknesses. Additionally, students also emphasized that the learning system should use voice, animations, pictures, and notes.

According to the survey, the students also asked that in this system, the teaching avatar must use attractive and friendly graphical designs, with great importance attached to preserving a natural and realistic look. This is being emphasized, as an attractive avatar is known to prevent boredom and keep learners engaged, making learning more fun and pleasant for them. The use of attractive animations is also recommended in this regard in order to keep students' attention fixed during learning.

3.2 System Overview

This chapter details the methodology behind designing, implementing, and evaluating our intelligent, adaptive Sinhala-language learning platform for A/L and O/L students. Moving away from traditional "one-size-fits-all" e-learning solutions, our system introduces a deeply personalized learning loop driven by advanced artificial intelligence and modern software architecture.

To bring this platform to life, we developed the system across four interconnected pillars:

- **AI Model Fine-Tuning & Knowledge Base:** The foundation of our system relies on domain-specific language models. We fine-tuned five selected language models using a custom dataset carefully curated from official textbooks, teacher guides, and past examination papers. To ensure accuracy, these resources were digitized, segmented, and stored in a ChromaDB database, creating a robust Retrieval-Augmented Generation (RAG) pipeline. This setup guarantees that all AI-generated Sinhala content is not only factually correct but also strictly aligned with the national curriculum.
- **Multi-Agent Backend Orchestration:** Our backend, built on FastAPI and LangGraph, coordinates a sophisticated Multi-Agent System. Instead of relying on a single monolithic model to handle every task, the system distributes the workload across seven specialized agents: Orchestrator, Evaluator, Personalization, Content Generator, Quiz, Explain, and Progress Tracker. This collaborative approach allows the platform to seamlessly grade answers, generate quizzes, provide tailored explanations, and track student progress.
- **Hybrid-BKT Personalization Engine:** At the heart of our adaptive experience is a novel Hybrid Bayesian Knowledge Tracing (Hybrid-BKT) engine. We combined Personalized Clustered BKT (PC-BKT)-which measures mastery using empirical probabilities and K-means clustering-with a neural network (LSTM) designed to predict learning patterns over time, such as knowledge retention and learning plateaus. The resulting mastery score dynamically adjusts the difficulty of each lesson to suit the student's current level, scaling from Beginner up to Expert.
- **Multimodal Content Delivery & Frontend:** We developed two distinct interfaces to deliver this personalized content effectively. Students engage with a Next.js web application that features dynamic quizzes, interactive lessons, and an AI Avatar capable of delivering spoken Sinhala explanations through a custom Text-to-Speech (TTS) pipeline. Meanwhile, parents and teachers can access a React Native mobile application equipped with real-time learning analytics, allowing them to conveniently monitor individual student progress and mastery trends on the go.

By bringing together a verified knowledge base, smart multi-agent orchestration, and deep-learning-driven personalization, this methodology establishes a strong and scalable framework for delivering an engaging secondary education platform in the Sinhala medium.

3.3 System Workflow

The overall system workflow was designed to seamlessly integrate various AI-driven components to deliver a personalized educational experience. As illustrated in Figure 3.3, the workflow consists of several interconnected pipelines that manage content creation, personalized learning, content delivery, and progress tracking.

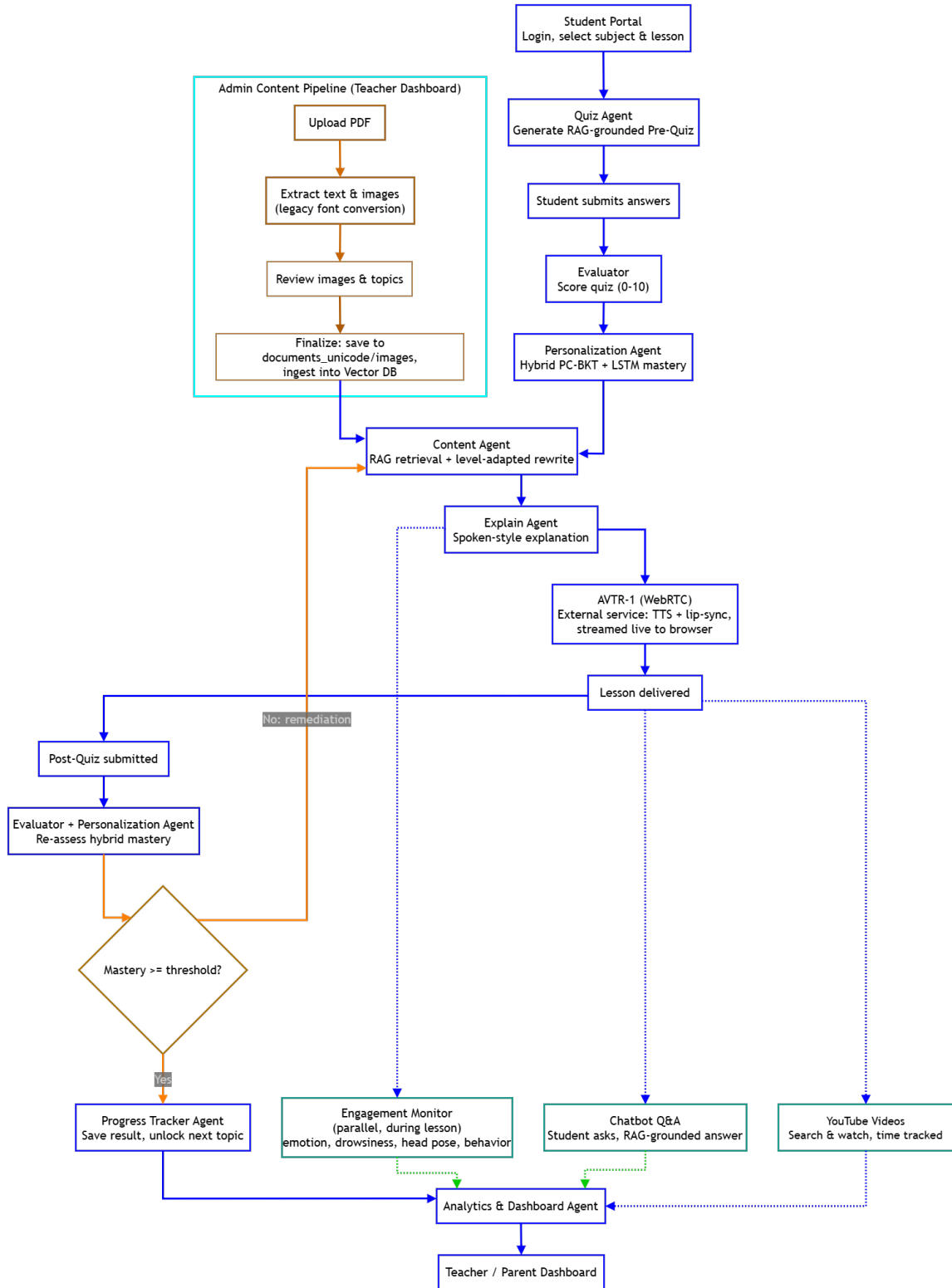


Figure 3.3: Overall System Workflow

The entire system workflow operates through a series of connected steps, starting from how teachers upload content to how students actually learn from it. First, teachers use their dashboard to upload standard curriculum PDFs. The system automatically extracts the text and images, fixing any old font issues along the way. Once the teachers review and approve this extracted material, it is saved into a ChromaDB

vector database. This database acts as the core knowledge source for the RAG pipeline. When a student logs into the platform and selects a lesson, the system first needs to understand their current knowledge level. It does this by triggering the *Quiz Agent* to create a quick pre-quiz based on the RAG pipeline. After the student answers the questions, the *Evaluator Agent* calculates their score. This score is immediately sent to the *Personalization Agent*, which uses a combination of PC-BKT and LSTM models to accurately determine how well the student understands the topic.

Once the system knows the student’s mastery level, it builds a custom lesson just for them. The *Content Agent* searches the vector database and rewrites the educational material so that it perfectly matches the student’s learning capacity. However, since reading long texts can be boring, the *Explain Agent* rewrites this content into a more conversational, spoken style. This script is then streamed via WebRTC to an external service that generates real-time speech and lip-syncs it to an interactive AI Avatar, which delivers the lesson directly on the student’s screen.

While the student is watching the avatar and learning, several background tools quietly monitor the session. An Engagement Monitor uses the webcam to track the student’s emotions, head movements, and signs of drowsiness. At the same time, the student can use a built-in chatbot to ask questions or watch relevant YouTube videos right within the platform. All this behavioral and engagement data is continuously sent to the Analytics and *Dashboard Agent*. Finally, at the end of the lesson, the student takes a post-quiz. The Evaluator and *Personalization Agents* check these new answers to see if the student’s mastery level has improved. If their score is too low, the system loops them back to receive easier, remedial content. If they pass the threshold, the *Progress Tracker Agent* unlocks the next topic. All of these results, along with the engagement data, are then organized and displayed on the teacher’s dashboard, giving educators a complete picture of how the student is performing.

3.4 Multi-Agent System Design

To ensure modularity and efficiency, the proposed platform is designed using a multi-agent architecture. Within this framework, agents operate as specialized autonomous components, each dedicated to a specific functional role. Rather than categorizing these agents solely by their responsibilities, they are grouped into two primary categories based on their execution triggers. The first group is orchestrated by a central *Supervisor* utilizing LangGraph. The second group consists of independent agents that are invoked directly on demand, such as during real-time quiz generation or rapid avatar speech synthesis.

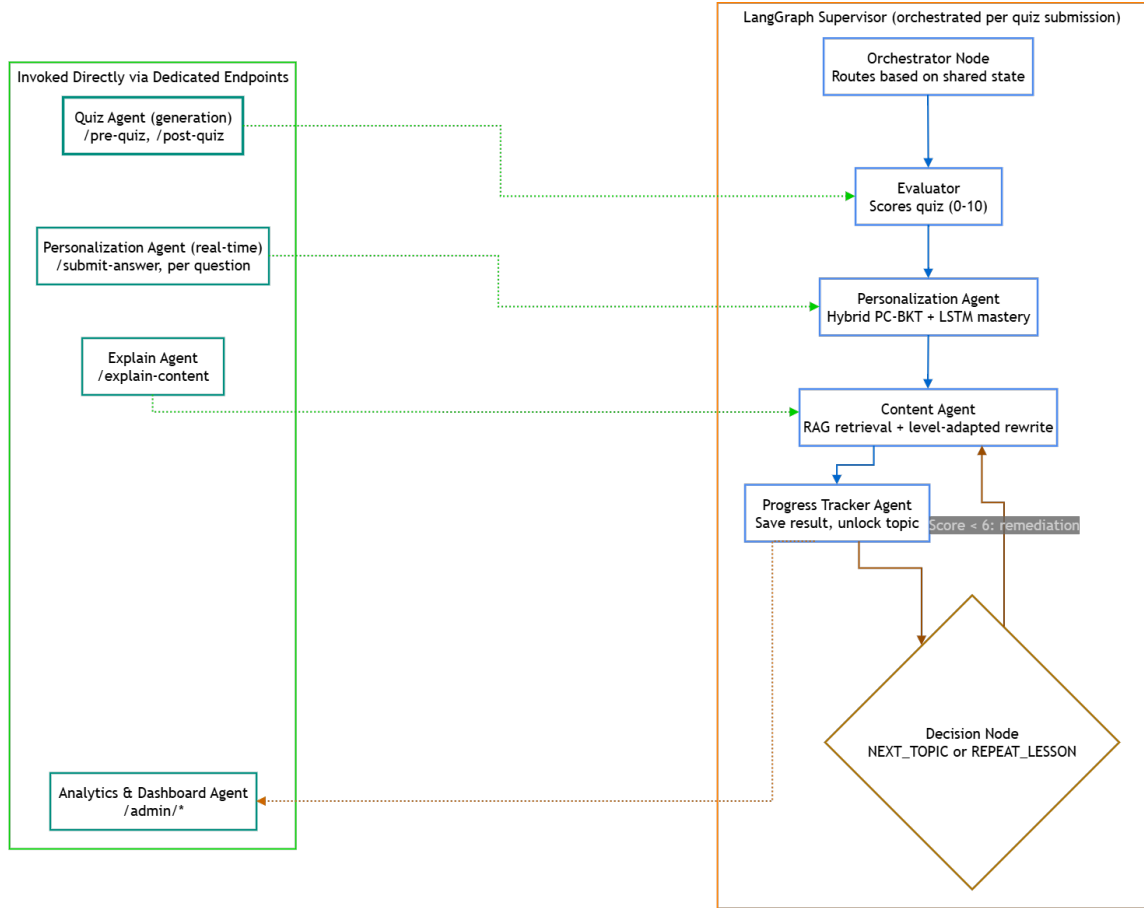


Figure 3.4: Multi-Agent System Architecture

3.4.1 State Management and Orchestration

All agents within the framework share a common, mutable state object (`LearningState`), which is explicitly managed by the LangGraph engine. This state ensures a consistent learner context across multiple node transitions by maintaining attributes such as the `student_id`, `score`, `mastery`, `hybrid_mastery`, `content`, and `quiz_type`. As the state is continuously passed and updated along the execution graph, agents can operate seamlessly without repetitive database queries.

The conditional routing mechanism creates an iterative refinement loop driven by the Orchestrator Node. This primary node evaluates the `LearningState` at every step to determine the next logical action. For instance, during a pre-quiz flow:

1. If the `score` is null, the Orchestrator routes execution to the *Evaluator Agent* to grade student answers.
2. If the answers are evaluated but the `mastery` is null, it routes to the *Personalization Agent* to update the student's learning profile.
3. Once the profile is updated, it triggers the *Content Generator Agent* to synthesize new, appropriately scaled learning material.
4. Finally, execution is routed to the *Progress Tracker Agent* to persist the entire transaction to the database.

3.4.2 Agents Managed by the Supervisor (LangGraph)

The proposed adaptive learning system implements a multi-agent architecture using LangGraph, where multiple specialized AI agents collaborate under the control of a *Supervisor Agent* [40]. Each agent is responsible for performing a specific task within the learning process, such as evaluating student performance, estimating knowledge level, generating personalized learning materials, and tracking student progress. To enable communication between agents, the system uses a shared memory structure called the *learning state*. The learning state maintains all important information required during the learning workflow, including student identification details, submitted quiz answers, calculated scores, learner classification, mastery level estimations, generated educational content, and progress information [41].

The *Supervisor Agent* continuously monitors this shared learning state and determines the next appropriate action based on the current status of the workflow. Unlike a traditional fixed pipeline, where tasks are executed in a predefined order, the LangGraph-based approach allows dynamic decision-making. The *Supervisor* selects the required agent according to the missing information or the current learning requirement of the student [40, 42]. The agent workflow is triggered when a student completes and submits either a pre-quiz or a post-quiz through the system endpoints (`/submit-pre-quiz/` and `/submit-post-quiz/`). After receiving the quiz submission, the supervisor coordinates the required agents to evaluate performance, personalize the learning experience, generate suitable learning content, and update the student's learning progress.

Orchestrator Node (Supervisor Agent)

The Orchestrator Node is the central control component of the LangGraph workflow. It is responsible for managing the execution order of other agents and making workflow decisions based on the information available in the learning state.

The *Supervisor* analyzes the current state and identifies incomplete tasks. For example, if the quiz score is not available, the *Supervisor* forwards the task to the *Evaluator Agent*. If the student's mastery level needs to be updated, the *Personalization Agent* is activated. This dynamic routing mechanism allows the system to adapt its workflow according to each student's situation. For a pre-quiz submission, the typical workflow begins with the *Evaluator Agent* calculating the student's score, followed by the *Personalization Agent* estimating the learner's knowledge level. Then, the *Content Agent* generates suitable learning materials, and the *Progress Tracker Agent* stores the learning information.

For post-quiz submissions, the supervisor evaluates the student's performance outcome. If the student achieves a score of 6 or above out of 10, the system allows progression to the next topic. If the score is below 6, the supervisor directs the workflow to generate additional remedial learning content before updating the student's progress.

Evaluator Agent

The *Evaluator Agent* is responsible for automatically assessing student quiz performance. It compares the answers submitted by the student with the predefined correct answers and calculates the final score.

After evaluation, the agent assigns an initial learner category based on the obtained score:

- Advanced: Score ≥ 8
- Intermediate: Score between 5 and 7
- Beginner: Score < 5

This classification provides an initial understanding of the student's current ability level. However, this estimation is only based on quiz performance and may not fully represent the student's actual knowledge state. Therefore, the *Personalization Agent* performs further analysis using AI-based knowledge estimation models to obtain a more accurate mastery level.

Personalization Agent (Batch Path)

The *Personalization Agent* is the main component responsible for adapting the learning experience according to individual student characteristics. It analyzes student performance, learning patterns, and previous interactions to determine suitable learning recommendations.

Initially, the agent identifies the student's performance category by analyzing available learning data. Students are categorized as high performers, medium performers, or low performers. For new students without previous learning history, the system assigns an initial learner group using available quiz information to avoid starting the personalization process without prior knowledge. The agent also updates question difficulty levels based on student responses. This allows the system to understand whether questions are appropriately challenging for different learners. After analyzing student performance, the *Personalization Agent* calculates a hybrid mastery score by combining two AI-based knowledge estimation models: Bayesian Knowledge Tracing (BKT) and Long Short-Term Memory (LSTM).

The BKT model estimates the probability that a student has mastered a particular concept based on previous interactions, while the LSTM model identifies learning patterns from sequential student data. Combining both models provides a more reliable estimation of student knowledge. If the predictions from BKT and LSTM show a significant difference, the system reduces the confidence of the final mastery score. This uncertainty handling mechanism prevents unreliable predictions caused by disagreement between the models. Based on the calculated mastery level, the *Personalization Agent* determines the next learning strategy. It can recommend easier questions for students who require additional support, more challenging questions for advanced learners, or additional hints and explanations when necessary.

Content Agent

The *Content Agent* is responsible for generating personalized educational materials based on the student's estimated knowledge level. It uses the learner profile and mastery information provided by the *Personalization Agent* to create suitable content.

The agent retrieves relevant lesson information and transforms it according to three learning levels:

- Beginner learners receive simplified explanations with basic concepts.

- Intermediate learners receive detailed explanations with examples.
- Advanced learners receive deeper explanations with more challenging information.

To overcome limitations related to large text processing by language models, the *Content Agent* applies a content segmentation approach. Large lessons are divided into smaller sections, usually one or two paragraphs, before processing. These sections are handled using batch processing to improve efficiency and maintain consistency. During content generation, the agent also preserves important placeholders for images, diagrams, and other visual resources. This ensures that important educational elements remain available after AI-based rewriting or Sinhala translation.

Progress Tracker Agent

The Progress *Tracker Agent* manages the storage and maintenance of student learning records. It is responsible for saving quiz results, mastery estimations, generated learning activities, and improvement information into the database.

After each quiz attempt, the agent records the student's performance data, which can later be used for progress analysis and teacher dashboard visualization. For post-quiz evaluations, the Progress *Tracker Agent* checks whether the student has achieved the required performance level. When a student obtains a score of 6 or above, the next learning topic is unlocked. If the required score is not achieved, the student is provided with additional learning support. Furthermore, this agent calculates the improvement between pre-quiz and post-quiz results. This measurement helps evaluate the effectiveness of the personalized learning approach and provides useful insights for teachers.

Decision Node

The Decision Node is the final stage of the LangGraph workflow. It determines the next action based on the student's post-quiz performance.

The decision process follows a predefined evaluation rule:

- Score ≥ 6 : The student has demonstrated sufficient understanding, and the system moves the student to the next topic (NEXT_TOPIC).
- Score < 6 : The student requires additional learning support, and the system recommends repeating the lesson with remedial content (REPEAT_LESSON).

The final decision is communicated to the frontend application, which uses this information to determine whether the student should continue with new content or receive additional learning materials. Overall, the *Supervisor*-controlled LangGraph architecture enables efficient collaboration between multiple AI agents while supporting adaptive and personalized learning. Each agent performs a specialized responsibility, and their combined operation creates an intelligent learning pipeline capable of evaluating students, generating suitable educational content, and continuously tracking learning progress.

3.4.3 Independent Agents (Directly Invoked)

In addition to the supervisor-controlled agents, the system also includes several independent AI agents that operate separately from the LangGraph workflow. These agents are not controlled by the *Supervisor Agent* and are directly called through their own dedicated system endpoints whenever their services are required.

These independent agents are mainly designed for real-time operations where immediate responses are needed and waiting for the completion of a full quiz workflow is not practical. Each agent performs a specific task independently while still supporting the overall personalized learning process.

Quiz Agent (Generation Path)

The *Quiz Agent* is responsible for automatically generating quizzes based on lesson content. When a student or system component requests a quiz, this agent uses the AI model to create a set of ten multiple-choice questions related to the selected lesson.

Since AI-generated outputs may sometimes contain incorrect formats or unsuitable content, the *Quiz Agent* applies a validation process before providing the final quiz. The generated questions are checked to ensure that they follow the required structure and contain valid answer options. The agent also verifies the language quality of the generated questions. Since the proposed system focuses on Sinhala-based learning, the agent removes questions that contain excessive English characters or do not meet the required Sinhala language standard. If the generated output does not contain enough valid questions, the agent automatically repeats the generation process. The system attempts up to three times to produce the best possible set of ten questions. This approach improves the reliability and quality of automatically generated quizzes.

Personalization Agent (Real-Time Path)

The Real-Time *Personalization Agent* provides immediate adaptation during the quiz process. Unlike the batch personalization approach, which analyzes student performance after completing an entire quiz, this agent updates the learner model after each individual question.

After every student response, the agent analyzes the current performance and updates the student's learning state. It performs similar calculations to the batch personalization process, including identifying the learner group, updating the Bayesian Knowledge Tracing (BKT) model, and combining the results with the LSTM model to estimate the student's current mastery level. This real-time approach allows the system to identify learning difficulties immediately. For example, if a student repeatedly answers questions incorrectly, the system can recognize that the student is struggling and provide suitable hints or additional support during the quiz itself.

Analytics and Dashboard Agent

The Analytics and *Dashboard Agent* is responsible for collecting and presenting learning data for teachers and administrators. This agent works independently from the quiz and personalization workflows.

It gathers information from the system database and generates useful insights, including the number of registered students, subject completion rates, topic-level performance, and average improvement of students over time. The collected information

is presented through the teacher dashboard, allowing educators to monitor student progress and understand learning patterns. Since this agent operates separately, it can provide real-time analytical information without affecting the main learning workflow.

Explain Agent

The *Explain Agent* focuses on improving the readability and naturalness of educational content. Its main purpose is to convert complex textbook-style explanations into a simpler and more conversational format.

This transformation is especially important for the AI Avatar component because the generated explanations need to sound natural when converted into speech. By rewriting content into a conversational style, the agent helps create a more engaging learning experience for students. For longer lessons, the agent processes multiple paragraphs together to improve efficiency and reduce processing time. Additionally, the system includes a failure-handling mechanism. If the AI model fails to generate a suitable explanation, the agent returns the original content instead of interrupting the learning process. Overall, these independent agents provide additional capabilities that support real-time quiz generation, immediate personalization, educational content explanation, and learning analytics. By operating separately from the LangGraph *Supervisor* workflow, they improve system flexibility and allow specific tasks to be performed efficiently whenever required.

TTS Agent and Align Agent

In the initial system architecture, the *TTS Agent* and *Align Agent* were designed as backend components responsible for generating speech output and synchronizing avatar lip movements with generated audio.

The *TTS Agent* was expected to convert AI-generated learning content into speech, while the *Align Agent* was responsible for calculating timing information required to synchronize the avatar's mouth movements with the generated speech. However, during implementation, it was identified that managing these tasks in the backend increased system complexity and processing time. A more efficient approach was found by moving these responsibilities to the frontend layer or external avatar services. For the Full Body Avatar, speech generation and lip-sync processing were integrated directly through dedicated Next.js routes on the frontend. This allowed faster interaction between the user interface and the avatar while reducing unnecessary communication between the frontend and backend.

For the AVTR-1 Avatar, these functionalities are handled through an external avatar service that provides built-in speech synthesis and lip-sync capabilities. Therefore, separate backend agents for TTS generation and alignment were no longer required. As a result, the *TTS Agent* and *Align Agent* were removed from the final agent workflow because their functionality was successfully replaced with more efficient frontend-based and external solutions.

Student Modeling Agent and Adaptive Learning Agent

The original system proposal included two separate agents called the *Student Modeling Agent* and the *Adaptive Learning Agent*. These agents were initially designed to analyze student performance and decide suitable learning paths.

The Student *Modeling Agent* was responsible for creating a student profile by analyzing quiz scores, while the Adaptive *Learning Agent* used this information to select the next learning activity. However, the initial approach mainly depended on simple quiz percentage calculations to determine the student’s ability level. Although this method provided basic classification, it was not sufficient for a fully adaptive learning environment because it did not consider detailed learning patterns, previous interactions, or knowledge development over time. To improve the intelligence of the system, these two agents were replaced by the newly designed *Personalization Agent* and Decision Node.

The *Personalization Agent* combines multiple factors, including student responses, learner groups, Bayesian Knowledge Tracing (BKT), and LSTM-based predictions, to calculate a more accurate mastery level. This provides a deeper understanding of the student’s actual knowledge state rather than relying only on quiz scores. The Decision Node then uses this mastery information and assessment results to determine the appropriate next action, such as moving the student to a new topic or providing additional remedial learning materials. Although the original Student *Modeling Agent* and Adaptive *Learning Agent* are not used in the final system execution, their concepts contributed to the development of the improved personalization and decision-making architecture. Their previous implementations are still maintained within the project repository for documentation and future improvements.

Overall, the removal of these modules represents the continuous improvement of the system design throughout the development process. By replacing less effective approaches with more advanced and integrated solutions, the final architecture achieves better performance, flexibility, and a more personalized learning experience.

3.5 Retrieval-Augmented Generation (RAG) Pipeline

The RAG pipeline is a key component of the proposed learning system that improves the accuracy and reliability of AI-generated educational content. LLMs can sometimes generate incorrect or unsupported information, a problem known as hallucination. To reduce this risk, the RAG pipeline connects the language model with a verified educational knowledge base [43]. Instead of generating answers only from the model’s internal knowledge, the RAG approach first retrieves relevant information from trusted curriculum resources and then provides this information as additional context to the language model [43]. This ensures that generated explanations, quizzes, and learning materials remain aligned with the official educational content. The RAG pipeline consists of three main stages: knowledge base construction, vector encoding and indexing, and contextual retrieval.

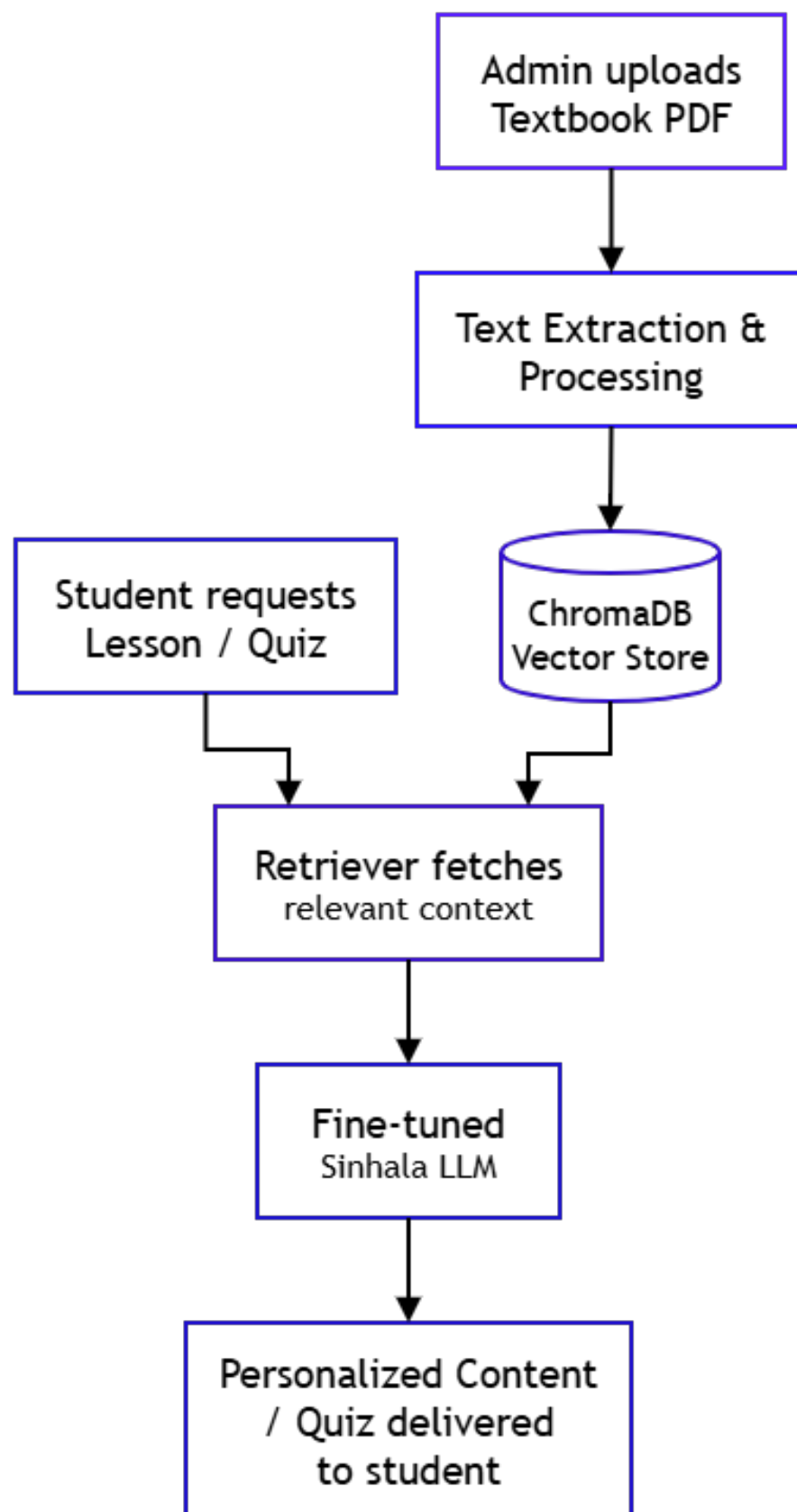


Figure 3.5: RAG Architecture

3.5.1 Knowledge Base Construction

The first stage of the RAG pipeline is the creation of a reliable educational knowledge base. The knowledge base was developed using Sinhala medium educational resources, including official school textbooks, teacher instruction manuals, and previous examination papers. Since many educational resources were available in document or scanned formats, Optical Character Recognition (OCR) technology was used to convert these documents into machine-readable text [44]. After the OCR process, the extracted text was manually reviewed and verified to remove recognition errors and ensure the accuracy of the stored educational information. To improve the retrieval performance, large documents were divided into smaller text sections called chunks [43]. This allows the system to retrieve only the most relevant parts of a document instead of processing the complete document every time.

In this implementation, documents were divided into chunks containing 512 tokens, with an overlap of 64 tokens between consecutive chunks. The overlapping approach helps maintain the connection between related sentences and prevents important information from being separated during the chunking process [45]. For example, when a lesson explanation continues across two sections, the overlapping tokens ensure that the meaning of the content is preserved during retrieval.

3.5.2 Vector Encoding and Indexing

After preparing the document chunks, the next step is converting the text data into numerical representations that can be understood by the retrieval system. This process is performed using a sentence transformer model called `intfloat/multilingual-e5-base` [46]. The model converts each text chunk into a high-dimensional vector representation called an embedding. In this system, every document chunk is converted into a 768-dimensional vector. These vectors represent the semantic meaning of the text, allowing the system to identify similar content even when different words are used [46]. The generated embeddings are stored and indexed using ChromaDB, which is a vector database designed for efficient similarity searching [47]. When a user submits a query or the system requires learning content, ChromaDB can quickly compare the query embedding with stored document embeddings and retrieve the most relevant educational information.

This vector-based approach provides better retrieval accuracy compared to traditional keyword-based searching because it focuses on the meaning of the content rather than only matching exact words [43].

3.5.3 Contextual Retrieval and Content Generation

During system operation, whether a student is asking a question, responding to the AI tutor, or requesting new learning materials, their text input is first converted into an embedding using the same `multilingual-e5-base` model used for document encoding. The generated query embedding is then compared with the indexed document embeddings stored in ChromaDB. Based on semantic similarity, the system retrieves the most relevant document sections from the knowledge base [43]. In this implementation, the top five most similar document chunks are selected and provided as additional context to the language model during content generation. Furthermore, the system applies mastery-aware prompting to personalize the generated output. The

student's current mastery level is included in the generation process so that the AI can adjust the complexity of explanations according to the learner's ability.

For example:

- Beginner students receive simpler explanations with basic examples.
- Intermediate students receive more detailed explanations.
- Advanced students receive deeper explanations with more complex information.

By combining retrieved curriculum information with student mastery information, the system can generate responses that are both academically accurate and suitable for the student's current learning level.

3.5.4 Retrieval Modes

The developed RAG system supports two different retrieval methods depending on the required task.

Document Order Retrieval Mode

This mode is mainly used for generating complete lesson content. Instead of selecting only the most similar chunks, the system retrieves all chunks related to a specific source document and organizes them according to their original paragraph order.

This maintains the natural flow and structure of the lesson because educational content often depends on the order in which concepts are introduced.

Vector Similarity Retrieval Mode

This mode is mainly used for quiz generation and question answering tasks. The system searches the knowledge base using semantic similarity and retrieves the most relevant chunks related to the requested topic.

This allows the *Quiz Agent* and other AI components to generate questions based on accurate curriculum information rather than unrelated generated knowledge.

3.5.5 Integration with Multimodal Learning System

The backend FastAPI server also provides access to educational images and illustrations stored in the system. A static image directory is mounted in the backend, allowing the Next.js frontend application to dynamically display relevant images together with the retrieved learning content.

This integration supports a multimodal learning experience by combining text explanations with visual learning materials such as diagrams and illustrations. As a result, students receive a more engaging and understandable learning experience. Overall, the RAG pipeline improves the reliability of the adaptive learning system by ensuring that AI-generated content is based on verified educational resources. By combining document retrieval, semantic vector search, and mastery-aware generation, the system can provide accurate, personalized, and curriculum-aligned learning materials while reducing the limitations of standalone generative AI models.

3.6 Real-Time Avatar Speech Delivery

The proposed adaptive learning system delivered lesson explanations through an AI-powered teaching avatar to create a more interactive and engaging learning experience. Instead of displaying only written text, the system converted personalized lesson content into natural speech and synchronized facial animation. This allowed students to learn through a virtual teacher that presented explanations in a more realistic and human-like manner. Before speech generation began, the lesson content was processed by the *Explain Agent*, which rewrote textbook-style content into a simple, conversational explanation suitable for spoken delivery. Once the explanation was generated, the system supported two different avatar delivery modes: AVTR-1 and Full Body Avatar. Although both modes used the same lesson explanation, they used different technologies to generate speech and animate the avatar.

The AVTR-1 mode delivered lessons through a live-streaming avatar using WebRTC technology and an external avatar-rendering service [48]. In contrast, the Full Body Avatar mode performed speech synthesis and lip synchronization within the frontend application using Gemini Text-to-Speech [49] and Rhubarb Lip Sync [50]. This dual implementation provided flexibility while maintaining the same personalized learning experience for students. The following sections describe the overall architecture of the avatar delivery system and explain the implementation of each avatar mode in detail.

3.6.1 Overall Avatar Speech Delivery Architecture

The real-time avatar speech delivery architecture was designed to transform AI-generated lesson content into an interactive learning experience. The overall workflow began after the adaptive learning system generated personalized lesson content for a student. Instead of presenting the lesson directly as text, the content was first converted into a conversational explanation using the *Explain Agent*. This made the lesson easier to understand when spoken by the avatar. After the explanation was generated, the system allowed the student to choose between two avatar delivery modes: AVTR-1 or Full Body Avatar. Both modes used the same explanation text but followed different processing pipelines to generate speech and facial animation.

In the AVTR-1 mode, the explanation text was sent to an external avatar-rendering service through a WebRTC connection. The external service performed speech synthesis, facial animation, and lip synchronization before streaming the generated avatar back to the student's browser in real time. This approach enabled smooth live video streaming while supporting personalized lesson content generated for each student. In the Full Body Avatar mode, the explanation text was processed within the frontend application. Speech was generated using Gemini Text-to-Speech, while lip synchronization was created using Rhubarb Lip Sync. The generated audio and animation data were then used to animate a three-dimensional avatar directly in the student's browser without requiring an external streaming service.

Although the two delivery modes used different technologies, both were integrated into the same adaptive learning framework and provided the same educational content. Students could therefore receive personalized lessons through either streaming or client-side avatar rendering, depending on the selected mode.

The overall architecture consisted of the following main components:

- **Explain Agent** – Converted educational content into a conversational explana-

tion suitable for spoken delivery.

- **Avatar Selection Module** – Allowed the student to choose either the AVTR-1 avatar or the Full Body Avatar.
- **Speech Generation Module** – Produced natural speech using either the external AVTR-1 service or Gemini Text-to-Speech.
- **Lip Synchronization Module** – Synchronized the avatar’s mouth movements with the generated speech.
- **Avatar Rendering Module** – Displayed the animated avatar and delivered the lesson to the student in real time.

This architecture separated content generation from speech generation and avatar animation, making each component independent and easier to maintain. It also allowed new avatar technologies to be integrated into the system in the future without changing the adaptive learning workflow.

3.6.2 AVTR-1 Streaming Avatar

The AVTR-1 Streaming Avatar was one of the two avatar delivery modes implemented in the proposed adaptive learning system. Its purpose was to present personalized lesson content through a realistic AI avatar that spoke and performed synchronized facial movements in real time. Unlike traditional educational videos, the avatar did not rely on pre-recorded content. Instead, it generated speech and facial animation dynamically based on the lesson explanation created for each student. After the *Explain Agent* generated a conversational version of the lesson, the explanation text was sent to the AVTR-1 avatar-rendering service. Along with the explanation, the system also sent the selected avatar, background, and playback speed. The avatar service then performed speech synthesis, lip synchronization, and facial animation before streaming the generated video back to the student’s browser.

To support this real-time communication, the AVTR-1 mode used WebRTC (Web Real-Time Communication). WebRTC was selected because it was designed for low-latency audio and video streaming between two connected devices. Since the lesson content was generated dynamically when a student requested it, there was no pre-generated video available for download. Therefore, using WebRTC allowed the system to begin streaming the avatar immediately while speech and animation were being generated. Unlike a traditional video streaming approach, the AVTR-1 service continuously sent audio and video frames as they were created. This enabled students to start watching the lesson without waiting for the entire video to be generated. As a result, the learning experience became more responsive and interactive.

The AVTR-1 streaming mode also supported real-time playback control. During lesson delivery, the browser and the avatar service communicated through a WebRTC data channel. This allowed the student to pause, resume, or change the playback speed while the lesson was being streamed. At the same time, the avatar service sent playback timing information to the browser, allowing the lesson text displayed on the screen to be highlighted in synchronization with the avatar’s speech. One of the main advantages of the AVTR-1 streaming approach was its ability to generate personalized educational content for every student. Since speech and facial animation

were created dynamically, the avatar could present different explanations based on the student's learning level without requiring multiple pre-recorded videos. This made the system more suitable for adaptive learning environments where lesson content changed according to student performance.

However, the AVTR-1 mode also depended on an external avatar-rendering service. The processes responsible for speech synthesis, facial animation, lip synchronization, and video rendering were performed outside the main learning platform. Therefore, the availability and response time of the streaming service could affect the overall user experience. Despite this dependency, the AVTR-1 streaming avatar provided a realistic and engaging method for delivering personalized educational content in real time.

3.6.3 WebRTC Communication Process

The AVTR-1 streaming avatar used WebRTC (Web Real-Time Communication) to establish a direct connection between the student's browser and the external avatar-rendering service. WebRTC was selected because it was specifically designed for low-latency audio and video communication. Unlike traditional HTTP-based video delivery, where users download a complete video file, WebRTC enabled live streaming of audio and video as they were generated. This was particularly important for the proposed adaptive learning system because the lesson explanation was personalized for each student and did not exist before the request was made. The WebRTC communication process consisted of three main stages: avatar initialization, connection establishment, and real-time media streaming.

Avatar Initialization

The communication process began when a student selected the AVTR-1 avatar mode to start a lesson. Before establishing a WebRTC connection, the browser retrieved the list of available avatars and background options from the avatar-rendering service. Since the service provided many avatar models, the system filtered the available options and displayed only those selected for the educational platform. At the same time, the browser requested the ICE (Interactive Connectivity Establishment) server configuration required for WebRTC communication. ICE servers helped establish a direct connection between the student's browser and the avatar-rendering service, even when both devices were located on different networks or behind firewalls. After receiving this information, the browser prepared the required communication settings for the streaming session.

WebRTC Connection Establishment

After the initialization stage, the browser created a WebRTC peer connection and prepared a data channel for exchanging control messages during lesson playback. The browser also added audio and video tracks that would receive the streamed media from the avatar service. Once the connection was prepared, the browser generated a Session Description Protocol (SDP) offer containing the communication details. This offer was sent to the avatar-rendering service together with the personalized lesson explanation, selected avatar, background, and playback speed. The avatar service processed the received information, generated an SDP answer, and returned it to the

browser. After the browser applied the received answer, the WebRTC connection was successfully established, allowing both systems to exchange media and control information in real time.

Real-Time Media Streaming

After the WebRTC connection had been established, the avatar-rendering service began generating speech, facial animation, and lip synchronization based on the lesson explanation. Instead of waiting until the complete lesson had been rendered, the generated audio and video were streamed continuously to the student's browser as they became available. In addition to audio and video streaming, the WebRTC data channel enabled real-time communication between the browser and the avatar service. The browser could send playback commands such as pause, resume, and playback speed adjustment during lesson delivery. The avatar service also sent playback timing information, including notifications when each paragraph began and ended. These timing events allowed the frontend to synchronize the displayed lesson text with the avatar's speech by highlighting the current paragraph while it was being spoken.

This real-time communication process provided a smooth and interactive learning experience. Students could control lesson playback without interrupting the streaming session, while synchronized speech, animation, and text highlighting improved the overall understanding of the lesson content.

3.6.4 Real-Time Playback Control

The proposed learning system allowed students to control lesson playback while the avatar was delivering explanations in real time. This functionality improved the learning experience by allowing students to learn at their own pace without restarting the lesson. The playback control mechanism was implemented using the WebRTC data channel, which enabled continuous communication between the student's browser and the AVTR-1 avatar-rendering service. Once the WebRTC connection had been established, the data channel remained active throughout the lesson. Unlike the audio and video streams, which were transmitted through media tracks, the data channel was used to exchange control messages and playback information between the browser and the avatar service.

During lesson delivery, students could perform several playback operations, including pausing the lesson, resuming playback, and adjusting the speaking speed of the avatar. When the student performed one of these actions, the browser sent a corresponding control message to the avatar service. The avatar service immediately processed the request and updated the ongoing speech generation without interrupting the streaming session. This enabled smooth and responsive lesson playback. In addition to receiving playback commands, the avatar service continuously sent playback status information back to the browser. These status messages indicated when the avatar started speaking a new paragraph and when it finished. Using this information, the frontend synchronized the displayed lesson text with the avatar's speech by automatically highlighting the paragraph currently being explained. This synchronization helped students follow the lesson more easily while listening to the explanation.

The real-time playback control mechanism provided several advantages for the adaptive learning platform. Students could temporarily pause the lesson to review important concepts, resume learning without restarting the session, and increase or

decrease the speaking speed according to their learning preference. Furthermore, synchronized text highlighting improved readability by showing students exactly which part of the lesson was being explained at any given moment. Overall, the real-time playback control component enhanced user interaction with the teaching avatar and created a more flexible, engaging, and personalized learning experience. It allowed students to actively control lesson delivery while maintaining smooth communication between the browser and the avatar-rendering service.

3.6.5 AVTR-1 Generative Model

The AVTR-1 generative model was responsible for producing realistic facial movements and lip synchronization for the teaching avatar. While the student's browser only sent the lesson explanation, selected avatar, and playback settings, all speech animation and facial motion generation were performed on the avatar-rendering server using the AVTR-1 model. Unlike traditional animation methods that generated an entire video at once, AVTR-1 created facial movements gradually in small time segments. This allowed the system to begin streaming the avatar while the remaining animation was still being generated, reducing the waiting time for students and supporting real-time lesson delivery. The AVTR-1 model combined speech analysis with AI-based motion generation to produce smooth and natural facial expressions that remained synchronized with the spoken lesson.

Speech Feature Extraction

Before generating facial animation, the lesson explanation was first converted into speech by the avatar-rendering service. The generated speech was then divided into small audio segments for processing. Instead of using the raw audio signal directly, the system extracted speech features using HuBERT speech embeddings. These embeddings represented important speech characteristics such as pronunciation, syllable boundaries, speaking rhythm, and voice intensity. By capturing these speech patterns, the model could better understand how the avatar's mouth and facial expressions should move during speech. This feature extraction stage provided the information required for generating natural lip synchronization and facial animation.

Autoregressive Motion Generation

After extracting the speech features, the AVTR-1 model generated facial movements using an autoregressive approach. Rather than predicting all facial movements at once, the model produced animation step by step. Each newly generated motion segment depended on both the current speech features and the facial movements generated in the previous step. This allowed the model to maintain smooth transitions between consecutive frames and prevented sudden or unrealistic facial movements. As a result, the avatar displayed continuous and natural head movements, facial expressions, and lip synchronization throughout the lesson.

Flow Matching Motion Prediction

To improve the realism of the generated animation, the AVTR-1 model applied a flow matching technique during motion prediction. Instead of learning only one fixed

animation sequence for a given speech input, the model learned how facial motion naturally changes over time. This approach enabled the model to generate more flexible and realistic facial movements because different people might naturally perform slightly different expressions while speaking the same sentence. By learning these motion patterns, the avatar produced smoother facial animation while maintaining accurate synchronization with the generated speech.

Chunk-Based Streaming

The AVTR-1 model generated speech animation in small chunks instead of processing the entire lesson before playback began. As soon as the first motion segment had been generated, the avatar-rendering service immediately started streaming it to the student's browser through the WebRTC connection. While the student was watching the first part of the lesson, the remaining speech and facial animation continued to be generated in the background. This chunk-based processing significantly reduced the initial waiting time and enabled real-time lesson delivery without requiring the complete lesson to be rendered in advance. This incremental generation approach was one of the key features that allowed the AVTR-1 streaming avatar to provide low-latency, personalized educational content while maintaining smooth speech, facial expressions, and lip synchronization throughout the lesson.

3.6.6 Current AVTR-1 Integration

The AVTR-1 model supported several advanced features for interactive avatar communication. One of its capabilities was dual audio input, where the avatar could speak using one audio stream while simultaneously receiving and responding to another incoming audio stream. This feature allowed the avatar to display natural listening behaviours during conversations, making interactions more realistic. However, in the current implementation of the proposed adaptive learning system, this capability was not fully utilized. The system sent only the personalized lesson explanation to the AVTR-1 avatar-rendering service. The service converted this text into speech and generated synchronized facial animation before streaming the lesson back to the student's browser. No live audio input was sent from the student to the avatar during lesson delivery.

As a result, the avatar functioned as an intelligent virtual teacher that delivered personalized lesson explanations but did not visually react while the student was speaking. The current integration focused on providing clear educational content through real-time speech synthesis, facial animation, and lip synchronization rather than supporting two-way voice conversations. Despite this limitation, the current implementation successfully met the requirements of the proposed adaptive learning platform by providing dynamic, personalized lesson delivery in real time. The modular architecture also allowed additional interactive features to be integrated in the future without requiring major changes to the existing system.

Future versions of the system could extend the current integration by enabling the avatar to receive live student audio during learning sessions. This would allow the avatar to display natural listening behaviour, respond to spoken questions, and support more interactive voice-based learning experiences. Such improvements could further enhance student engagement and create a more realistic AI-assisted teaching environment.

3.6.7 Full Body Avatar Mode

The Full Body Avatar mode was the second avatar delivery method implemented in the proposed adaptive learning system. Unlike the AVTR-1 streaming avatar, this mode performed speech generation and avatar animation within the frontend application. It was developed as an alternative solution that did not depend on an external avatar-rendering service while still providing an interactive learning experience. After the *Explain Agent* generated a conversational lesson explanation, the text was sent to a Next.js API route running within the frontend server. This API was responsible for generating speech and preparing the information required for lip synchronization. The generated audio and animation data were then returned to the student's browser, where a three-dimensional avatar model presented the lesson. The Full Body Avatar mode consisted of three main processes: speech synthesis, lip synchronization, and client-side animation.

Speech Synthesis

The first step in the Full Body Avatar pipeline was speech generation. The explanation text produced by the *Explain Agent* was sent to a Next.js API route, where Gemini Text-to-Speech (TTS) was used to convert the text into natural speech. The generated audio provided the spoken lesson that would be played during avatar animation. Unlike the original backend implementation, speech synthesis was handled entirely within the frontend environment. As a result, the backend *TTS Agent* was no longer required in the current system. This approach simplified the overall architecture and reduced communication between the frontend and backend during lesson delivery.

Lip Synchronization

After the speech had been generated, the system performed lip synchronization using Rhubarb Lip Sync. This tool analyzed the generated audio and identified the sequence of mouth shapes, also known as visemes, that corresponded to the spoken words. The generated viseme timeline was returned together with the audio file. During playback, these visemes were used to synchronize the avatar's mouth movements with the generated speech. This created the appearance that the avatar was naturally speaking the lesson, improving the realism of the learning experience. Since lip synchronization was performed directly from the generated audio, the backend *Align Agent* that was originally proposed was no longer required in the final implementation.

Client-Side Animation

Once the speech audio and viseme timeline had been generated, they were sent to the student's browser for rendering. The browser loaded the three-dimensional avatar model and played the generated audio while simultaneously applying the viseme information to animate the avatar's mouth movements. Because the animation was performed on the client side, students could view the lesson directly within the web application without requiring an external video streaming service. The avatar's facial movements remained synchronized with the generated speech throughout the lesson, providing a smooth and interactive presentation of the educational content. The client-side rendering approach also allowed the system to maintain responsive playback while reducing dependency on external services. Combined with the *Explain Agent* and

Gemini Text-to-Speech, the Full Body Avatar mode provided an effective alternative for delivering personalized lessons through a realistic three-dimensional virtual teacher.

3.6.8 Comparison of Avatar Delivery Modes

The proposed adaptive learning system provided two different avatar delivery modes: AVTR-1 Streaming Avatar and Full Body Avatar. Although both modes presented the same personalized lesson content, they used different technologies for speech generation, lip synchronization, and avatar animation. Providing two delivery modes increased the flexibility of the system and allowed it to operate in different deployment environments. The AVTR-1 Streaming Avatar relied on an external avatar-rendering service. The lesson explanation was sent to the service through a WebRTC connection, where speech synthesis, facial animation, and lip synchronization were performed. The generated avatar was then streamed back to the student’s browser in real time. This approach produced realistic facial expressions and smooth animations while supporting live playback controls such as pause, resume, and playback speed adjustment.

In contrast, the Full Body Avatar performed speech generation and animation within the frontend application. The explanation text was converted into speech using Gemini Text-to-Speech, while Rhubarb Lip Sync generated the lip synchronization information. The three-dimensional avatar was then animated directly in the student’s browser without requiring an external streaming service. This reduced external dependencies while still providing an interactive learning experience.

Although both approaches provided personalized lesson delivery, each had its own advantages. The AVTR-1 mode offered highly realistic streaming animation with advanced facial motion generation, making it suitable for immersive learning experiences. However, it depended on an external avatar-rendering service and required a stable network connection. The Full Body Avatar mode was simpler to deploy because speech generation and animation were handled within the application itself. It also reduced dependency on external services while maintaining synchronized speech and lip movements. Table 3.1 presents a comparison of the two avatar delivery modes implemented in the proposed system.

Table 3.1: Comparison of Avatar Delivery Modes

Feature	AVTR-1	Full Body Avatar
Speech Generation	External avatar-rendering service	Gemini Text-to-Speech
Communication Method	WebRTC	Next.js API
Lip Synchronization	AVTR-1 generative model	Rhubarb Lip Sync
Avatar Rendering	External streaming service	Client-side browser rendering
Streaming Support	Real-time live streaming	Local playback after audio generation
External Dependency	Required	Not required
Playback Controls	Pause, Resume, Speed Control	Standard browser playback
Personalization	Supported	Supported

3.6.9 Deprecated Avatar Pipeline

During the early development stages of the project, an additional avatar generation approach was implemented as an experimental solution. This approach used an offline avatar rendering pipeline that generated a complete lip-synchronized video before presenting it to the student. The implementation was based on a separate processing service (`avatar_service.py`) that performed avatar rendering through a GPU-accelerated process. In this approach, the complete lesson content was processed first, and the system generated a full video containing the avatar's speech, facial movements, and lip synchronization. After the rendering process was completed, the generated video could be played back by the student. This method was suitable for fixed lesson content where the video could be prepared before the learning session.

However, this approach became unsuitable after introducing personalized and adaptive lesson generation into the system. In the proposed learning platform, lesson explanations were dynamically generated based on each student's knowledge level, quiz performance, and learning requirements. Since the final lesson content was determined only when a student requested a lesson, generating complete videos in advance was not practical. The offline rendering approach also introduced additional limitations, including longer waiting times before lesson playback, higher computational requirements, and reduced flexibility when lesson content needed to be changed. Any modification to the generated explanation required the entire video rendering process to be repeated.

Due to these limitations, the offline avatar pipeline was replaced by the two current avatar delivery approaches: AVTR-1 Streaming Avatar and Full Body Avatar. These approaches were more suitable for the adaptive learning environment because they supported dynamic content generation and provided faster lesson delivery. Although the deprecated pipeline was no longer used in the final system, the implementation remained within the project source code as a reference for the development process. The transition from offline video generation to real-time avatar delivery improved the system's ability to provide personalized, interactive, and responsive learning experiences.

3.7 Teacher Dashboard and Learning Analytics

The teacher dashboard is implemented as a role-gated administrative panel: on load, the system checks the authenticated user's role and redirects any non-administrator away from the page, ensuring the analytics view is only accessible to teaching staff. The interface is organized around two coordinated panels. The left panel presents a searchable roster of all registered students, filterable by name or email, allowing a teacher to quickly locate a specific learner in a large class. The right panel displays that student's learning analytics once selected, alongside four summary statistics at the top of the page: total registered students, the currently selected student, their active subject, and their overall completion percentage.

Selecting a student loads two things: the list of subjects that student has engaged with, presented as colour-coded subject chips, and an improvement trend panel summarizing the change between pre-quiz and post-quiz scores across all completed topics, including an average-improvement statistic and a per-topic breakdown showing each topic's initial and final marks with a coloured delta badge (green for improvement,

red for decline). Selecting a specific subject then loads a lesson-progress summary (lessons completed out of the total available, shown with an animated progress bar) and a list of every topic the student has attempted within that subject.

Each topic in this list can be expanded to reveal a detailed, per-topic record:

- **Quiz performance** - the student's initial and final quiz marks (out of ten), each shown with its own progress indicator, alongside the topic's assessed mastery level (Beginner, Intermediate, or Advanced) and whether the topic has been unlocked for the student to proceed.
- **Delivered lesson content** - the actual personalized lesson text that was generated and shown to that student for that topic, allowing a teacher to review exactly what the student was taught.
- **Engagement sessions** - for each lesson session, the average, minimum, and maximum engagement score recorded by the Engagement Monitor (Section 3.13) during that session, together with a compact timeline chart plotting engagement score across the duration of the session, letting a teacher see not just whether a student was engaged on average but when their attention dropped during the lesson.
- **YouTube watch history** - supplementary video resources associated with the topic that the student watched, with the watched duration recorded per video.
- **Student Q&A history** - a log of the questions the student asked the AI teaching avatar while studying the topic, together with the answers the system gave, giving teachers visibility into what students are actually asking about and where confusion persisted beyond the quiz results.

On the backend, this view is served by eight dedicated REST endpoints (registered student list, per-student subjects, per-subject lesson progress, per-topic details, pre/post-quiz improvement, engagement history, YouTube watch history, and Q&A history), backed by the Analytics and *Dashboard Agent* together with three additional data collections - engagement sessions, question-and-answer logs, and video watch sessions - beyond the core student-progress and delivered-content collections that drive the adaptive learning engine. This gives the dashboard a materially richer analytics surface than the 'quiz performance, topic-wise strengths, engagement metrics' description in the original proposal: it combines assessment data, content-delivery records, attention/engagement telemetry, supplementary resource usage, and conversational interaction logs into a single per-student, per-topic view.

Architecturally, the dashboard is a read-time aggregation layer rather than a separate analytics pipeline: no data is duplicated, pre-computed, or written specifically for reporting purposes. Each underlying event - a quiz result, an engagement reading, a YouTube watch session, a question asked of the avatar - is written to its own collection by whichever component produced it, at the moment it happens, entirely independently of the dashboard. The Analytics and *Dashboard Agent*'s role is purely to read across these collections on request and compute the derived values a teacher sees (completion percentages, score deltas, session averages) at query time rather than ahead of time. This keeps the dashboard consistent with the live system by construction - there is no separate data store that could fall out of sync with what a

student actually experienced - at the cost of every aggregate being recomputed on each request rather than cached, which is an acceptable trade-off at the scale of a single class or school but would need revisiting (e.g. pre-aggregated summary documents) if the system were extended to many concurrent teachers monitoring large student populations.

3.8 Evaluation Strategy

To determine the overall effectiveness and reliability of the proposed intelligent tutoring system, a comprehensive evaluation strategy was designed. This strategy focused on measuring both the educational impact on students and the technical performance of the AI-driven components. The evaluation was conducted using the following key criteria:

3.8.1 Learning Gain Analysis

The primary objective of any educational system is to improve student knowledge. To measure this improvement quantitatively, the system conducted a learning gain analysis by comparing student performance before and after using the platform. For each lesson, students completed a pre-quiz to establish their baseline knowledge, followed by a post-quiz after they engaged with the personalized learning materials. The difference between these two scores served as a direct indicator of the student's academic progress and helped determine how effectively the system's adaptive content contributed to actual learning.

3.8.2 Adaptivity and Knowledge Estimation Accuracy

A core feature of the proposed system is its ability to personalize content based on individual student needs. Evaluating this capability involved assessing how accurately the *Personalization Agent* and the hybrid BKT-LSTM models estimated a student's mastery level. The evaluation analyzed whether the system correctly assigned students to appropriate difficulty levels (Beginner, Intermediate, or Advanced) and whether the subsequent questions and remedial content generated by the system genuinely matched the student's current understanding. High accuracy in this area ensured that students were neither overwhelmed by overly complex material nor bored by content that was too simple.

3.8.3 Avatar and Speech Interaction Quality

Since the system utilized an AI teaching assistant to deliver conversational explanations, the quality of this interaction was critical to student engagement. The evaluation measured the speech interaction accuracy by assessing the naturalness, clarity, and synchronization of the AI avatar. This included reviewing how accurately the generated Sinhala text was converted into speech, how well the avatar's lip movements aligned with the audio, and whether the overall conversational style successfully mimicked a real teaching environment. These technical metrics helped validate the effectiveness of integrating real-time avatars into educational applications.

3.8.4 User Satisfaction and Usability

Beyond technical and academic metrics, it was essential to understand the end-user experience from both the student and teacher perspectives. User satisfaction surveys and usability testing sessions were conducted to gather qualitative feedback. Students were asked about the system's ease of use, the clarity of the AI-generated explanations, and their overall engagement with the avatar. Similarly, teachers provided feedback on the usefulness of the analytics dashboard, the accuracy of the progress tracking features, and how well the system supported their classroom objectives. This human-centric evaluation provided valuable insights into the practical adoption of the system in real-world learning environments.

3.9 Ethical Considerations

The implementation of an AI-driven educational platform necessitates strict adherence to ethical guidelines, particularly concerning student data privacy, fairness, and transparency. The system addresses these considerations across several key dimensions:

3.9.1 Data Privacy and Security

Safeguarding student information is a primary concern. All data interactions between the mobile application, backend services, and cloud databases are secured using industry-standard encryption protocols. User authentication is strictly enforced, and role-based access control (RBAC) ensures that only authorized personnel, such as verified teachers and administrators, can access sensitive student performance metrics. Furthermore, any data utilized for training or refining the cognitive models (like BKT) is strictly anonymized, stripping personally identifiable information (PII) to protect student identities.

3.9.2 Fairness and Bias Mitigation

AI models, including the LLMs used for content generation and the Knowledge Tracing algorithms used for assessment, can inadvertently perpetuate biases. To mitigate this, the system's evaluation and knowledge tracing processes are continuously monitored for fairness. The capability clustering generated by PC-BKT is designed to be purely based on academic performance and interaction metrics, ensuring equitable treatment. Additionally, the instructional content generated in Sinhala undergoes rigorous prompt engineering and validation to prevent the introduction of cultural, gender, or linguistic biases.

3.9.3 Transparency and Explainability

A common criticism of AI in education is the opaque nature of its decision-making. This system counters that by emphasizing explainability. The teacher dashboard provides clear, data-backed rationales for the system's adaptivity—such as displaying the specific probability scores ($P(Know)$) that led to a student's mastery classification or the reasoning behind a remedial module recommendation. This transparency ensures that educators can understand, trust, and, if necessary, override the system's automated decisions.

3.9.4 AI as an Educational Aid

The system is fundamentally designed to act as an intelligent assistant rather than a replacement for human educators. While the AI teaching avatar handles personalized tutoring, continuous assessments, and content delivery, the ultimate pedagogical authority remains with the teacher. The platform aims to augment the teacher’s capabilities by automating analytics and routine tutoring tasks, thereby freeing up valuable time for high-level, personalized, and empathetic instructional interventions.

3.10 Personalization and Adaptive Learning

This section describes the design, implementation, and evaluation of personalization and adaptation strategies for lesson delivery in the AI-Based Sinhala Assistant will be covered in this section. It uses a Hybrid-BKT approach that includes PC-BKT together with LSTM-based deep learning techniques. These two methods help in identifying the individual performance, speed, and interaction pattern of students, based on which personalized lessons are provided by this application.

3.10.1 System Architecture for Personalized Learning

As illustrated in Figure 3.6, personalization engine is structured on a hierarchical architecture that consists of the frontend assessment interface (Next.js), the backend orchestrator (FastAPI + LangGraph), and the Hybrid-BKT inference engine with MongoDB as the state persistence layer. This architecture adheres to the "Backbone + Head" approach wherein PC-BKT represents the pedagogically interpretable backbone, while LSTM forms the temporal prediction head.

The architecture integrates the following key components:

- **Multi-Agent Orchestrator (LangGraph):** StateGraph-based supervisor routing student data into evaluation, content creation, and progress tracking agents.
- **Frontend Quiz Engine:** Provides pre-quiz and post-quiz assessments, records students’ responses, and presents adaptive feedback such as BKT mastery level.
- **Hybrid-BKT Inference Engine:** Core of the personalization engine with PC-BKT and LSTM models for mastery predictions.
- **RAG-Integrated Content *Creation Agent*:** Creates personalized lesson content using RAG based on the student mastery level.
- **MongoDB Persistence Layer:** Stores BKT states, capability matrices, LSTM hidden states, student progress, and interaction logs.

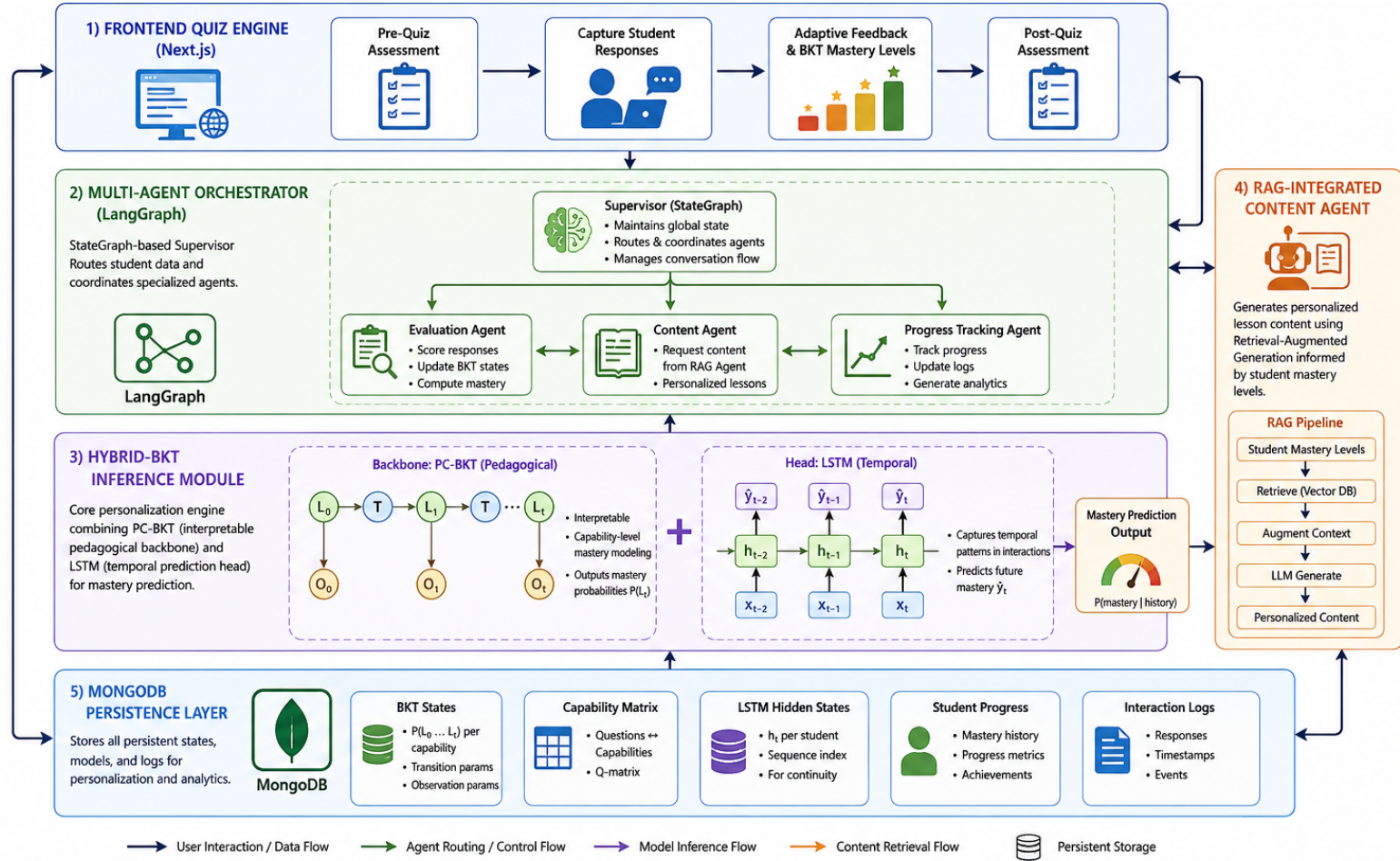


Figure 3.6: System Architecture for Personalized Learning

3.10.2 Research Foundation

The personalization methodology adopted in this research is grounded in two peer-reviewed studies that were analyzed, implemented, and extended to develop the proposed Hybrid-BKT personalization framework. As illustrated in Figure 3.6, the system combines interpretable probabilistic student modeling with temporal sequence learning to improve adaptive educational content generation and mastery prediction.

PC-BKT Model

The Personalized Clustered Bayesian Knowledge Tracing (PC-BKT) algorithm developed by Nedungadi and Remya [51] is a novel approach that modifies the original Bayesian Knowledge Tracing algorithm by incorporating personalized and clustering approaches.

These are the three main improvements of the PC-BKT model:

1. **Personalized Initial Mastery Priors:** The initial mastery probability $P(L_0)$ is calculated individually for each student and each knowledge component using Correct First Attempt (CFA) analysis instead of assigning a global default probability.
2. **Capability Matrix-Based Clustering:** A capability matrix $B(N \times K)$ is constructed to represent each student's performance across all Knowledge Components (KCs). The matrix stores the proportion of correct responses for each skill, enabling K-Means clustering of learners into ability groups such as High, Medium, and Low performers.
3. **Empirical Probability Estimation:** Instead of using manually predefined transition and error probabilities, the model estimates:

$$P(\textit{Guess}), \quad P(\textit{Slip}), \quad P(\textit{Transit})$$

empirically from annotated student interaction sequences.

Such improvements enable the use of PC-BKT to offer a more customized and comprehensible mastery estimation in comparison to ordinary BKT models.

BKT-LSTM Model

The BKT-LSTM model proposed by Sein Minn [52] reformulates student learning prediction as a temporal sequence forecasting problem by integrating Bayesian Knowledge Tracing outputs with Long Short-Term Memory (LSTM) neural networks. At each interaction timestep t , the model constructs an enriched feature vector:

$$f_t = [P(L)_{BKT}, \textit{Difficulty}, \textit{Correctness}, \textit{Attempt Count}]$$

The most recent 10 conversations between students are then fed into the stacked LSTM framework through a sliding window.

This LSTM framework exploits temporal learning behavior that cannot be easily captured by standard probabilistic models, such as:

- Learning plateaus

- Forgetting effects
- Long-term temporal dependencies
- Cross-skill knowledge transfer patterns

This will allow the model to generate more contextually and temporally aware mastery predictions.

Hybrid Ensemble Approach

The designed system leverages the benefits of both the PC-BKT and BKT-LSTM frameworks by applying a weighted ensemble approach, which can be mathematically represented as follows:

$$P_{Hybrid} = 0.65 \times P_{PC-BKT} + 0.35 \times P_{LSTM}$$

The use of the ensemble weights allows the system to retain pedagogical interpretability via structural reasoning provided by PC-BKT and benefit from temporal pattern recognition capabilities of LSTMs at the same time. From experimental results, the designed hybrid model managed to achieve the most balanced performance, reaching an accuracy level of 56.2% and 0.49 RMSE, which is significantly better than the performance of either framework alone.

3.10.3 Personalization Mechanisms

The presented personalization methodology attempts to surpass the conventional one-size-fits-all model of education by incorporating changes in instruction based on the learner's behavior, ability, and past success. As illustrated in Figure 3.7, the personalization strategy utilizes student clustering, personalized rate of learning, empirical parameter estimation, and LSTM time-series prediction to produce personalized estimates of skill acquisition.

Student Clustering and Capability Matrix

Capability Matrix $B(N \times K)$ is constructed, wherein each value of B_{ik} denotes the proportion of correct answers provided by student i to knowledge component k . The Capability Matrix includes the distribution of all students' performance with respect to all skills and is used as a basis for the identification of learners' abilities. The creation of the capability matrix is performed via the function that is implemented in `pc_bkt_engine.py`. Here, the function provides averaged grouped counts of correct answers for each student-skill combination. Following the construction of the matrix, K-means clustering technique is employed with `n_clusters=3` and `random_state=42` to make sure the reproducibility of the results.

The clustering procedure distinguishes between several ability groups, namely:

- High-performing learners
- Medium-performing learners
- Low-performing learners

Next, cluster-level mastery priors are determined using the `_compute_cluster_priors()` function, which determines the mean probability of mastery for each knowledge component within each cluster.

Cold Start Problem The cold start problem is one of the common issues that occur in intelligent tutoring systems. Rather than giving all new students the same initial value for their mastery level, such as:

$$P(L_0) = 0.3$$

The proposed model assigns a new learner to the closest ability cluster and then uses the corresponding cluster average prior as the prior probability of mastery. The result is that the "warm-start" technique provides much better predictions and, experimentally, decreases initial prediction errors by about 12%, compared to the standard BKT initialization approach.

Personalized Learning Rate $P(T)$

The traditional BKT assumes a universal learning rate for everyone. In reality, however, every learner has his or her own speed, and thus the proposed model estimates the personalized learning rate of each learner separately.

he personalized transition probability is calculated as:

$$P(T)_s = \frac{\sum_t (1 - K_{t-1}) K_t}{\sum_t (1 - K_{t-1})}$$

where K_t represents the annotated knowledge state at timestep t . The equation calculates the frequency with which a learner moves from not knowing a particular information state to a known information state. This is an indication of the pace of learning for each unique learner. To ensure that there are no fluctuations in the parameters used, the value is bound within:

$$P(T) \in [0.01, 0.5]$$

Empirical Parameter Estimation (EP)

Instead of predefining values for guessing and slipping probability manually, the system derives such values directly from actual data of interactions of students by implementing the Empirical Probability (EP) annotation approach.

Step 1 – Knowledge Annotation The system detects the most suitable learning point t^* for each student and knowledge component pair. Learning point t^* is chosen in such a way that incorrect answers mostly appear prior to t^* while correct answers mostly appear after t^* . All interactions that follow the detection of learning point t^* are labeled as:

$$K = 1$$

implying that it is taken for granted that the learner has mastered the knowledge component.

Step 2 – Estimation of the Parameters Following the annotation of knowledge components, the parameters are estimated using the formulas below:

$$P(Guess)_{kc} = \frac{Count(Correct \text{ AND } NotKnown)}{Count(NotKnown)}$$

$$P(Slip)_{kc} = \frac{Count(Incorrect \text{ AND } Known)}{Count(Known)}$$

To maintain stable parameter estimation, the probabilities are clipped to predefined ranges:

$$P(G) \in [0.01, 0.4]$$

$$P(S) \in [0.01, 0.3]$$

LSTM Feature Enrichment

The LSTM layer improves temporal forecasting with its ability to take a full multi-dimensional feature vector in each timestep of an interaction. The feature vector is constructed with the `_build_feature_vector()` function from `inference.py`.

These feature components include:

- **Mastery Vector:** Probability values $P(L)$ for each skill produced by the PC-BKT algorithm.
- **One-Hot Cluster Encoding:** Cluster encoding of the learner ability profile acquired using K-Means clustering.
- **Difficulty of Problem Encoding:** 10 levels of difficulty encoded using global success rates of problems.
- **Other Features:** Current correctness indicator and attempt ratio indicating learning behaviors of the learner.

The temporal prediction module is based on a two-layer stacked LSTM network, with 128 neurons per layer and a dropout probability of 0.3. The output from the LSTM layer is further fed into a fully-connected layer and then a sigmoid activation function for binary mastery classification.

The model parameters are tuned as follows:

- Loss Function: Binary Cross Entropy Loss (BCELoss)
- Optimizer: Adam with learning rate $lr = 0.005$
- Number of training epochs: 30
- Mini-batch size: 32

With this improved modeling capability, the system is able to encode complex behavioral dynamics like learning curves, learning plateaus, and forgetfulness, which cannot be easily modeled with conventional probabilistic approaches.

Bayesian Knowledge Tracing Update Equations

This new personalization framework will use Bayesian Knowledge Tracing (BKT) update equations to update the probabilities of student mastery for each interaction between the learner and the system. In doing so, it is able to estimate at any one time point the probability that a certain student had successfully mastered a particular knowledge component.

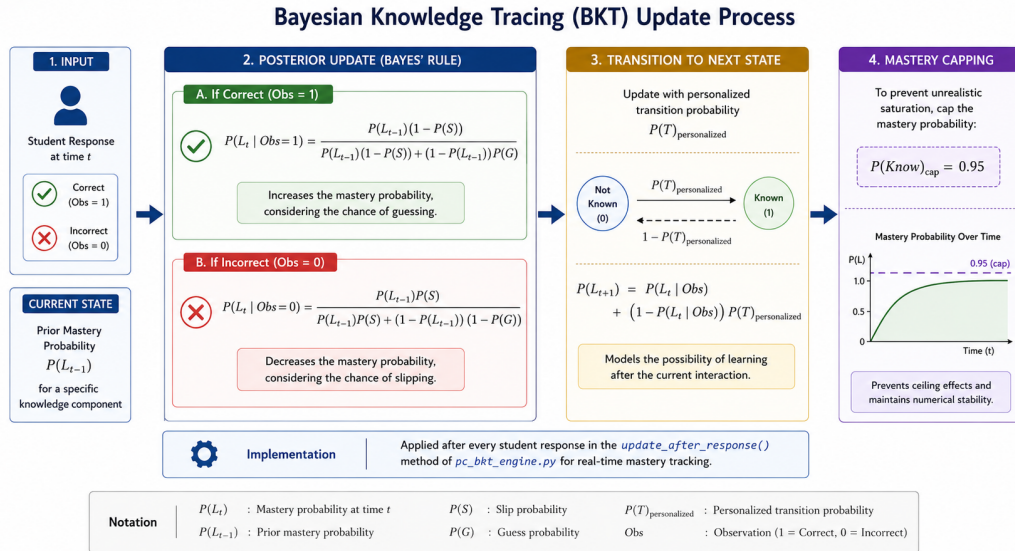


Figure 3.8: Bayesian Knowledge Tracing Update Process in the Proposed System

To do this, the probabilities are calculated for posterior mastery based on the correctness of the student's answer. As shown in Figure 3.8, the steps involved in updating include the estimation of posterior mastery, transition update, and mastery cap. If Correct:

$$P(L_t | Obs = 1) = \frac{P(L_{t-1})(1 - P(S))}{P(L_{t-1})(1 - P(S)) + (1 - P(L_{t-1}))P(G)}$$

If Incorrect:

$$P(L_t \mid Obs = 0) = \frac{P(L_{t-1})P(S)}{P(L_{t-1})P(S) + (1 - P(L_{t-1}))(1 - P(G))}$$

Transition to next state:

$$P(L_{t+1}) = P(L_t \mid Obs) + (1 - P(L_t \mid Obs))P(T)_{personalized}$$

In order to avoid unrealistic saturation problems during Bayesian updates, a maximum mastery probability of

$$P(Know)_{max} = 0.95$$

is assumed. Thus, it ensures that the model will not attain absolute confidence, and the mathematical stability of Bayesian updates can be guaranteed.

3.10.4 Adaptive Assessment and Content Delivery

The suggested approach for adaptive learning makes use of a multi-agent orchestration architecture to facilitate dynamic control of assessment and personalization processes, as well as the provision of content. The design of the system leverages a LangGraph **StateGraph** workflow that controls various specialized agents for evaluating, generating and tracking progress. With this system, continuous adaptation of learning materials and assessment behavior based on the learner’s performance becomes possible.

Mastery-Based Adaptive Thresholds

This model places the learner at one of four mastery levels based on the hybrid $P(Know)$, where varying methods of content adaptation will be applied according to those levels.

Table 3.2: Mastery levels based on hybrid $P(Know)$ score

Mastery Level	$P(Know)$ Threshold	Adaptive Strategy
Advanced	0.60 – 0.85	Reinforcement, standard practice (Bloom L2–L3)
Intermediate	0.35 – 0.60	Structured explanations, guided practice
Beginner	< 0.35	Foundational concepts, simplified, high-support

The selected level has a direct bearing on how the RAG prompt is generated, and therefore how difficult the language complexity is for the LLM.

The entire personalized learning process can be seen as a cycle comprising Pre-Quiz, → Lesson → Post-Quiz phases:

Phase 1 - Pre-Assessment: The student is administered a pre-quiz in a specified KC. The questions are dynamically generated through an LLM-based question generation pipeline which guarantees contextual and adaptive question generation consistent with Bloom’s taxonomy and target KCs. The responses are used to update the BKT model in real-time.

Phase 2 - Adaptive Content Generation: Based on the results obtained from the pre-quiz, the *Evaluator Agent* will categorize the student as beginner, intermediate, or advanced. The *Content Generator Agent* will then employ the RAG framework to generate a personalized lesson plan using mastery-aware prompts.

Phase 3 - Post Assessment: Following the study session, the learner is then asked to complete the post-assessment quiz. The system compares both pre- and post-levels of mastery. If $P(\text{Know}) > 0.85$, the learner is advised to move ahead. If $P(\text{Know}) < 0.60$, the system generates remedial material that addresses the weak topics.

Phase 4 - Continuous Adaptation: All user interactions are recorded in the system, and the capability matrix is updated accordingly. Once sufficient new data accumulates, the clustering algorithm may be retrained via the `/hybrid/train` endpoint, refining student ability profiles and improving prediction accuracy over time.

3.11 Data Collection

The core of the AI Teaching Avatar system relied on accurate, well-organized, and continuous data collection. The data strategy was specifically tailored for the Sinhala-medium education system in Sri Lanka to ensure high pedagogical relevance and accuracy, while also feeding robust datasets into the Hybrid-BKT and LSTM models.

3.11.1 National Curriculum Data Collection and OCR

The foundational data source was the Sinhala-medium national curriculum published by the Ministry of Education of Sri Lanka. The curriculum data was strictly organized according to the following hierarchy:

- Subject domains (e.g., Mathematics, Science, ICT)
- Education levels (G.C.E. O/L and A/L)
- Units, specific lessons, and targeted learning outcomes (Knowledge Components)

To efficiently ingest physical educational materials such as printed textbooks, teacher guides, and past examination papers, the system utilized OCR technology. These documents were scanned, processed through OCR to extract Sinhala text, and subsequently converted into structured formats (such as JSON files) for seamless integration into the MongoDB backend.

3.11.2 Digitization and Analysis of Sinhala Subject Content

Following OCR extraction, the raw text was digitized and analyzed using specialized Natural Language Processing (NLP) techniques tailored for the Sinhala language. This automated analysis included:

- Identifying and tagging key curriculum concepts and entities.
- Extracting critical definitions, theorems, and mathematical formulas.
- Identifying common examination question patterns from historical past papers.

- Processing and formatting the data into structured knowledge chunks that served as the knowledge base for the RAG pipeline.

This structured digitization process ensured that the AI teaching avatar could retrieve contextually accurate and pedagogically sound educational content for dynamic lesson generation.

3.11.3 Student Performance and Interaction Data Collection

Once the system was active, continuous telemetry data was collected during the student's learning process. This data was critical for driving the system's adaptive algorithms. The system captured granular interaction metrics, including:

- **Assessment Metrics:** Quiz scores, success/failure rates on specific Knowledge Components (KCs), and exact incorrect answers submitted.
- **Temporal Data:** Response times for individual questions, session durations, and the frequency of lesson revisits.
- **Behavioral Patterns:** The number of times a student requested AI-generated re-explanations or utilized voice-based interactions.

These interaction logs were asynchronously routed via the LangGraph orchestrator into the MongoDB persistence layer. From there, the data fed directly into the PC-BKT clustering algorithm to identify student capability levels and into the LSTM model to predict future performance trajectories.

3.11.4 Creation of Personalized Learning Plans

The aggregation of curriculum data and real-time student performance metrics formed the basis for automated personalized learning plans. By analyzing this comprehensive dataset, the AI system dynamically provided:

- Additional, simplified Sinhala explanations precisely targeted at identified weak KCs.
- Accelerated or advanced content pathways for fast learners clustered into high-performing groups.
- Timely revision modules and supplementary practice activities triggered by the LSTM model's predictive warnings.

This data-driven feedback loop ensured that the educational experience was continuously refined, offering each student an adaptive learning pathway tailored to their unique cognitive and behavioral profile.

3.12 Frontend Development and Implementation

3.12.1 Framework and Approach

To ensure that all users can easily access and interact with the system, two distinct frontend applications were developed: a mobile application and a web-based platform.

The web application is specifically designed for students, providing a highly accessible and engaging digital learning environment. It was developed using the Next.js framework paired with Tailwind CSS. Next.js ensures that the learning platform loads quickly and handles dynamic lessons efficiently, while Tailwind CSS allows for the creation of a modern, responsive, and visually appealing user interface that works well on computer screens of any size. On the other hand, the mobile application serves as a convenient dashboard for teachers and parents to monitor student progress on the go. It was built using React Native and the Expo framework, along with TypeScript. This combination was chosen because it allows the application to run smoothly on both Android and iOS devices using a single codebase. Furthermore, TypeScript helps catch errors early during development, making the code much more reliable and easier to maintain over time.

Because different users need to see different information, the entire frontend architecture is built around a role-based access control system. When a user logs in, the system checks their role and directs them to the appropriate interface with the right permissions. For instance, students use the web app to access their personalized lessons and quizzes, while teachers and parents use the mobile app to view analytics, learning trends, and overall student progress. Finally, to keep user sessions secure without requiring them to log in every time they open the platform, local data storage solutions were implemented. The mobile app uses AsyncStorage to securely save authentication tokens directly on the phone, while the web application relies on the browser's localStorage. This approach ensures a seamless and secure experience for everyone using the system.

3.12.2 Design and Prototyping

During the design phase, the main goal was to create a modern, user-friendly interface that feels welcoming and easy to navigate for everyone, whether they are students, teachers, or parents. Because students might spend a considerable amount of time reading lessons or taking quizzes, a dark-themed design was chosen. This dark mode helps reduce eye strain during long study sessions and gives the platform a sleek, professional appearance.

To make the platform visually engaging, a color palette featuring blue gradients was used for the main branding elements, which creates a calm and focused atmosphere. Additionally, secondary colors like green and purple were carefully used to separate different subject categories and highlight important buttons, making it easy for users to find what they need at a glance. The layouts were also customized based on the device being used. For the mobile application, which is mainly used by parents and teachers on the go, a simple bottom tab navigation system was implemented. This allows users to quickly switch between viewing analytics, checking student progress, and adjusting settings with a single tap. The buttons and menus were explicitly designed to be large and touch-friendly to improve overall usability.

For the student-facing web application, a persistent sidebar layout was chosen. This makes it incredibly easy for students to jump between their personalized lessons, the AI Avatar interface, and their quizzes without getting lost in multiple menus. Finally, to bring the entire learning experience to life, Lottie animations and modern icons were integrated throughout both platforms. These small, engaging animations provide friendly visual feedback when a user interacts with the app, making the

educational journey much more interactive and enjoyable.

3.12.3 UI Styling and Responsiveness

Because students, teachers, and parents access the platform using many different devices-rom large desktop monitors to small smartphone screens was crucial to make the design responsive and adaptable. The goal was to ensure that the interface always looks great and functions perfectly, regardless of the screen size.

For the student-facing web application, Tailwind CSS was chosen as the primary styling tool. Tailwind CSS makes it incredibly easy to build responsive designs because it allows developers to define specific rules for different screen sizes right inside the code. By using flexible box (Flexbox) layouts and predefined breakpoints, the web dashboard automatically resizes and rearranges its content. For example, a lesson module that displays side-by-side with the AI Avatar chat on a wide laptop screen will neatly stack vertically if a student views it on a smaller tablet screen.

For the mobile application used by teachers and parents, React Native's built-in StyleSheet system was utilized. This approach allowed for platform-specific adjustments, ensuring that the app looks and feels natural on both Android and iOS devices. Special care was taken to handle "safe areas" such as the camera notches or home indicator bars on modern smartphone-so that important analytics charts and navigation buttons are never hidden behind the physical hardware of the phone. Even though two entirely different technologies were used to build the web and mobile apps, a shared design system was strictly followed. This means that both platforms use the exact same color codes, spacing rules, and font styles. By keeping these visual elements consistent, the entire project feels like one unified, professional educational ecosystem, no matter how or where a user logs in.

3.12.4 Component Structure and Reusability

To keep the software well-organized and easy to update in the future, the frontend was built using a component-based approach. Instead of writing the same code over and over again, individual pieces of the user interface were built as independent, reusable building blocks called components.

For the teacher and parent mobile application, several distinct components were created. For example, student profile cards are used repeatedly to display different students in a class list. Themed text elements, navigation tabs, and animated scroll views were also built as reusable blocks. This means if the design of a button needs to change, it only has to be updated in one central file, and the change will automatically apply everywhere across the app. Similarly, the student-facing web application was assembled using reusable page layouts and dashboard widgets. Because students frequently interact with lessons, quizzes, and the AI Avatar, these features were built as modular pieces that can be easily plugged into any page without having to rewrite the core programming logic.

Beyond just the visual layouts, the underlying technical logic was also neatly organized. Important configuration detail-like color themes, backend server addresses, and security rules-were stored in dedicated configuration files. Additionally, custom helper functions (known as React Hooks) were created to handle repetitive tasks, such as switching visual themes or safely communicating with the backend database. This

structured approach makes the entire codebase much cleaner, highly efficient, and significantly easier for developers to maintain over time.

3.12.5 State Management

Managing the data inside the app-like whether a user is logged in or what they are currently typing was kept as simple as possible. Instead of using complex external libraries, we relied on standard React Hooks.

For the mobile app, we used `useState` to handle everyday things like form inputs and loading screens. To remember the user's login session even after they close the app, we saved their authentication data using `AsyncStorage`. The web app follows a similar straightforward approach. We used `useState` and `useEffect` for handling the current information on the screen, and stored login details using the browser's `localStorage`. For moving around the app, we let the frameworks do the heavy lifting: Expo Router handles navigation on mobile, and Next.js routing takes care of it on the web. Finally, our backend API serves as the central hub, ensuring that all information stays perfectly synchronized no matter which device the student or teacher is using.

3.12.6 Backend Communication

Our frontend applications need to talk to the backend to get data and save student progress. We built this communication bridge using REST API endpoints powered by FastAPI.

To give you an idea of how this works in our Sinhala Teaching AI AVATAR project, we created several specific paths. We have authentication paths for when a user logs in or signs up. We also have paths for the core learning activities, such as enrolling in courses, submitting a pre quiz, answering questions, and sending post quiz results. For teachers and parents using the mobile app, we included special dashboard paths that pull real time student details. How do the apps actually make these requests? In the mobile app, we used the built in Fetch API because it is lightweight and built right into React Native. For the web application, we chose Axios since it makes handling requests and reading data much easier.

We also confirm that the communication is secure and reliable. Whenever a user logs in, they receive an authentication token. The app saves this token and checks it every time it opens to keep the session secure. Finally, if there is a network error or the server sends an invalid response, our apps catch these issues smoothly without crashing.

3.12.7 Testing and Debugging

We tested and fixed issues constantly while building the project to make sure everything worked smoothly. For the student web app, we relied on browser developer tools and React DevTools to catch visual and logical errors. For the mobile app, we used the Expo CLI along with Android emulators and real physical devices to see exactly how teachers and parents would experience it.

Since the Sinhala Teaching AI AVATAR project involves many moving parts, we carefully checked specific features. We ensured that users could log in without problems, navigate between lessons and dashboards easily, and send quiz answers to the backend reliably. We also tested the layout on different screen sizes to ensure it

always looked right. To figure out what went wrong when something broke, we used simple console logging, watched the network requests, and used breakpoint debugging to pause the code and inspect it. Finally, to keep our code clean and easy to read, we used ESLint in both the Next.js web app and the React Native mobile app. This tool helped us follow good coding rules and catch small mistakes early.

3.13 Engagement Detection System

To ensure students remain focused and receive the maximum benefit from the personalized learning experience, the platform integrates a real-time Engagement Detection System. This subsystem continuously evaluates the student's attentiveness and affective state during a lesson by analyzing visual and behavioral cues through the webcam.

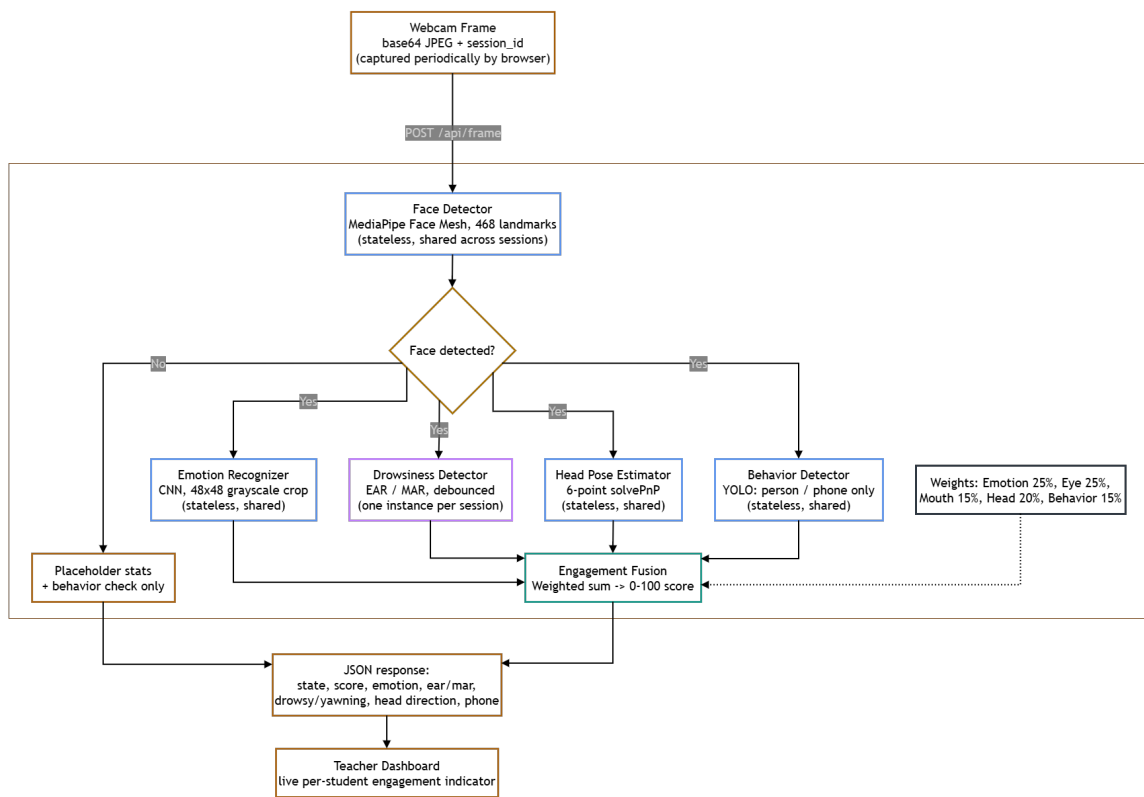


Figure 3.9: Engagement Detection System Architecture

3.13.1 Feature Extraction and Metrics

The engagement engine computes an overall engagement score by aggregating five distinct metrics, each capturing a different aspect of the student's physical state. The extraction of these features relies on facial landmark detection and behavioral pattern analysis [53].

- **Emotion Analysis:** The system classifies the student's current facial expression into discrete emotional states. Each emotion is mapped to an engagement weight based on its pedagogical implication: Happy (1.0) and Neutral (0.7) indicate positive or focused states; Surprise (0.6) suggests active processing; while negative

states such as Sad (0.3), Fear (0.2), Angry (0.2), and Disgust (0.1) indicate low engagement or frustration. Facial emotion recognition is commonly used for analysing affective states from visual information [54].

- **Drowsiness Detection (Eye Aspect Ratio - EAR):** The engine monitors eye movements to detect signs of drowsiness. The EAR is calculated to determine how open the eyes are. If the EAR falls below 0.20, it indicates closed eyes or severe drowsiness, yielding a score of 0.0. An EAR between 0.20 and 0.25 indicates partial drowsiness (score of 0.4), while a value above 0.25 represents alertness (score of 1.0). The Eye Aspect Ratio is a widely used method for detecting eye closure and driver/student fatigue states from facial landmarks.
- **Yawning Detection (Mouth Aspect Ratio - MAR):** Yawning is a strong indicator of fatigue and disengagement. The system calculates the MAR to assess the degree of mouth opening. An MAR exceeding 0.6 strongly suggests yawning, dropping the metric score to 0.0, whereas values below 0.6 indicate a normal state (score of 1.0). Mouth landmark-based analysis is commonly applied for yawning and fatigue detection.
- **Head Pose Analysis:** The orientation of the student's head (pitch, yaw, and roll) is evaluated to ensure they are looking directly at the educational content. This generates a normalized head score reflecting visual attention. Head pose estimation is commonly used in computer vision applications to determine user attention and viewing direction [55].
- **Behavioral Analysis:** In addition to facial metrics, general behavioral patterns—such as excessive movement or prolonged static postures—are quantified into a behavioral score, ensuring a holistic view of the student's physical activity.

3.13.2 Weighted Fusion Mechanism

Once the individual metrics are extracted and normalized to a scale of 0.0 to 1.0, they are combined using a weighted fusion algorithm. This approach prioritizes certain features over others based on their correlation with actual cognitive engagement. Multi-feature fusion approaches are commonly used in affective computing systems to improve reliability by combining multiple behavioural indicators [56].

- Emotion: 25%
- Eye (Drowsiness): 25%
- Head Pose: 20%
- Mouth (Yawning): 15%
- Behavior: 15%

The final aggregated value is multiplied by 100 to produce a percentage-based engagement score ranging from 0 to 100.

3.13.3 Engagement Classification

The continuous engagement score is categorized into four distinct states to provide actionable feedback for the personalization engine and the teacher dashboard:

1. **Highly Engaged (Score > 75):** The student is alert, displaying positive emotions, and maintaining strong visual focus.
2. **Moderately Engaged (Score > 50):** The student is generally focused but may show slight signs of distraction or neutral affect.
3. **Low Engagement (Score > 30):** The student is showing clear signs of fatigue, such as partial eye closure or negative emotions.
4. **Not Engaged (Score $\leq$ 30):** The student is severely distracted, yawning, or physically turning away from the interface.

These real-time classifications allow the AI Avatar to proactively intervene—for example, by pausing the lesson, asking a checking question, or adjusting the pacing—if the student’s engagement drops below critical thresholds. Additionally, these metrics are logged and visualized in the teacher dashboard, as discussed in Section 3.7.

3.14 Model Fine-Tuning: Task-Specific Fine-Tuning Iterations

3.14.1 Selection of the Base Language Model

The selection of an appropriate base language model was one of the most important decisions made during the development of the intelligent tutoring system. Since the objective of this research was to build an AI-powered educational platform that can understand Sinhala instructions, answer educational questions, generate quizzes, explain lessons, and personalize learning content, the foundation model needed to demonstrate strong language understanding, reasoning ability, and instruction-following performance. Rather than choosing a model based only on its popularity or model size, several candidate LLMs were evaluated to determine which one was most suitable for the educational tasks required by the system.

Five pre-trained language models were selected for evaluation: SinLlama (8B), Qwen (14B), Llama-3.1 (8B), DeepSeek (14B), and Mistral (7B). Each model was tested using the same educational question-answering dataset so that their performance could be compared under identical conditions. The evaluation focused on how accurately each model understood Sinhala instructions, how well it generated educational responses, and how consistently it produced correct and meaningful answers. Since the system was designed to support Sinhala-medium students, the selected model also needed to demonstrate good performance when processing Sinhala text and educational terminology.

To ensure a fair comparison, each candidate model was evaluated using the same prompts, the same educational questions, and the same reference answers. The generated responses were assessed using several automatic evaluation metrics together with qualitative analysis. Metrics such as ROUGE [57] measured the overlap between

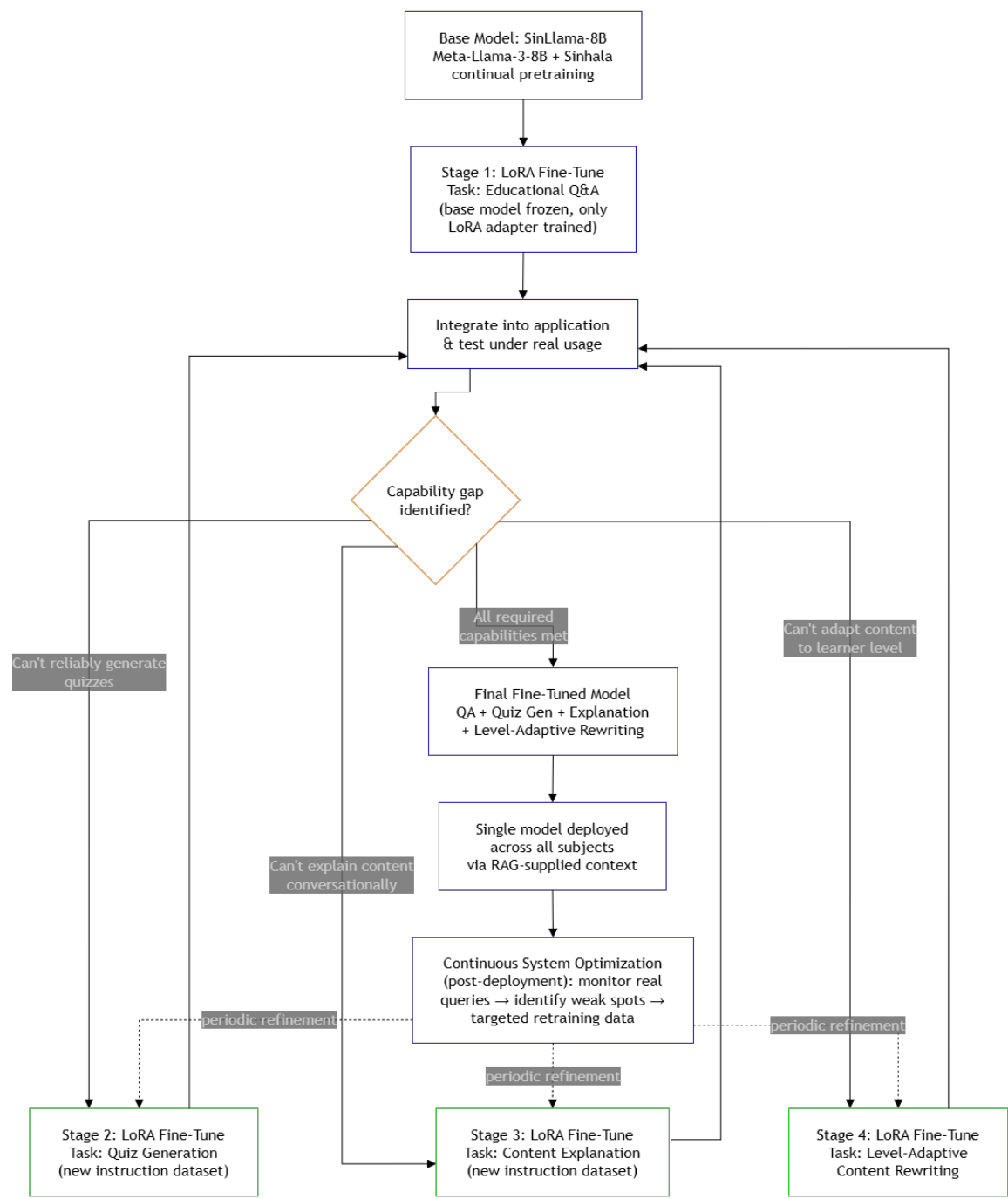


Figure 3.10: Model Fine-Tuning Process

the generated answers and the reference answers, while BERTScore [58] evaluated semantic similarity by measuring how closely the generated responses matched the intended meaning of the reference text. In addition to these automated metrics, an LLM-as-a-Judge [59] evaluation was performed to assess the overall quality, correctness, clarity, and usefulness of the generated responses. Using multiple evaluation methods provided a more comprehensive assessment than relying on a single evaluation metric.

The evaluation results showed that SinLlama [7] achieved the highest overall performance among the five candidate models. It consistently generated more accurate answers, followed Sinhala instructions more reliably, and produced responses that were clearer and more relevant to the educational context. Compared with the other evaluated models, SinLlama demonstrated a better understanding of Sinhala grammar, vocabulary, and sentence structure. It also generated responses that required fewer corrections and contained fewer factual or formatting errors. These characteristics made SinLlama the most suitable choice for the intelligent tutoring system.

Another important factor that influenced the selection was the model’s ability to adapt through fine-tuning. Since the final system required several specialized capabilities beyond simple question answering, the selected model needed to learn new instruction-following behaviours efficiently. Preliminary experiments indicated that SinLlama responded well to parameter-efficient fine-tuning techniques and maintained stable performance after training. This flexibility was considered an important advantage because the model would later be fine-tuned multiple times for additional educational tasks such as quiz generation, lesson explanation, and personalized content rewriting.

Computational efficiency was also considered during the model selection process. Larger language models generally require more GPU memory, longer training times, and greater computational resources. These requirements can become challenging within the limited hardware resources available for an undergraduate research project. SinLlama provided a good balance between model capability and computational efficiency. It was able to produce high-quality educational responses while remaining practical to fine-tune using the available GPU resources. This made it possible to complete multiple rounds of fine-tuning without requiring expensive computing infrastructure.

Another advantage of selecting SinLlama was its compatibility with modern parameter-efficient training techniques such as LoRA [60] and optimized training frameworks such as Unsloth. These tools allowed the model to be adapted to new educational tasks without modifying every parameter of the original language model. As a result, the research could focus on improving task-specific performance while keeping training costs manageable.

The decision to select SinLlama also influenced the overall architecture of the intelligent tutoring system. Once the model had been selected, all subsequent stages of development, including dataset preparation, instruction tuning, evaluation, and deployment, were built around this single foundation model. Instead of maintaining multiple language models for different educational tasks, one continuously improved SinLlama model was used throughout the system. This simplified the system architecture, reduced storage requirements, and ensured consistent behaviour across different learning features.

It is important to note that the selection of SinLlama was based on experimental evaluation rather than assumption. Every candidate model was assessed under

the same testing conditions, allowing the comparison to be objective and repeatable. Selecting the best-performing model before beginning fine-tuning reduced the likelihood of training a model that was fundamentally less suitable for Sinhala educational applications. This also ensured that subsequent fine-tuning efforts focused on enhancing an already capable model instead of compensating for weaknesses in the underlying language model.

Based on the evaluation results, SinLlama was selected as the base language model for the remainder of this research. All later fine-tuning stages described in this chapter, including question answering, quiz generation, lesson explanation, and level-adaptive content rewriting, were performed using this model. By choosing the strongest-performing model at the beginning of the development process, the research established a reliable foundation for building an intelligent tutoring system capable of supporting multiple educational tasks across different school subjects.

3.14.2 Base Fine-Tuning for Educational Question Answering

After selecting SinLlama as the most suitable base language model, the next stage of the research focused on fine-tuning the model for educational question answering. Although the original pre-trained model possessed general language understanding and reasoning capabilities, it had not been specifically trained to answer educational questions in Sinhala. Therefore, an additional training process was required to adapt the model to the educational domain and improve its ability to generate accurate, context-aware responses for students.

The primary objective of this fine-tuning stage was to teach the model on how to understand an instruction, process the educational context provided by the system, and generate a correct answer in Sinhala. At this stage, the model was not expected to generate quizzes, explain lessons, or rewrite educational content for different learner levels. Instead, the focus was entirely on developing a strong and reliable question-answering capability that could serve as the foundation for all later system features.

To achieve this objective, a large instruction-tuning dataset was created using educational content collected from several school subjects. The dataset included examples from G.C.E. O/L Buddhism, Science, Mathematics, History, G.C.E. A/L Economics and Biology. These subjects were selected because they represent different types of educational knowledge, including factual information, conceptual explanations, numerical reasoning, and historical events. By training on multiple subjects rather than a single domain, the model could develop a broader understanding of educational content and become capable of supporting different lessons within the intelligent tutoring system.

Each training example followed a structured instruction-response format. Every sample consisted of three main components: an instruction, an educational context, and the expected answer. The instruction represented the student's question or task, while the context contained the relevant textbook information needed to answer that question. The target response was the correct answer written in Sinhala. This instruction-based training format closely matched how the model would later receive requests during actual system operation. As a result, the model learned not only factual knowledge but also how to follow instructions and generate responses in the desired format.

Using an instruction-based dataset offered several advantages over traditional

language model training. Instead of simply predicting the next word in a sentence, the model learned to recognize different types of educational instructions and produce responses that directly addressed the student’s request. This improved the model’s ability to follow prompts consistently and generate answers that were relevant to the provided educational context rather than relying solely on information stored within the pre-trained model. The complete dataset contained approximately 13,000 carefully prepared instruction-response examples. Before training, the dataset was reviewed to improve consistency in formatting and remove unnecessary variations that could confuse the learning process. Maintaining a consistent instruction format throughout the dataset helped the model recognize common instruction patterns and improved its ability to generalize to similar questions during inference.

During fine-tuning, the model was trained for 15 epochs, allowing it to process the entire dataset multiple times. Repeating the training over several epochs enabled the adapter parameters to gradually learn the relationships between educational questions, supporting context, and correct answers. The number of epochs was selected as a balance between providing sufficient learning opportunities and avoiding excessive training that could increase the risk of overfitting. Throughout the training process, the model gradually improved its ability to understand educational instructions and generate more accurate responses.

Unlike many machine learning workflows, this research did not reserve a separate validation dataset during the initial fine-tuning stage. Instead, all available educational examples were used for training. This decision was made because the dataset already covered several different school subjects. Dividing the dataset further into training and validation subsets would have reduced the number of examples available for each subject, limiting the amount of information the model could learn from during training. Given the relatively moderate size of the dataset, maximizing the available training data was considered more beneficial for the objectives of this research.

Although this approach allowed the model to learn from every available example, it also introduced an important limitation. Since no validation dataset was used during training, model checkpoints could not be selected based on validation loss or unseen examples. Instead, the final model checkpoint was selected after completing the planned number of training epochs. Consequently, the model’s ability to generalize to new data could not be measured during training itself. To address this limitation, the completed model was evaluated after training using several automatic evaluation techniques, including ROUGE, BERTScore, and LLM-as-a-Judge, to assess response quality and overall performance.

After completing the fine-tuning process, SinLlama demonstrated a significant improvement in educational question answering compared with the original pre-trained model. It was able to understand Sinhala educational questions more accurately, identify the relevant information within the provided context, and generate clear, grammatically correct answers. The responses also became more consistent in style and followed the instruction format learned during training. These improvements made the model suitable for serving as the core question-answering component of the intelligent tutoring system.

However, practical testing within the complete application revealed that strong question-answering performance alone was not sufficient for the system’s overall objectives. While the model successfully answered direct questions, it struggled when asked to perform more complex educational tasks such as generating structured multiple-

choice quizzes, transforming textbook content into conversational explanations, or rewriting lessons for learners with different levels of prior knowledge. These tasks required different instruction-following behaviours that were not represented in the original question-answering dataset.

This observation highlighted an important characteristic of instruction tuning. A model trained to answer questions does not automatically acquire the ability to perform other educational tasks, even when those tasks use the same underlying knowledge. Different tasks often require different output structures, writing styles, and reasoning processes. As a result, additional task-specific fine-tuning stages became necessary to teach these new capabilities.

The successful completion of the base question-answer fine-tuning stage established a strong foundation for the remainder of the research. It provided the model with reliable educational question-answering abilities while preserving the general language understanding learned during pre-training. More importantly, it created a stable starting point upon which additional capabilities could be added through subsequent fine-tuning iterations. Rather than attempting to teach every required behaviour in a single training stage, the research adopted a gradual and systematic approach, where each new capability was introduced through its own dedicated instruction dataset. This iterative strategy ultimately resulted in a more reliable, flexible, and accurate language model capable of supporting the diverse educational functions required by the intelligent tutoring system.

3.14.3 Parameter-Efficient Fine-Tuning Using LoRA

After establishing the base question-answering capability of SinLlama, the next important decision was selecting an appropriate fine-tuning method. LLMs contain billions of parameters that have been learned during pre-training on massive text datasets. Updating every parameter of such a model, commonly known as full fine-tuning, requires a large amount of GPU memory, long training times, and powerful computing hardware. These requirements are often beyond the resources available for academic research projects. Therefore, this research adopted a more efficient approach known as Low-Rank Adaptation (LoRA).

LoRA is a Parameter-Efficient Fine-Tuning (PEFT) technique that enables a pre-trained language model to learn new tasks without modifying all of its original parameters. Instead of updating the complete model, LoRA introduces a small number of additional trainable matrices, called adapter layers, into selected parts of the transformer architecture. During training, the original model weights remain frozen, while only these newly added adapter parameters are updated. This allows the model to acquire new knowledge and behaviours while preserving the information already learned during its original pre-training.

The main advantage of LoRA is that it significantly reduces the number of trainable parameters. Since only a small fraction of the model is updated, the computational cost of training becomes much lower than full fine-tuning. This reduction in trainable parameters decreases GPU memory usage, shortens training time, and reduces the storage space required for saving the fine-tuned model. These advantages make LoRA particularly suitable for research environments where computing resources are limited.

In this project, LoRA adapters were inserted into both the attention mechanism and the feed-forward network of the SinLlama model. Specifically, adapter layers

were added to the query (Q), key (K), value (V), and output (O) projection layers within the self-attention mechanism. These projection layers play a crucial role in determining how the model processes relationships between different words in an input sequence. By updating these layers through LoRA adapters, the model learns to pay greater attention to task-specific patterns while maintaining its existing language knowledge.

Additional LoRA adapters were also inserted into the gate, up, and down projection layers of the feed-forward network. These layers are responsible for transforming internal feature representations after the attention process has been completed. Updating these components enables the model to learn new reasoning patterns and generate outputs that better match the desired instruction format. Applying LoRA to both the attention layers and the feed-forward layers allows the model to adapt effectively while modifying only a very small percentage of its total parameters.

One of the key reasons for selecting LoRA is that it prevents unnecessary modification of the original pre-trained model. During training, the frozen base weights retain the extensive linguistic and reasoning knowledge learned during large-scale pre-training, while the lightweight adapter layers capture only the new task-specific information. This separation between the original model and the adapter parameters reduces the risk of catastrophic forgetting, where a model loses previously learned knowledge while learning a new task. As a result, SinLlama retained its strong general language understanding while gradually acquiring new educational capabilities through fine-tuning.

Another important advantage of LoRA is its flexibility. Since the adapter weights are stored separately from the original model, different adapters can be trained for different applications without creating multiple copies of the complete language model. In practical terms, this means that only the relatively small adapter files need to be saved after training, while the original SinLlama model remains unchanged. This approach reduces storage requirements and makes it easier to manage multiple versions of the model if additional educational tasks need to be introduced in the future.

In this project, the LoRA rank (r) was set to 16, while the LoRA alpha parameter was set to 32. The rank determines the size and learning capacity of the adapter matrices. A higher rank allows the adapters to learn more complex relationships but also increases the number of trainable parameters. Since the training dataset contained approximately 13,000 educational instruction-response pairs, a rank of 16 was considered sufficient to capture the required task-specific knowledge without introducing excessive model complexity. The alpha value was selected as twice the rank, following common recommendations in the LoRA literature, to provide appropriate scaling for the adapter updates during training.

Another benefit of using LoRA was its compatibility with 4-bit quantization and the Unsloth training framework used throughout this project. Because only the adapter parameters required updating, the frozen base model could remain quantized, significantly reducing memory usage while maintaining model performance. This combination of LoRA and quantization enabled efficient fine-tuning on the available GPU hardware without compromising the quality of the generated educational responses.

The use of LoRA also supported the iterative training strategy adopted in this research. Instead of training separate language models from scratch for each new capability, the same SinLlama model was progressively improved through multiple rounds

of task-specific fine-tuning. Separate adapter weights were trained for educational question answering, quiz generation, lesson explanation, and level-adaptive content rewriting. This approach simplified the development process while ensuring that each new capability could be added efficiently without requiring expensive full-model retraining.

Although LoRA provides many practical advantages, it also has certain limitations. Since only a small subset of parameters is updated, the learning capacity of the adapters is naturally lower than that of full fine-tuning. For tasks requiring substantial changes to the model’s underlying knowledge, full fine-tuning may achieve slightly better performance. However, for instruction-following tasks such as educational question answering, quiz generation, content explanation, and personalized rewriting, previous research has shown that LoRA can achieve performance very close to full fine-tuning while requiring only a fraction of the computational resources.

Overall, LoRA proved to be an effective and practical fine-tuning technique for this research. It allowed SinLlama to learn multiple educational tasks while keeping computational requirements within the available hardware limitations. By updating only lightweight adapter layers instead of the entire model, the research achieved efficient training, reduced memory consumption, and faster experimentation without sacrificing the quality of the generated educational content. These advantages made LoRA the most appropriate fine-tuning approach for developing the intelligent tutoring system presented in this project.

3.14.4 Training Configuration

After selecting the fine-tuning method, the next step was to configure the training process to ensure that the model could be trained efficiently while maintaining good performance. The training configuration defines how the model learns from the dataset, including the numerical precision, memory optimization techniques, learning rate schedule, number of training epochs, and other parameters that influence the overall training process. Selecting an appropriate configuration is important because it directly affects both the quality of the final model and the computational resources required for training.

In this study, the SinLlama model was fine-tuned using the Unsloth framework. Unsloth is an optimized framework designed for parameter-efficient fine-tuning of large language models. It provides faster training and lower memory consumption compared to standard training implementations while remaining fully compatible with techniques such as LoRA and model quantization. These optimizations allowed the training process to be completed within the available hardware resources while maintaining stable model performance.

To further reduce memory usage, the frozen base model was loaded using 4-bit quantization. Quantization is a technique that stores model weights using fewer bits than the standard floating-point representation. Since the original parameters of SinLlama remained frozen throughout training, they could be stored in a compressed format without significantly affecting the model’s ability to learn new tasks. Only the LoRA adapter parameters remained trainable. This approach greatly reduced GPU memory requirements, making it possible to fine-tune the model on hardware with limited memory while preserving the quality of the generated responses.

Training computations were performed using bfloat16 (Brain Floating Point 16-bit)

precision. Compared to traditional 32-bit floating-point calculations, bfloat16 requires less memory and allows faster computations while maintaining sufficient numerical precision for deep learning tasks. Using bfloat16 helped improve training efficiency without introducing noticeable reductions in model accuracy. The combination of 4-bit quantization for the frozen model weights and bfloat16 precision for trainable parameters provided an effective balance between computational efficiency and model performance.

Another important optimization used during training was gradient checkpointing. Large language models generate a considerable number of intermediate activation values during the forward pass, and storing all of these values requires a large amount of GPU memory. Gradient checkpointing reduces this memory requirement by storing only selected activations during the forward pass. The remaining activations are recomputed when needed during backpropagation instead of being permanently stored. Although this increases computation time slightly, it significantly lowers memory usage and enables larger models to be trained on GPUs with limited memory capacity. For this project, this trade-off was acceptable because reducing memory consumption was more important than minimizing training time.

The learning rate is another important factor that influences the fine-tuning process. A learning rate that is too high can cause unstable parameter updates and prevent the model from converging, while a learning rate that is too low may result in very slow learning. To achieve stable training, a cosine learning-rate scheduler with a short warm-up period was used. During the warm-up stage, the learning rate gradually increased from a very small value to the maximum learning rate. Starting with a small learning rate helps prevent large parameter updates during the early stages of training when the adapter weights are still randomly initialized. After the warm-up period, the cosine scheduler gradually reduced the learning rate following a smooth cosine curve. As training progressed, the updates became progressively smaller, allowing the model to refine its learned parameters more carefully and improving convergence toward the final stages of training.

The model was trained for 15 epochs, meaning that the complete training dataset was processed fifteen times. Training for multiple epochs allows the model to repeatedly learn from the same examples, gradually improving its understanding of the relationship between instructions, educational context, and expected responses. The selected number of epochs provided sufficient learning opportunities while reducing the risk of excessive training that could lead to overfitting. During experimentation, this configuration produced stable learning behaviour and consistent improvements in response quality.

Since the available dataset contained approximately 13,000 educational instruction-response pairs, all available examples were used during training. A separate validation dataset was not created because reducing the size of the training data could have limited learning across the multiple educational subjects included in the dataset. Instead, the model was trained using the complete dataset, and its overall performance was evaluated after training using automated evaluation methods. Although this approach maximized the amount of training data available, it is acknowledged as a limitation because model checkpoints could not be selected based on validation performance during training.

The overall training configuration was designed to achieve a balance between computational efficiency, memory usage, and model accuracy. By combining LoRA

adapters, 4-bit quantization, bfloat16 precision, gradient checkpointing, and a cosine learning-rate scheduler, the model could be fine-tuned efficiently within the available hardware constraints while maintaining high-quality educational responses. These techniques worked together to reduce computational requirements without sacrificing the effectiveness of the fine-tuning process. Another advantage of this training configuration is that it can be reproduced easily for future improvements. If new educational subjects or additional instruction-following tasks are introduced, the same configuration can be reused with minimal modifications. This provides a scalable foundation for extending the intelligent tutoring system while maintaining consistent training procedures across future fine-tuning iterations.

Overall, the selected training configuration provided a practical and efficient solution for adapting the SinLlama model to the educational domain. It enabled stable fine-tuning using limited computational resources while producing a model capable of supporting accurate question answering and serving as the foundation for the additional task-specific fine-tuning stages described in the following sections.

3.14.5 Selection of Hyperparameters

Selecting appropriate hyperparameters is an important step in the fine-tuning process because they directly influence the model's learning ability, training stability, and computational efficiency. Hyperparameters are predefined values that control how the model is trained and are chosen before the training process begins. In this research, the hyperparameters were selected based on the size of the training dataset, the available hardware resources, and commonly recommended practices for LoRA-based fine-tuning.

One of the most important hyperparameters is the LoRA rank (r), which determines the learning capacity of the adapter layers. The rank controls the size of the trainable low-rank matrices that are added to the transformer model during fine-tuning. A higher rank increases the number of trainable parameters, allowing the adapters to learn more complex relationships from the training data. However, increasing the rank also increases memory usage, training time, and the possibility of overfitting, especially when the available dataset is relatively small. Overfitting occurs when the model memorizes the training examples instead of learning patterns that can be applied to unseen data.

For this project, the LoRA rank was set to 16. The training dataset contained approximately 13,000 educational instruction-response examples, which is considered a moderate-sized dataset for parameter-efficient fine-tuning. A rank of 16 provided sufficient learning capacity to capture the required instruction-following behaviour while keeping the number of trainable parameters relatively small. This allowed the model to learn effectively without introducing unnecessary computational complexity. Another important hyperparameter is the LoRA alpha value, which controls the scaling applied to the adapter outputs during training. The alpha value determines how strongly the learned adapter updates influence the original model. If the alpha value is too small, the adapters may not contribute enough to learning. Conversely, if it is too large, the training process may become unstable because the adapter updates can have an excessive influence on the model.

In this study, the LoRA alpha value was set to 32, which is exactly twice the selected LoRA rank. This ratio is commonly recommended in LoRA implementations

because it provides appropriate scaling for the adapter updates while maintaining stable optimization throughout training. Using an alpha value of 32 enabled the model to learn new educational tasks efficiently without affecting the stability of the training process. Overall, the selected hyperparameters were chosen to balance learning performance, computational efficiency, and model stability. Considering the available hardware resources and the size of the dataset, a LoRA rank of 16 and an alpha value of 32 provided sufficient capacity for learning new instruction-following behaviours while minimizing the risk of overfitting and keeping GPU memory usage within practical limits. These settings contributed to the successful fine-tuning of the SinLlama model for the educational tasks required in this research.

3.14.6 Training Dataset and Evaluation Strategy

The training dataset used in this research contained approximately 13,000 educational instruction-response pairs. The dataset was designed to teach the model how to understand educational instructions, use the provided context, and generate accurate responses in Sinhala. Each training example consisted of an instruction, relevant educational context, and the expected answer. This structure enabled the model to learn both subject knowledge and instruction-following behaviour.

During the fine-tuning process, the entire dataset was used for training without creating a separate validation dataset. This decision was made after considering the overall size of the available data. Reserving a portion of the dataset for validation would have reduced the number of training examples available for learning, which could have negatively affected the model's performance. Therefore, all available examples were used during training to maximize the learning opportunities provided by the dataset. Using the complete dataset allowed the model to learn from every available example and improve its ability to generate accurate educational responses. Since the primary objective of this research was to develop a reliable educational language model, maximizing the amount of training data was considered more beneficial than creating a separate validation set. As a result, the model was trained for the planned 15 epochs, allowing it to repeatedly learn from the entire dataset.

However, using all available data for training also introduced an important limitation. Since no validation dataset was available, it was not possible to monitor validation loss or automatically select the best-performing model checkpoint during training. Instead, the final model checkpoint was selected after completing the predefined number of training epochs. Although this approach simplified the training process and maximized the use of the available data, it also meant that the model's ability to generalize to completely unseen examples could not be measured during training. After the fine-tuning process was completed, the model was evaluated using three automated evaluation methods: ROUGE, BERTScore, and LLM-as-a-Judge. Each evaluation method measures a different aspect of the generated responses, providing a more comprehensive assessment of the model's overall performance.

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) measures the lexical similarity between the generated response and the reference answer by comparing overlapping words and phrases. It is widely used to evaluate text generation systems because it provides an indication of how closely the generated output matches the expected response. BERTScore measures semantic similarity rather than exact word matching. It uses contextual embeddings generated by a pre-trained BERT model to

determine whether the generated response conveys the same meaning as the reference answer, even when different words or sentence structures are used. This makes BERTScore particularly suitable for evaluating natural language generation tasks where multiple correct responses may exist.

The third evaluation method, LLM-as-a-Judge, uses a large language model to assess the overall quality of generated responses. Unlike traditional evaluation metrics, it considers factors such as correctness, completeness, relevance, clarity, and fluency. This provides a human-like assessment of the generated answers and complements the quantitative scores produced by ROUGE and BERTScore. By combining these three evaluation methods, the model's performance was evaluated from multiple perspectives, including lexical similarity, semantic understanding, and overall response quality. Using multiple evaluation metrics provides a more reliable assessment than relying on a single evaluation method because each metric captures different aspects of the generated output.

The reliability of the evaluation results depends on the dataset used for testing. If the evaluation dataset was completely independent from the training dataset, the reported scores provide a good indication of the model's ability to generalize to unseen examples. However, if the same data were used for both training and evaluation, the reported performance may be higher than the model's actual performance in real-world applications because the model has already learned from those examples during training. Therefore, if an independent evaluation dataset was not available, this should be acknowledged as a limitation of the research. Future work could improve the evaluation process by creating a separate test dataset or applying cross-validation techniques to obtain a more accurate measurement of the model's generalization ability.

3.14.7 Need for Task-Specific Fine-Tuning

Although the base fine-tuned SinLlama model demonstrated good performance in answering direct educational questions, further testing within the complete intelligent tutoring system revealed that question answering alone was not sufficient to support all of the system's required features. The base model had learned how to understand an instruction, use the given context, and produce an appropriate answer. However, the tutoring system required several additional capabilities that involved different forms of instruction following rather than simple question answering.

These additional tasks included generating multiple-choice quizzes from lesson content, explaining educational topics in a conversational teaching style, and rewriting the same lesson for students with different learning levels. Each of these tasks requires the model to perform a different type of language generation. For example, quiz generation requires the model to create structured questions and answer options, content explanation requires transforming formal textbook language into simple spoken explanations, and content rewriting requires changing the complexity of the same information while preserving its original meaning. These behaviours are significantly different from generating direct answers to educational questions.

When these tasks were tested using only the base fine-tuned model, the generated outputs were often inconsistent. The model occasionally failed to follow the required instruction format, produced incomplete responses, or generated outputs that did not match the expected structure. For example, generated quizzes sometimes contained

missing answer choices or inconsistent formatting, while lesson explanations often resembled short factual answers rather than detailed teaching-style explanations. Similarly, the model struggled to consistently rewrite educational content according to different learner levels.

These observations demonstrated that instruction-following capabilities learned from question-answer training did not automatically generalize to structurally different educational tasks. Although the model possessed the necessary language understanding, it had not been trained to perform these specialized tasks. As a result, relying only on the initial question-answer fine-tuning stage was insufficient for developing a complete intelligent tutoring system. To overcome these limitations, this project adopted an iterative task-specific fine-tuning strategy. Instead of expecting a single fine-tuning process to teach every required capability, each missing feature was treated as an independent learning task. Whenever testing identified a capability that the model could not perform reliably, a dedicated instruction dataset was created specifically for that task. The model was then fine-tuned again using the new dataset while retaining the knowledge acquired during previous training stages.

This iterative approach provided several advantages. First, it allowed each capability to be learned independently without affecting the performance of previously learned tasks. Second, the instruction datasets were carefully designed to match the exact behaviour expected from the model, resulting in more reliable and consistent outputs. Finally, this strategy makes it easier to expand the system in the future because additional capabilities can be introduced by creating new instruction datasets and performing further fine-tuning instead of retraining the entire model from the beginning.

Overall, adopting task-specific fine-tuning enabled the SinLlama model to gradually acquire all of the specialized capabilities required by the intelligent tutoring system. Instead of relying solely on a general-purpose question-answering model, the final system was built through multiple fine-tuning iterations, with each stage adding a new instruction-following capability. This methodology produced a more robust, reliable, and practical educational language model capable of supporting a wide range of learning activities beyond direct question answering.

3.14.8 Quiz Generation Fine-Tuning

One of the main features of the intelligent tutoring system developed in this project is the ability to automatically generate quizzes from lesson content. Instead of using pre-written assessment questions, the system generates multiple-choice quizzes dynamically based on the educational material selected by the student. This feature allows students to assess their understanding of each lesson immediately after studying the content and provides a more interactive learning experience.

During the initial implementation, the base fine-tuned SinLlama model was prompted to generate quizzes directly from educational passages. Although the model was capable of producing questions related to the lesson, the generated results were not sufficiently reliable for practical use. In many cases, the output did not follow a consistent structure. Some quizzes contained fewer than the required number of answer options, while others included duplicate questions, incomplete answers, or incorrectly formatted responses. Occasionally, the model also generated explanations instead of multiple-choice questions, making the output unsuitable for direct use.

within the application.

These observations indicated that quiz generation is fundamentally different from direct question answering. The base fine-tuned model had learned to answer educational questions, but it had not learned how to transform lesson content into a structured assessment. Generating a quiz requires the model to identify important concepts from the lesson, create meaningful questions, generate suitable distractor options, and clearly identify the correct answer while maintaining a consistent output format. Since these requirements were not included in the original question-answer training dataset, the model was unable to perform this task reliably.

To address this limitation, a dedicated instruction dataset was developed specifically for quiz generation. The dataset consisted of pairs of educational content and the corresponding structured quiz output. Each training example demonstrated how lesson content should be converted into a multiple-choice assessment. The generated output included the quiz question, four answer options, and the correct answer in a predefined format. By training on many examples of this structure, the model gradually learned the expected output pattern and became more consistent in generating quizzes. The same LoRA-based fine-tuning approach used during the initial training stage was applied for this task-specific fine-tuning process. Instead of retraining the entire model, only the LoRA adapter parameters were updated while the original SinLlama model remained frozen. This approach reduced computational requirements while allowing the model to learn the new quiz-generation behaviour efficiently.

After completing this additional fine-tuning stage, the model showed a significant improvement in quiz generation performance. The generated quizzes followed the required structure more consistently and produced meaningful questions that were closely related to the lesson content. The answer options became more complete and the correct answers were identified more accurately than before. As a result, the model became suitable for integration into the quiz generation component of the intelligent tutoring system.

Although the fine-tuned model generated much more reliable outputs, minor formatting inconsistencies occasionally remained. For example, some responses contained small variations in spacing, punctuation, or JSON formatting. To ensure that every generated quiz could be processed correctly by the application, an additional validation stage was implemented. The generated output was checked using JSON parsing and validation techniques before being displayed to the student. If minor formatting issues were detected, they were corrected automatically by the application without affecting the quiz content.

This additional fine-tuning stage successfully enabled the SinLlama model to perform automatic quiz generation as a dedicated instruction-following task. By combining task-specific fine-tuning with output validation, the intelligent tutoring system was able to generate structured, reliable, and educationally meaningful quizzes directly from lesson content. This capability became one of the core learning features of the proposed system and significantly improved the interactivity and effectiveness of the learning experience for students.

3.14.9 Content Explanation Fine-Tuning

Another important feature of the intelligent tutoring system developed in this project is the AI teaching assistant, which provides lesson explanations to students in a clear

and easy-to-understand manner. Instead of simply displaying textbook content, the teaching assistant explains each lesson using natural language, making the learning experience more interactive and engaging. This feature is especially useful for students who may find formal textbook language difficult to understand.

During the initial implementation, the base fine-tuned SinLlama model was tested by providing textbook content and asking it to explain the lesson. Although the model generated factually correct responses, the explanations were generally short and resembled direct answers rather than complete lesson explanations. In many cases, the responses lacked sufficient detail, smooth transitions between ideas, and a conversational teaching style. As a result, the generated content was not suitable for presenting complete lessons through the AI teaching assistant.

These observations showed that content explanation is a different instruction-following task from question answering. While question answering requires the model to provide a direct response to a specific question, lesson explanation requires the model to organize information logically, simplify complex concepts, and present the content in a way that students can easily understand. Since the original question-answer dataset did not contain examples of teaching-style explanations, the model was unable to perform this task consistently.

To overcome this limitation, a dedicated instruction dataset was created specifically for content explanation. Each training example contained educational content together with its corresponding explanation written in a natural and conversational style. The explanations were designed to preserve all important educational concepts while simplifying sentence structure, improving readability, and presenting the information in a manner similar to a teacher explaining a lesson in a classroom. The same LoRA-based fine-tuning approach used in the previous training stages was applied to teach this new capability. During training, only the LoRA adapter parameters were updated while the original SinLlama model remained frozen. This parameter-efficient approach enabled the model to learn the new explanation behaviour without requiring full model retraining, making the process both memory-efficient and computationally practical.

After completing this fine-tuning stage, the model demonstrated a significant improvement in generating lesson explanations. Instead of producing short factual answers, it generated more detailed and well-structured explanations that followed a logical flow. The explanations became easier to understand because technical terms were described more clearly, and the overall writing style became more natural and conversational while still preserving the accuracy of the educational content.

The fine-tuned model was then integrated into the AI teaching assistant component of the intelligent tutoring system. When a student selects a lesson, the system retrieves the relevant educational content and provides it to the fine-tuned SinLlama model. The model generates a clear explanation, which is then presented by the AI teaching assistant to support the student's learning process. This allows students to receive lesson explanations in a more engaging and interactive format instead of reading only the original textbook content.

Overall, this task-specific fine-tuning stage successfully enabled the SinLlama model to perform content explanation as a dedicated educational task. The improved explanations made the teaching assistant more effective by presenting lesson content in a clear, structured, and student-friendly manner. This capability became one of the key features of the proposed intelligent tutoring system and contributed to creating a more personalized and interactive learning experience.

3.14.10 Level-Adaptive Content Rewriting

One of the main objectives of the intelligent tutoring system developed in this project was to provide a personalized learning experience for students. Since students have different levels of prior knowledge, learning speed, and understanding, presenting the same lesson in a single format may not be suitable for everyone. To address this challenge, the system was designed to rewrite the same educational content for three different learning levels: Beginner, Intermediate, and Advanced. This allows students to receive explanations that match their current level of understanding, making the learning process more effective and engaging.

During the initial implementation, the base fine-tuned SinLlama model was tested by asking it to rewrite lesson content for different learner levels. Although the model occasionally simplified or expanded the text, the results were inconsistent. In many cases, the generated content did not clearly match the selected learning level. Some beginner explanations still contained technical terms that were difficult for new learners, while some advanced explanations lacked the additional detail expected by more experienced students. This showed that the model had not learned how to consistently adjust educational content according to different learner levels.

Unlike direct question answering, level-adaptive content rewriting requires the model to preserve the original meaning while changing the presentation of the information. The model must simplify vocabulary, modify sentence structure, adjust the amount of detail, and change the depth of explanation without introducing incorrect information. This makes the task more complex than simply answering a question because the educational concepts must remain accurate while the language is adapted to suit different learners. To enable this capability, a third task-specific instruction dataset was created for level-adaptive content rewriting. Each training example contained the original educational content together with three rewritten versions corresponding to the Beginner, Intermediate, and Advanced learning levels. These examples demonstrated how the same lesson should be presented differently depending on the learner's level while maintaining the same educational meaning.

For the Beginner level, the rewritten content used simple vocabulary, shorter sentences, and clear explanations of basic concepts. Difficult technical terms were either avoided or explained using simple language so that students with limited prior knowledge could understand the lesson more easily. For the Intermediate level, the explanations included a moderate amount of detail and assumed that students already possessed a basic understanding of the subject. More subject-specific terminology was introduced where appropriate, while the explanations remained clear and easy to follow. For the Advanced level, the rewritten content provided more detailed explanations, included technical terminology, and discussed concepts in greater depth. This version was intended for students who already had a strong understanding of the subject and required more comprehensive learning material.

The same LoRA-based fine-tuning approach used in the previous stages was applied to train the model for this new capability. During training, only the LoRA adapter parameters were updated while the original SinLlama model remained frozen. This parameter-efficient approach enabled the model to learn the new instruction-following behaviour without requiring full retraining of the entire language model. After completing this fine-tuning stage, the model demonstrated a significant improvement in generating personalized educational content. The rewritten lessons more consistently matched the selected learning level while preserving the accuracy and meaning of the

original educational material. The model successfully adjusted vocabulary, sentence complexity, explanation depth, and overall writing style according to the requested learner level, resulting in more suitable learning materials for different groups of students.

The fine-tuned model was then integrated into the personalization component of the intelligent tutoring system. When a student selects a preferred learning level, the system retrieves the relevant lesson content and provides it to the fine-tuned SinLlama model together with the appropriate instruction. The model then generates a rewritten version of the lesson that matches the selected learner level before presenting it to the student. This allows the system to provide personalized learning content without maintaining separate lesson materials for different types of learners.

Overall, this task-specific fine-tuning stage successfully enabled the SinLlama model to perform level-adaptive content rewriting as a dedicated educational task. The ability to generate personalized explanations for Beginner, Intermediate, and Advanced learners became one of the key personalization features of the proposed intelligent tutoring system. By adapting educational content to individual learning needs, the system provides a more flexible, engaging, and effective learning experience while preserving the accuracy of the original lesson content.

3.14.11 Iterative Fine-Tuning Strategy

The fine-tuning process used in this project followed an iterative development strategy rather than treating fine-tuning as a single training stage. Instead of expecting the model to learn every required capability during one fine-tuning process, the model was gradually improved through multiple rounds of task-specific fine-tuning. This approach allowed new capabilities to be added one at a time while preserving the knowledge learned during previous training stages.

The first stage focused on teaching the SinLlama model to answer educational questions using the provided context. After this initial fine-tuning stage was completed, the model was integrated into the intelligent tutoring system and tested with the application's different features. Although the model successfully answered educational questions, testing showed that it could not reliably perform several other tasks required by the system, such as quiz generation, content explanation, and level-adaptive content rewriting.

Whenever a limitation was identified during testing, the missing capability was treated as a separate learning task. A new instruction dataset was then created specifically for that capability, and another round of LoRA-based fine-tuning was performed. This process was repeated until the model was able to perform each required task consistently. By following this step-by-step approach, the model gradually acquired all the capabilities needed for the intelligent tutoring system instead of attempting to learn everything from a single dataset.

This iterative strategy also made it easier to evaluate the effectiveness of each fine-tuning stage. After every training iteration, the updated model was integrated into the application and tested under real usage scenarios. The generated outputs were carefully examined to determine whether the newly learned behaviour met the system requirements. If additional improvements were required, the instruction dataset was refined and another fine-tuning iteration was carried out. This continuous process of training, testing, and improvement helped increase the overall quality and reliability

of the model.

Another advantage of the iterative approach was that each capability was learned using a dedicated instruction dataset. Instead of assuming that one large question-answer dataset could teach every type of educational task, separate datasets were prepared for quiz generation, content explanation, and level-adaptive content rewriting. Each dataset contained examples that demonstrated the exact behaviour expected from the model. This enabled the model to learn specialized instruction-following patterns more effectively and produced more accurate outputs for each feature.

The iterative fine-tuning strategy also influenced the design of the intelligent tutoring system itself. Since each capability was trained using a specific instruction format, the application was designed to use fixed task-specific prompts during inference. Each system feature sends a predefined instruction that closely matches the instruction format used during fine-tuning. Maintaining this consistency between training and inference helps the model recognize the requested task more accurately and produce outputs that follow the expected structure. Using fixed prompts also improves the stability of the generated responses. If the wording of the prompts changes significantly from those used during training, the model may interpret the task differently and generate inconsistent results. By using standardized prompts for each system feature, the model remains within the instruction patterns it learned during fine-tuning, resulting in more reliable and consistent outputs.

Overall, the iterative fine-tuning strategy played a key role in the successful development of the proposed intelligent tutoring system. Rather than relying on a single fine-tuning stage, the model was continuously improved through multiple task-specific training iterations. This approach enabled the SinLlama model to gradually acquire all the capabilities required by the system while maintaining high-quality educational responses. As a result, the final model was able to answer questions, generate quizzes, explain lessons, and rewrite educational content for different learner levels with greater accuracy and consistency than the initial base fine-tuned model.

3.14.12 Single Fine-Tuned Model for Multiple Subjects

The intelligent tutoring system developed in this project supports multiple educational subjects while using only one fine-tuned SinLlama model. Instead of training and maintaining a separate language model for each subject, a single model was trained to perform all required educational tasks. This approach simplified the overall system architecture and reduced the computational resources required for deployment and maintenance.

The initial question-answer fine-tuning dataset contained educational examples collected from several different subjects. This allowed the SinLlama model to learn how to understand and answer questions from a variety of educational domains rather than becoming specialized in a single subject. As a result, the model developed a general instruction-following capability that could be applied across different types of educational content. The additional fine-tuning stages introduced in this project were task-specific rather than subject-specific. Separate instruction datasets were created to teach the model how to generate quizzes, explain lesson content, and rewrite educational material for different learner levels. These datasets focused on teaching new instruction-following behaviours instead of introducing knowledge from a particular subject. Therefore, the same fine-tuned model could perform these tasks

regardless of the subject selected by the student.

During system operation, subject-specific information is not stored directly within the fine-tuned model. Instead, the system uses a Retrieval-Augmented Generation (RAG) pipeline to provide relevant educational content at the time of inference. When a student selects a lesson, the RAG component searches the vector database and retrieves the textbook passages that are most relevant to the student's request. These retrieved passages are then provided to the fine-tuned SinLlama model together with the appropriate task-specific instruction.

Using the retrieved lesson content as context allows the model to generate responses that are directly related to the selected topic. Rather than relying only on the knowledge learned during fine-tuning, the model combines its instruction-following capability with the information retrieved from the textbook database. This enables the system to produce accurate, context-aware responses while reducing the possibility of generating incorrect or unrelated information. The combination of a single fine-tuned language model and the RAG pipeline provides several practical advantages. Since only one model needs to be maintained, the overall system becomes simpler to develop, deploy, and update. It also reduces storage requirements because multiple subject-specific models are not required. In addition, system maintenance becomes easier, as improvements to the language model automatically benefit every supported subject without requiring separate retraining processes.

Another important advantage of this design is its scalability. If new educational subjects need to be added in the future, the language model does not necessarily require additional fine-tuning. Instead, new educational materials can be processed, converted into vector embeddings, and stored in the vector database. Once the new knowledge is available through the RAG pipeline, the existing fine-tuned SinLlama model can retrieve and use the new content in the same way as the existing subjects. This makes the system more flexible and significantly reduces the effort required to expand the learning platform.

Overall, using a single fine-tuned SinLlama model together with a Retrieval-Augmented Generation (RAG) pipeline provides an efficient and scalable solution for supporting multiple educational subjects. The fine-tuned model is responsible for understanding instructions and generating responses, while the RAG component supplies the relevant lesson content during inference. This separation of responsibilities improves response accuracy, simplifies system maintenance, reduces computational requirements, and allows the intelligent tutoring system to support multiple subjects using a single language model.

3.14.13 Continuous System Optimization

Following the deployment of the intelligent tutoring system, a continuous optimization process was implemented to further enhance model performance. This strategy involved monitoring student interactions, identifying common queries that the system struggled to address, and periodically updating the fine-tuned adapters with new, targeted instruction data. This ongoing refinement cycle ensured that the model remained aligned with evolving curriculum requirements and student needs, effectively bridging the gap between static training and dynamic real-world usage. By systematically incorporating feedback into new training iterations, the system maintained high levels of accuracy and relevance, demonstrating the adaptability of the chosen fine-tuning

methodology in a production environment.

3.15 Digitization and Analysis of Sinhala Subject Content

A major challenge in developing the Sinhala-medium adaptive learning system was preparing reliable digital learning content from existing curriculum resources. Many Sinhala educational textbooks were available as PDF documents; however, some of these PDFs had been created using older Sinhala font encoding methods rather than modern Unicode standards. In legacy Sinhala font-based documents, the characters stored inside the PDF did not directly represent actual Sinhala Unicode characters. Instead, the underlying character codes were mapped to Latin characters, and the correct legacy Sinhala font was required during rendering to display readable Sinhala text. Therefore, when extracting text using normal PDF extraction tools, the output did not contain meaningful Sinhala Unicode text. It produced incorrect Latin character sequences instead.

This issue was different from scanned document processing because the PDF already contained a text layer. The main challenge was not recognizing characters from images but converting the existing legacy character encoding into proper Sinhala Unicode. Therefore, a font-aware character conversion approach was required instead of traditional OCR-based extraction. To overcome this challenge, a dedicated content ingestion pipeline was implemented within the teacher dashboard. This pipeline allowed administrators to upload Sinhala textbook PDFs and automatically convert them into clean, Unicode-based, curriculum-structured content suitable for the Retrieval-Augmented Generation (RAG) pipeline.

The developed pipeline followed a three-stage human-in-the-loop workflow:

- Content Extraction
- Content Review and Correction
- Finalization and Storage

Each stage was executed as a background process because PDF processing required several seconds or minutes depending on the size and complexity of the document. The administrator could monitor the processing status through the dashboard and review the generated content before it became available to the learning system.

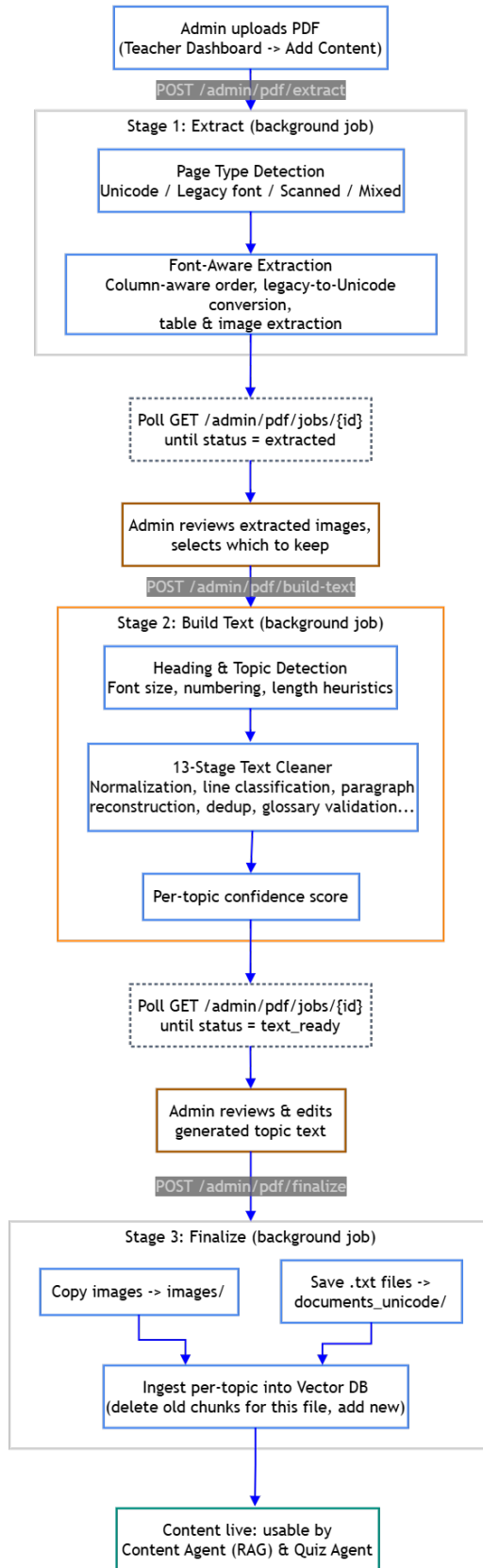


Figure 3.11: Digitization and Analysis Pipeline

3.15.1 Page Type Detection

The first stage of the pipeline identified the type of content available on each PDF page. Each page was classified into one of four categories:

- Unicode Sinhala text
- Legacy Sinhala font encoded text
- Scanned/image-only content
- Mixed content containing multiple formats

This classification was performed by analyzing the extracted text information, including the percentage of Sinhala Unicode characters, the presence of known legacy Sinhala font identifiers, and whether meaningful text content existed within the page. The purpose of this step was to determine the correct extraction method for each page. Unicode-based pages could be directly processed, while legacy encoded pages required font conversion before further processing. Pages containing only scanned images were detected separately because they required OCR processing, which was not implemented in the system.

3.15.2 Font-Aware Text Extraction and Conversion

After identifying the page type, the system performed font-aware extraction of text and images from the PDF. The extraction process maintained the original reading order by considering the document layout. For multi-column pages, the system extracted content from the left column first and then the right column, while maintaining the top-to-bottom order within each column. During extraction, the system examined font metadata associated with each text segment. If a text segment used a legacy Sinhala font, it was passed through a dedicated legacy-to-Unicode conversion process. This conversion ensured that the final extracted content was stored as actual Sinhala Unicode text, independent of the original font encoding used in the PDF.

In addition to normal text extraction, the system handled other document elements:

- **Tables:** Extracted separately and converted into structured key-value text to avoid losing important information.
- **Images and diagrams:** Image regions were identified, and related diagram labels were detected by checking the position relationship between text segments and image boundaries.

This approach helped maintain important educational information that might otherwise be lost during simple text extraction.

3.15.3 Heading and Topic Detection

After text extraction, the system automatically identified headings and separated the document into meaningful learning topics.

Topic detection was performed using a set of formatting-based rules, including:

- Relative font size compared with normal body text.

- Numbering patterns such as chapter and section numbering.
- Text length characteristics, since headings are generally shorter than paragraphs.

For example, major headings such as "1.1 Introduction" and smaller subheadings could be identified based on their formatting patterns. Using this approach, a complete textbook chapter could be automatically divided into separate topic-level content files. This reduced manual preparation work and created a structured format suitable for later retrieval and content generation processes.

3.15.4 Thirteen-Stage Text Cleaning Pipeline

After topic separation, each extracted topic passed through a multi-stage text cleaning process to improve content quality before storage.

The cleaning pipeline contained thirteen sequential processing steps:

1. Text preprocessing, including line ending and whitespace normalization.
2. Sinhala Unicode normalization.
3. Correction of known extraction errors using a verified correction dictionary.
4. Text classification into categories such as headings, paragraphs, bullet points, activities, diagram labels, and noise.
5. Removal of unnecessary content such as page numbers and extraction artifacts.
6. Paragraph reconstruction by combining incorrectly separated lines.
7. Sentence boundary correction.
8. Missing space correction between words.
9. Bullet-point reconstruction.
10. Heading format standardization.
11. Duplicate content removal using exact and similarity-based comparison.
12. Subject terminology validation using a predefined glossary.
13. Structural validation of extracted educational content.

The final validation step checked whether the extracted content maintained logical correctness. For example, if a section stated that there were "three types" of something, the system verified whether the extracted content actually contained three related items. Additionally, a confidence score was generated for each topic based on the number of corrections and validation issues detected. This score helped administrators identify topics that required more attention during the review process.

3.15.5 Human Review and Content Validation

Although the pipeline automated most of the extraction and preparation process, a human review step was included before storing the final content.

The workflow intentionally followed a human-in-the-loop approach:

- Extract → Review Images → Review and Edit Topics → Finalize

During the review stage, administrators could:

- Check the extracted topic structure.
- Correct incorrectly detected headings.
- Edit generated text.
- Select appropriate images and diagrams.

Nothing was stored permanently in the curriculum database or vector database until the administrator confirmed finalization. This approach prevented incorrect automatically generated content from entering the student learning environment and improved the reliability of the educational resources.

3.15.6 Non-Destructive Content Ingestion

After final approval, the processed content was stored using a structured naming system based on subject, lesson, and topic information. Each topic was saved as a separate .txt file in the curriculum content repository. Extracted images were stored separately in the image asset directory used by the student learning interface. The processed topic content was then added to the vector database for RAG-based retrieval. Before adding new data, existing vector chunks related to the same topic file were removed. The updated content was then inserted as new chunks. This non-destructive approach ensured that updating one topic did not affect other existing curriculum content. Unlike rebuilding the entire vector database, this method allowed efficient updates when corrections or improvements were made to individual lessons.

3.15.7 Contribution of the Developed Pipeline

The developed digitization pipeline improved the efficiency of preparing Sinhala educational resources for the adaptive learning system. Previously, adding new curriculum content required manually converting PDF files into Unicode text, preparing topic files, and manually inserting content into the vector database. The implemented solution allowed administrators to directly upload raw textbook PDFs and automatically transform them into structured, searchable, and RAG-ready educational content through the teacher dashboard. This significantly reduced manual preparation effort while improving consistency and scalability of the learning content management process.

3.15.8 Limitations

Although the implemented pipeline provided an effective solution for Sinhala textbook digitization, some limitations remained. The heading and topic detection process relied on formatting-based heuristics such as font size, numbering patterns, and text length. Therefore, documents with inconsistent formatting styles might require additional manual correction during the review stage. Currently, scanned image-only pages could be detected but could not be automatically converted because OCR functionality had not yet been implemented. Additionally, table extraction and diagram-label identification were based on layout heuristics such as bounding-box positions and column structures. Complex or unusual document layouts could result in incorrect associations, which were handled through the human review process.

3.16 System Deployment

The developed platform is deployed using a hybrid deployment architecture, where different parts of the system are hosted according to their computational and operational requirements. The main application services, including the backend, engagement engine, and frontend, are deployed on Microsoft Azure. However, the two components that require significant GPU resources—the fine-tuned Sinhala language model and the AVTR-1 avatar-rendering service—are hosted on a separate GPU-enabled machine. These locally hosted services are connected to the Azure-based application through secure HTTPS tunnels. This approach was selected because it provides the reliability, scalability, and automated deployment capabilities of cloud infrastructure while allowing the project to use locally available GPU resources for computationally expensive AI operations.

3.16.1 Cloud-Hosted Components

The main application services are deployed separately on Azure based on their individual responsibilities. This separation makes the system easier to maintain, update, and deploy because a change in one service does not require the entire application to be redeployed.

The FastAPI backend and Flask-based Engagement Engine are packaged as Docker containers and deployed using Azure Container Apps. The backend is responsible for the main application logic, API processing, database communication, retrieval operations, and communication with the AI services. In addition, it performs some machine-learning operations using TensorFlow, Keras-based LSTM models, and sentence-embedding generation. Therefore, the backend requires more computing resources than a simple API service. It is configured with 4 vCPUs and 8 GiB of memory to provide sufficient resources for these operations. At least one container replica is kept running continuously. This configuration reduces cold-start delays because the machine-learning models do not need to be loaded into memory every time the application receives a new request after a period of inactivity.

The Engagement Engine, implemented using Flask, is also containerized and deployed through Azure Container Apps. It handles engagement-related functionality independently from the main backend. Separating this functionality into its own

service improves the modularity of the system and allows the service to be updated or scaled independently when required.

The Next.js frontend is deployed using Azure Static Web Apps. This service is responsible for providing the user interface through which students interact with the learning platform. During deployment, the Next.js application is built into production-ready static assets and distributed through Azure's content-delivery infrastructure. The URLs of the production backend and Engagement Engine are provided to the frontend through environment variables during the build process. Therefore, the deployed frontend communicates directly with the appropriate production services rather than local development endpoints.

The Docker images required by the backend and Engagement Engine are stored in an Azure Container Registry (ACR). ACR provides a centralized location for storing and managing the application's container images. When a new version of a service is developed, its Docker image is built and pushed to the registry, after which the corresponding Azure Container App can retrieve and run the updated image.

3.16.2 Persistent Storage Strategy

Persistent data is handled differently depending on the type of data being stored. The student-facing educational resources, including processed document text and extracted images, are stored using an Azure Files share. These files are mounted into the backend container so that they remain available even when the container is restarted, replaced, or redeployed. This is important because container storage is generally temporary, and storing important educational resources directly inside a running container could result in data loss when the container is recreated.

The ChromaDB vector database uses a different storage approach. Instead of mounting ChromaDB from Azure Files, the vector database is included directly inside the backend Docker image. This decision was made because ChromaDB uses a SQLite-based storage mechanism, and SQLite does not provide reliable file locking when its database files are accessed through a network-mounted file system.

During the deployment process, an attempt was made to store the ChromaDB files on the mounted Azure Files share. However, the application experienced an issue where the vector database initialization remained stuck indefinitely. This indicated that the network file system was not suitable for the SQLite-based database operations required by ChromaDB. As a result, the vector database was moved into the Docker image itself. This means that the database is available locally within the backend container and avoids the file-locking problems associated with network storage.

Therefore, the deployment uses two different storage approaches for a specific reason: Azure Files is used for persistent educational assets, while ChromaDB is packaged with the application container to ensure reliable database operation. This was a practical deployment decision based on the actual behaviour observed during system testing.

3.16.3 Continuous Integration and Deployment

The Azure-hosted services use an automated Continuous Integration and Continuous Deployment (CI/CD) process implemented with GitHub Actions. The purpose of this pipeline is to reduce manual deployment work and ensure that changes made to the production branch can be transferred to the deployed system automatically.

Separate GitHub Actions workflows are configured for the different services. These workflows use path-based triggers, meaning that a change to the backend code triggers the backend deployment, while a change to the frontend or Engagement Engine triggers only the corresponding deployment process. This prevents unnecessary rebuilding and redeployment of services that have not changed.

The Docker images are built directly using GitHub Actions runners. This approach was necessary because the Azure trial-tier environment used for the project had limitations regarding Azure Container Registry's managed cloud-build functionality. Instead of depending on ACR to build the images, GitHub Actions performs the Docker build process and then pushes the resulting image to the Azure Container Registry.

Authentication between GitHub Actions and Azure is handled using OpenID Connect (OIDC). Rather than storing a permanent Azure password or access key inside GitHub, the workflow uses a federated identity configuration to obtain a short-lived authentication token for each deployment. A dedicated Azure application identity is given the required permissions at the deployment resource-group level. This provides a more secure approach because long-lived credentials do not need to be stored in the repository.

When a developer pushes a change to the production branch, the appropriate GitHub Actions workflow is automatically triggered. The workflow builds a new Docker image, pushes the image to ACR, authenticates with Azure, and updates the relevant Azure Container App to use the newly created image. As a result, a code change can move from the source repository to the live production environment without requiring manual deployment steps.

Sensitive configuration values are also kept separate from the source code. Database connection strings, third-party API credentials, and URLs for external AI services are stored as encrypted environment variables within the Azure services. These values are injected into the containers when the applications start. This approach prevents confidential information from being directly included in the source code or Docker images and makes it easier to change configuration values without modifying the application itself.

3.16.4 GPU-Hosted AI Services

Although the main application services are hosted on Azure, two important AI components are hosted outside the Azure environment because they require GPU acceleration. These components are the fine-tuned Sinhala language model and the AVTR-1 avatar-rendering server.

The fine-tuned Sinhala language model is responsible for several AI-based features of the platform, including question answering, educational content generation, and quiz generation. Running this model requires considerably more computational resources than the normal backend API operations. Similarly, the AVTR-1 service performs computationally intensive operations such as text-to-speech processing, lip-sync generation, and avatar video rendering. These operations require GPU acceleration to achieve an acceptable response time.

The Azure Container Apps configuration used for the project did not provide an economically suitable GPU environment within the available trial-level cloud resources. Therefore, instead of hosting these services on Azure, they were deployed on a locally

available GPU-powered machine. This allowed the project to make use of existing GPU hardware without introducing the additional cost of dedicated cloud GPU infrastructure.

Since the Azure-hosted backend needs to communicate with the locally running AI services, the local services are exposed through ngrok HTTPS tunnels. Each service has its own tunnel, which provides a public HTTPS endpoint and forwards incoming requests to the corresponding service running on the local GPU machine.

For example, the backend uses the `SINHALA_LLM_URL` environment variable to identify the public endpoint of the fine-tuned Sinhala language model. Similarly, the frontend uses the `NEXT_PUBLIC_AVTR_HOST` variable to communicate with the AVTR-1 service. This allows the cloud-hosted application to interact with the locally hosted AI components almost as if they were external cloud services.

However, this hybrid approach also introduces an operational limitation. The GPU machine must remain powered on, and both AI services and their corresponding ngrok tunnels must remain active. If the GPU machine is switched off or a service stops, AI-based functions that depend on that service will no longer be available. Therefore, the cloud-hosted components can continue running independently, but features such as AI content generation, quiz generation, and avatar-based narration depend on the availability of the local GPU services.

Another limitation is associated with the free version of ngrok. When a tunnel is restarted, it may receive a different randomly generated public URL. Since the Azure applications use these URLs to communicate with the local AI services, a URL change requires the corresponding environment variable to be updated. This currently introduces a small amount of manual configuration whenever the tunnel endpoint changes.

For a future production deployment, this limitation could be reduced by using a fixed ngrok domain or another permanent secure tunnelling solution. Another possible improvement would be to migrate the GPU-dependent services to a cloud provider offering suitable GPU-enabled infrastructure. This would eliminate the dependency on a locally available machine and allow the complete system to operate within the cloud.

3.16.5 Overall Deployment Architecture

Overall, the platform follows a service-oriented hybrid-cloud deployment model. The frontend, backend, and Engagement Engine are deployed on Microsoft Azure because these services benefit from managed cloud hosting, containerization, and automated CI/CD. Persistent educational files are maintained using Azure Files, while the ChromaDB vector database is included within the backend container to avoid SQLite network file-locking problems.

At the same time, the computationally intensive AI components are separated from the main cloud infrastructure and deployed on a GPU-enabled machine. Secure ngrok tunnels connect these local services with the Azure-hosted application. This architecture allowed the project to achieve a practical balance between cloud reliability, automated deployment, computational requirements, and cost limitations.

The resulting deployment provides a complete production-accessible learning platform while making effective use of the resources available during the project. Although the current architecture has limitations, particularly the dependency on the

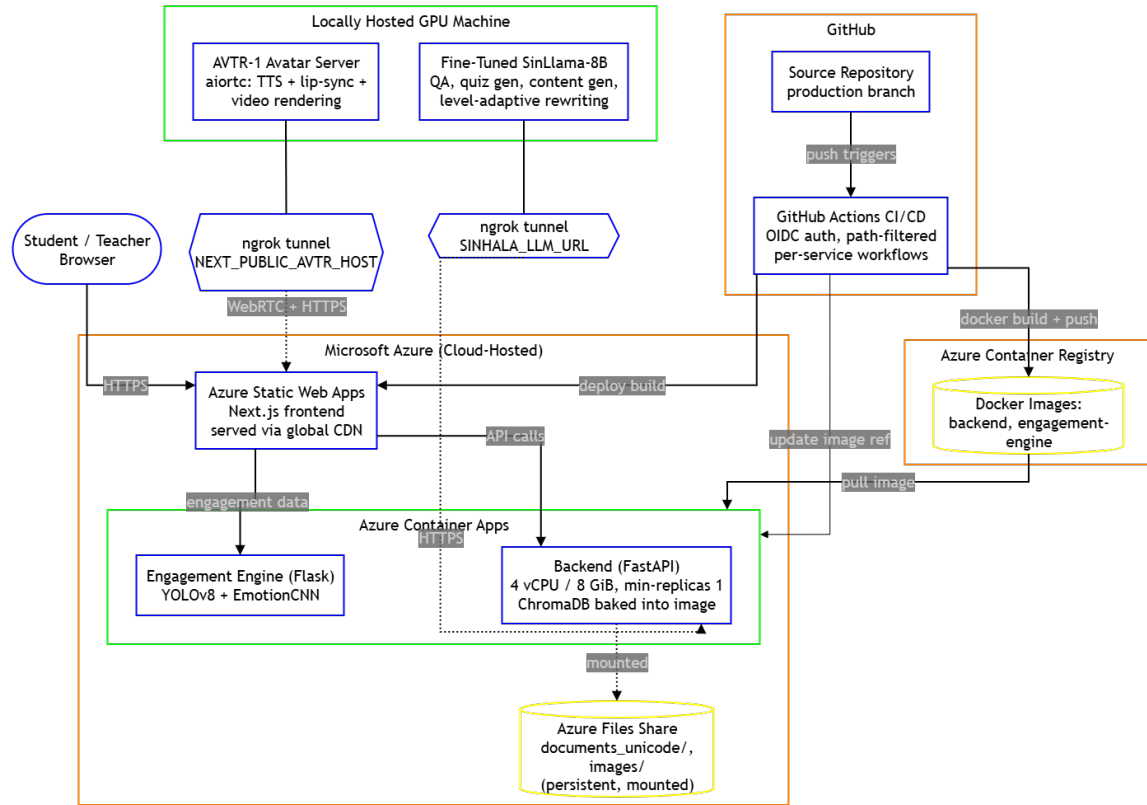


Figure 3.12: System Deployment Architecture

local GPU machine and changing ngrok endpoints, it provides a suitable deployment solution for the current project environment. Future improvements can focus on moving the GPU-based services to dedicated cloud infrastructure and introducing permanent service endpoints, which would make the system more independent, scalable, and suitable for large-scale deployment.

Chapter 4

Results

4.1 Overview

This chapter presents the comprehensive evaluation and final outcomes of the proposed adaptive learning platform. Building upon the methodologies established in the previous chapter, the results are structured into several key areas to demonstrate the system’s overall effectiveness and practical functionality. Initially, the performance of various fine-tuned language models is evaluated to ensure the reliable generation of Sinhala educational content. Following this, the accuracy of the Hybrid-BKT personalization engine and the real-time engagement detection system is analyzed. The chapter then highlights the successful integration of major architectural components, including the RAG pipeline, the multi-agent orchestration framework, and the interactive teaching avatar. Finally, the complete user interfaces for both the web and mobile applications are showcased, illustrating how teachers, parents, and students interact with the finalized platform, before concluding with a summary of the project’s academic achievements.

4.2 Model Fine-Tuning

The primary objective of this experiment was to identify the most suitable LLM for generating Sinhala educational content and accurately answering student queries. Several candidate models were fine-tuned using the curated Sinhala educational dataset and subsequently evaluated to determine which model yielded the optimal performance for the proposed learning platform.

Rigorous model selection was essential because the system must accurately comprehend Sinhala and generate responses that are clear, meaningful, and pedagogically appropriate. Beyond the fundamental quality of the generated text, additional factors—such as semantic similarity to expected reference answers, response generation speed, and overall educational suitability—were also evaluated. This comprehensive approach enabled a balanced comparison from both a performance and practical usability perspective.

The following five LLMs were selected for the evaluation:

Table 4.1: Tested Large Language Models

Large Language Model	Parameters
SinLlama	8B
Qwen	14B
Llama-3.1	8B
DeepSeek	14B
Mistral	7B

To ensure an unbiased comparison, each model was fine-tuned under identical conditions using the same educational dataset. Following the fine-tuning phase, the models were tested on specific Sinhala educational question-answering and content generation tasks. Each model was evaluated on the same held-out set of $N = 200$ Sinhala Buddhism question-answer pairs, none of which were used during the fine-tuning process, ensuring a fair and consistent comparison across all models. The generated outputs were then analyzed using multiple evaluation methodologies to comprehensively assess their respective strengths and limitations.

The following evaluation metrics were utilized:

- **ROUGE Score:** Utilized to measure the direct lexical similarity between the generated responses and the expected reference answers.
- **BERTScore:** Employed to evaluate the semantic similarity between the generated and reference responses, ensuring that the underlying meaning is preserved even if different wording is used.
- **Token Generation Speed:** Measured to assess how rapidly each model could generate responses, which is a critical factor for maintaining real-time interactivity on the platform.
- **LLM-as-a-Judge Evaluation:** Used to qualitatively assess the overall standard of the generated responses based on accuracy, relevance, clarity, and educational utility.

By employing this combination of automatic quantitative metrics and qualitative assessment, a complete and robust comparison of the models was achieved. Based on these comprehensive results, the most capable model was selected for full integration into the adaptive learning platform.

4.2.1 ROUGE Score Evaluation

ROUGE evaluation was used to measure how closely the responses generated by each fine-tuned model matched the expected reference answers. This evaluation was particularly useful for comparing the models because the system is expected to generate answers that contain the important information required for Sinhala educational questions. A higher ROUGE score indicates that the generated response has more similarity with the reference answer.

Three ROUGE measures were considered in this evaluation:

- **ROUGE-1:** Measures the overlap of individual words between the generated response and the reference answer. It provides an indication of whether the model included important words from the expected answer.
- **ROUGE-2:** Measures the overlap of two-word sequences between the generated and reference responses. This provides a stronger indication of whether the model captured important phrases and relationships between words.
- **ROUGE-L:** Measures the longest common sequence between the generated response and the reference answer. It helps evaluate how well the model maintained similar wording or sentence structure while conveying the expected information.

Table 4.2: ROUGE Score Comparison

Large Language Model	ROUGE-1	ROUGE-2	ROUGE-L
SinLlama	0.7714	0.6418	0.6111
Qwen	0.3310	0.1678	0.1059
Llama-3.1	0.3983	0.1949	0.1418
DeepSeek	0.4485	0.2304	0.1845
Mistral	0.4925	0.2280	0.1594

The results show a clear difference between SinLlama and the other evaluated models. SinLlama achieved the highest score in all three ROUGE measures, with a ROUGE-1 score of 0.7714, a ROUGE-2 score of 0.6418, and a ROUGE-L score of 0.6111. These results indicate that SinLlama was able to produce responses that were much closer to the expected educational answers than the other models.

The ROUGE-1 result is particularly notable because a score of 0.7714 indicates a strong overlap of important words between the generated and reference answers. The ROUGE-2 score of 0.6418 also shows that SinLlama was able to capture important word combinations and phrases from the expected responses. Similarly, the ROUGE-L score of 0.6111 suggests that the generated responses maintained a relatively strong sequence-level similarity with the reference answers. Among the remaining models, Mistral produced the second-highest ROUGE-1 score of 0.4925, while DeepSeek achieved 0.4485. However, both models were still considerably below SinLlama. Llama-3.1 and Qwen produced lower scores across all three measures, with Qwen recording the lowest results overall.

Overall, the ROUGE evaluation suggests that SinLlama was the strongest model for the Sinhala educational tasks considered in this experiment. Its consistently higher scores indicate that it was better able to generate responses containing information similar to the expected answers. This result supported the selection of SinLlama as the preferred model for the subsequent stages of the project.

4.2.2 BERTScore Evaluation

BERTScore was used to evaluate the semantic similarity between the answers generated by each fine-tuned model and the corresponding reference answers. While ROUGE mainly measures the overlap of words or phrases, BERTScore focuses more on the meaning of the responses. This is important for educational applications because a student's answer does not always need to use exactly the same words as the reference

answer to be considered correct. Different words or sentence structures can still express the same idea.

BERTScore uses contextual representations of words to determine how closely the meaning of the generated response matches the reference response. Therefore, it provides a useful measure for evaluating whether the models were able to understand the meaning of Sinhala educational content and generate responses with the expected information.

Table 4.3: BERTScore Comparison

Large Language Model	BERTScore
SinLlama	0.9424
Qwen	0.8022
Llama-3.1	0.8799
DeepSeek	0.8659
Mistral	0.8555

The results show that SinLlama achieved the highest BERTScore of 0.9424, which was considerably higher than the scores obtained by the other models. This indicates that the responses generated by SinLlama were semantically closer to the expected reference answers. In other words, even when the generated responses did not use exactly the same wording as the reference answers, they were more likely to convey the same meaning.

Among the other models, Llama-3.1 achieved the second-highest score of 0.8799, followed by DeepSeek with 0.8659 and Mistral with 0.8555. Qwen recorded the lowest BERTScore of 0.8022. The difference between SinLlama and the other models indicates that SinLlama provided stronger semantic matching for the Sinhala educational questions used in the evaluation.

The high BERTScore obtained by SinLlama also supports the findings from the ROUGE evaluation. Both evaluation methods showed that SinLlama was able to produce responses that were closer to the expected educational answers. This suggests that the fine-tuned SinLlama model was better suited to understanding the meaning of domain-specific Sinhala educational content and generating relevant responses. Therefore, the BERTScore results further supported the selection of SinLlama for integration into the proposed learning platform.

4.2.3 Token Generation Speed Evaluation

Token generation speed was evaluated to determine how quickly each model could generate a response after receiving an input. This is an important factor for the proposed learning platform because students interact with the system during learning activities and need responses within a reasonable time. A model that provides accurate answers but takes too long to respond could negatively affect the learning experience.

The generation speed of each fine-tuned model was measured in tokens per second (tokens/sec). The model size was also recorded to provide an indication of the computational resources required by each model during inference.

Table 4.4: Token Generation Speed Comparison

Large Language Model	Size (GB)	Token/sec
SinLlama	6.1	62.6
Qwen	10.3	45.1
Llama-3.1	16.4	55.4
DeepSeek	10.3	444.9
Mistral	4.0	37.5

The results show a clear difference in the generation speed of the evaluated models. DeepSeek achieved the highest speed of 444.9 tokens per second, which was significantly higher than all other models. SinLlama generated 62.6 tokens per second, followed by Llama-3.1 at 55.4 tokens per second and Qwen at 45.1 tokens per second. Mistral recorded the lowest speed at 37.5 tokens per second.

Although DeepSeek performed significantly better in terms of generation speed, speed alone was not considered sufficient for selecting the final model. The main purpose of the platform is to provide accurate, relevant, and meaningful educational support to students. Therefore, response quality and semantic understanding were also important factors in the model selection process. When the token generation results are considered together with the ROUGE and BERTScore results, SinLlama provides a better overall balance between response quality and generation speed. Although its generation speed of 62.6 tokens per second was much lower than DeepSeek, it was still suitable for interactive learning. At the same time, SinLlama achieved considerably higher ROUGE and BERTScore values, indicating stronger response quality and semantic similarity.

Therefore, the results show that SinLlama provided the most suitable balance between response quality and generation speed for the proposed learning platform. While DeepSeek was the fastest model, SinLlama was preferred because its stronger educational response quality was more important for the intended use of the system.

4.2.4 LLM-as-a-Judge Evaluation

To further assess the quality of the responses generated by the fine-tuned models, an LLM-as-a-Judge evaluation was conducted. This evaluation was used to assess the responses from an educational perspective rather than only measuring their similarity to reference answers. This was important because a response may have good word or semantic similarity but still lack sufficient explanation or clarity for a student.

Each model-generated response was evaluated using four criteria: Accuracy, Completeness, Clarity, and Depth. Accuracy measures whether the information provided in the response is correct. Completeness considers whether the response includes the important information needed to answer the question. Clarity measures how easily a student can understand the response, while Depth considers how well the response explains the topic and whether it provides sufficient detail. Each criterion was scored from 1 to 5, giving a maximum possible total score of 20 for each model.

Table 4.5: LLM-as-a-Judge Evaluation

Model	Accuracy	Completeness	Clarity	Depth	Total/20
SinLlama	4.78	4.03	4.50	2.38	15.88
Qwen	2.97	2.59	4.34	1.84	11.75
Llama-3.1	4.28	2.72	4.34	2.25	13.59
DeepSeek	4.22	2.91	4.36	2.09	13.58
Mistral	4.50	3.59	4.34	2.53	14.97

The results show that SinLlama achieved the highest overall score of 15.88 out of 20. It also achieved the highest accuracy score of 4.78, indicating that its generated responses were more reliable and factually appropriate for the educational questions used in the evaluation. SinLlama also achieved the highest clarity score of 4.50, showing that its responses were generally easy to understand and suitable for student use. Its completeness score of 4.03 was also the highest among the evaluated models, indicating that it was able to include a larger portion of the important information required in the answers.

Although SinLlama achieved a lower depth score of 2.38 than Mistral, which recorded the highest depth score of 2.53, its overall performance was stronger when all four criteria were considered together. Mistral achieved a total score of 14.97, while SinLlama achieved 15.88, mainly due to its stronger performance in accuracy, completeness, and clarity. Qwen recorded the lowest overall score of 11.75, while Llama-3.1 and DeepSeek achieved similar total scores of 13.59 and 13.58, respectively.

Overall, the LLM-as-a-Judge evaluation further supports the results obtained from the ROUGE and BERTScore evaluations. SinLlama consistently demonstrated stronger performance in producing accurate, relevant, and understandable Sinhala educational responses, making it a suitable candidate for the proposed learning platform.

4.2.5 Selected Model for the Proposed System

Based on the results from all four evaluation methods, SinLlama 8B was selected as the final LLM for the proposed adaptive learning system. The decision was not based on a single evaluation measure but on the overall performance of the models across response similarity, semantic understanding, generation speed, and educational quality.

SinLlama achieved the highest ROUGE scores across ROUGE-1, ROUGE-2, and ROUGE-L, indicating strong similarity between its generated responses and the expected answers. It also obtained the highest BERTScore of 0.9424, demonstrating strong semantic similarity with the reference responses. In the LLM-as-a-Judge evaluation, SinLlama achieved the highest overall score of 15.88/20, with particularly strong results in accuracy, completeness, and clarity. Although DeepSeek achieved a much higher token generation speed, SinLlama’s 62.6 tokens per second was considered sufficient for interactive learning.

The comparison showed that some models performed better in individual areas. For example, DeepSeek had the highest generation speed, while Mistral achieved a slightly higher depth score. However, SinLlama provided the strongest overall balance across the evaluation criteria, particularly in accuracy, semantic understanding, and

educational response quality.

This consistent advantage across all four evaluation methods can be explained by SinLlama’s architecture: unlike Qwen, Llama-3.1, DeepSeek, and Mistral, which are general-purpose multilingual models, SinLlama is the first large language model specifically extended for Sinhala. It is built on Meta-Llama-3-8B and adapted through tokenizer vocabulary extension and continual pretraining on a 10-million-sentence Sinhala corpus, giving it substantially stronger native-language grounding than models that only support Sinhala as one of many languages. This explains why SinLlama consistently outperformed larger models such as Qwen-14B and DeepSeek-14B despite having fewer parameters-parameter count matters less than Sinhala-specific pretraining for this task.

Therefore, the fine-tuned SinLlama 8B model was selected and integrated into the final system to support Sinhala question answering, educational content generation, and personalized learning activities.

4.3 Personalization Performance Evaluation

The personalization and adaptive learning capabilities of the proposed system are driven by a novel Hybrid Bayesian Knowledge Tracing (Hybrid-BKT) engine. As outlined in the methodology, this engine combines a Personalized Clustered BKT (PC-BKT) approach with a Long Short-Term Memory (LSTM) neural network. This integration allows the system to not only estimate student mastery based on personalized prior probabilities and empirical learning rates but also capture temporal learning dynamics, such as forgetting or sudden performance drops.

To evaluate the effectiveness of this approach, the Hybrid-BKT engine was assessed using an experimental dataset consisting of over 6,500 student interactions across 47 Buddhism Knowledge Components. The performance of the proposed Hybrid-BKT model was compared against a Standard BKT model and a baseline BKT-LSTM model. The evaluation focused on three primary metrics: Area Under the Curve (AUC), Accuracy, and Root Mean Square Error (RMSE).

Table 4.6: Performance Comparison of BKT Variants

Metric	Standard BKT	BKT-LSTM	Hybrid-BKT (Proposed)
AUC	0.60	0.59	0.60
Accuracy	48.1%	54.7%	56.2%
RMSE	0.51	0.53	0.49

The evaluation results demonstrate the advantages of the hybrid architecture. The proposed Hybrid-BKT model achieved the highest predictive accuracy at 56.2%, outperforming both the Standard BKT (48.1%) and the baseline BKT-LSTM (54.7%). Furthermore, the Hybrid-BKT model recorded the lowest RMSE of 0.49, compared to 0.51 for Standard BKT and 0.53 for BKT-LSTM. A lower RMSE signifies a reduction in prediction errors, showcasing the benefits of incorporating structural BKT with temporal LSTM elements to reliably determine a student’s actual mastery level at any given time point.

Several key observations further support the effectiveness of the proposed personalization mechanisms:

- **Improved Cold-Start Predictions:** The cold-start approach, based on clustering prior knowledge as described in the PC-BKT methodology, led to a 12% lower prediction error compared to standard BKT models that rely on default priors.
- **Inference Efficiency:** The inference latency time remained below 200 ms per update. This rapid processing speed is highly suitable for practical implementations within the interactive, quiz-based learning platform, ensuring that mastery scores and adaptive content generation occur in real-time.
- **Training Stability:** The LSTM training process exhibited stable convergence, with the loss decreasing from 0.58 to 0.28 within 50 iterations.

Finally, the empirical BKT parameter ranges observed during the evaluation were:

$$\begin{aligned} P(\text{Guess}) &= 0.121 [0.010 - 0.300] \\ P(\text{Slip}) &= 0.157 [0.010 - 0.290] \\ P(\text{Transition}) &= 0.464 [0.387 - 0.500] \end{aligned}$$

By accurately predicting student mastery and maintaining low inference latency, the personalization engine is able to dynamically categorize learners into Expert, Advanced, Intermediate, or Beginner levels. This reliable categorization ensures that the RAG-integrated content creation agents can generate and deliver educational materials that are perfectly aligned with each student's current cognitive state, thereby providing a deeply personalized adaptive learning experience.

4.4 Engagement Detection System Performance Evaluation

The effectiveness of the proposed engagement detection system heavily relies on the accuracy and robustness of its underlying computer vision models. To validate the system's performance, the evaluation was divided into two primary analytical components: the YOLOv8 model, responsible for precise object and facial feature detection, and the EmotionCNN model, which handles the complex task of facial expression recognition. Both models underwent rigorous testing to ensure they could consistently handle real-world classroom conditions, such as varying lighting, diverse seating arrangements, and partial occlusions, thereby meeting the high reliability standards required for real-time student engagement monitoring.

4.4.1 YOLOv8 Performance Comparison

To achieve optimal detection capabilities within a dynamic educational context, it was necessary to move beyond off-the-shelf pre-trained models. A fine-tuned version of YOLOv8 was developed and compared against the baseline YOLOv8 model. The fine-tuning process utilized a specialized custom dataset, heavily curated to represent actual classroom environments, complex student postures, and various camera angles typical of a learning setting.

Table 4.7: YOLOv8 Performance Comparison

Metric	Base YOLOv8	Fine-Tuned YOLOv8	Improvement
mAP@50	0.737	0.833	+0.096
Precision	0.827	0.858	+0.031
Recall	0.659	0.687	+0.027

As illustrated in the results, the fine-tuned YOLOv8 model demonstrated substantial improvements across all key evaluation metrics, validating the custom training approach.

The most significant enhancement was observed in the Mean Average Precision at an Intersection over Union (IoU) threshold of 0.50 (mAP@50), which surged from 0.737 to 0.833. This substantial improvement of 0.096 indicates that the fine-tuned model is vastly superior at accurately localizing students and placing bounding boxes tightly around relevant features, even in crowded classroom settings.

Furthermore, the model’s Precision saw a notable gain of 0.031, reaching 0.858. In the context of engagement detection, high precision ensures that the system minimizes false positives—meaning it is less likely to mistake inanimate objects or background posters for actual students. Similarly, the Recall metric improved by 0.027 to reach 0.687. This improvement translates to a reduction in false negatives, ensuring that the system successfully detects a larger proportion of the actual students present, including those seated in the back rows or partially obscured by desks.

4.4.2 EmotionCNN Performance

While YOLOv8 isolates the student faces, the EmotionCNN model is tasked with interpreting the subtle, nuanced micro-expressions that indicate a student’s level of engagement, such as confusion, focus, or boredom. Evaluating this model is crucial, as facial expression recognition is inherently challenging due to natural human variations in displaying emotion.

Table 4.8: EmotionCNN Performance Evaluation

Metric	Performance
Overall Accuracy	65.97%
Top-2 Accuracy	83.69%
F1-Score	61.41%
Weighted F1-Score	65.09%

The EmotionCNN model achieved a solid overall accuracy of 65.97% when predicting the exact primary emotion of a student. Given the complexity of classifying nuanced cognitive states in a non-controlled environment, this represents a strong baseline performance.

More importantly, the model achieved an impressive Top-2 Accuracy of 83.69%. This metric signifies that in over 83% of cases, the student’s actual emotion was within the model’s top two predictions. This is particularly valuable in practical educational settings where emotional states often overlap and can be subjectively ambiguous (for example, the visual difference between a ”neutral” resting face and a

deeply "concentrated" expression can be minimal). The high Top-2 accuracy ensures the system rarely misinterprets the broader context of a student's engagement level.

Additionally, standard accuracy metrics can sometimes be skewed by class imbalances (e.g., students spend the majority of a lecture looking "neutral," making it the dominant class). To address this, the F1-Score and Weighted F1-Score were analyzed. The model recorded a standard F1-Score of 61.41% and a Weighted F1-Score of 65.09%. The higher Weighted F1-Score confirms that the model maintains reliable predictive capabilities across all emotion categories, appropriately penalizing misclassifications in less frequent, but highly critical states like "frustration" or "confusion".

Collectively, these metrics validate that the fine-tuned detection pipeline—comprising both YOLOv8 and EmotionCNN—is robust, highly sensitive to educational environments, and fully capable of driving the real-time engagement analytics engine.

4.5 System Implementation Outcomes

4.5.1 Retrieval-Augmented Generation (RAG) Pipeline Outcomes

To ensure the AI consistently provides correct and reliable information, a RAG pipeline was successfully implemented. First, official Sinhala textbooks and past papers were digitized, broken down into small, readable chunks, and stored securely in a ChromaDB vector database. When a student asks a question or requests a lesson, the system quickly searches this database to find the most relevant educational material. By feeding this exact context to the fine-tuned SinLlama model, the AI is effectively prevented from making up false information (hallucinating). The result is a highly accurate system that consistently stays true to the national curriculum, generating trustworthy and curriculum-aligned lessons for every student.

4.5.2 Multi-Agent System Orchestration

Managing a personalized learning platform requires coordinating multiple dynamic components; therefore, a multi-agent system powered by LangGraph was integrated into the platform. Instead of forcing every student through a rigid, step-by-step process, a central Orchestrator acts as an intelligent supervisor. It continuously evaluates the student's needs and routes tasks to specialized agents—such as the Evaluator to grade a quiz, or the Content Generator to create a new lesson. Because these agents share a common 'learning state,' they collaborate seamlessly. This coordinated approach ensures the system can make real-time decisions, instantly adapt to the student's performance, and deliver a smooth learning experience without noticeable background delays.

4.5.3 Real-Time Avatar Speech Delivery

To make learning more interactive and engaging, an AI-powered teaching avatar that speaks directly to the students was successfully integrated. This was implemented using the AVTR-1 streaming avatar approach. Initially, the Explain Agent takes complex textbook lessons and rewrites them into a friendly, conversational script. The system then converts this script into natural-sounding Sinhala speech, perfectly

synchronizing the avatar's lip movements with the audio. Additionally, real-time playback controls were incorporated, allowing students to easily pause, resume, or adjust the avatar's speaking speed whenever they need to take notes, all without interrupting the live streaming connection.

4.6 User Interfaces in Web Application

4.6.1 Public Landing

Figure 4.1 shows the landing page, which is the main entry point to the System. It introduces the system as an AI-powered adaptive learning platform designed to provide a personalized learning experience based on each learner's level, pace, and goals. The left side of the page presents the system name and its main purpose, along with feature tags such as AI-Powered, Adaptive, Progress Tracking, and Multi-Level, giving users a quick understanding of the system's key capabilities. On the right side, users can select between two roles, Student and Administrator. Students can access the learning dashboard and begin their personalized learning activities, while administrators can manage students and view overall system analytics. This role-based design directs users to the appropriate interface and provides the foundation for role-based navigation throughout the platform.

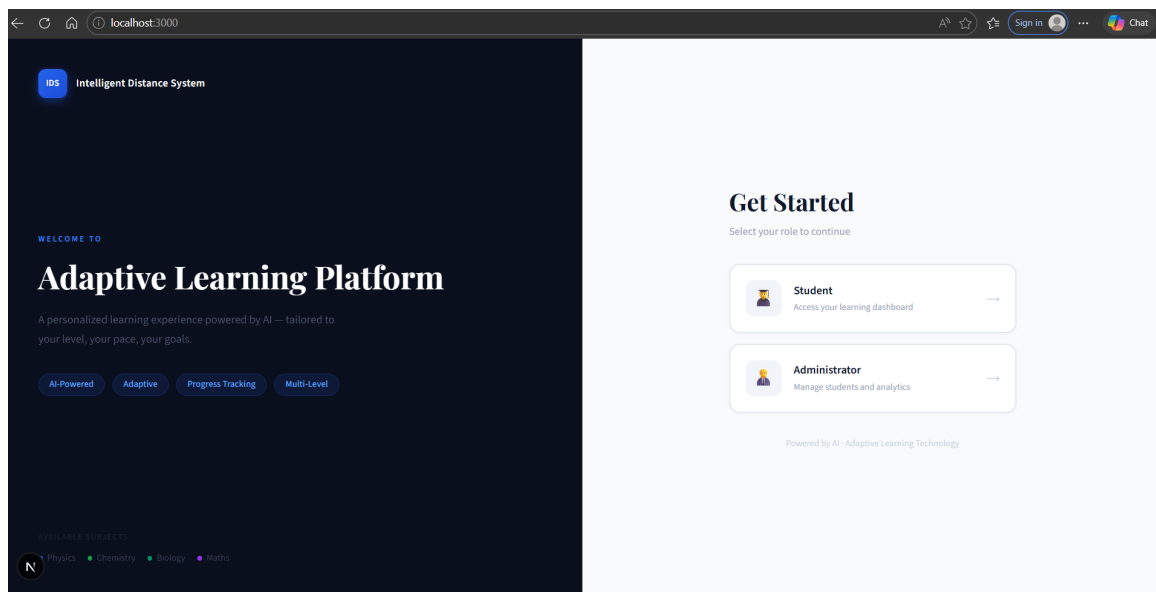


Figure 4.1: Landing page

4.6.2 Account Registration and Login

As shown in Figure 4.2, the login page allows registered students to securely access the platform using their email address and password. The page has a simple and clean design, with clearly labeled input fields and a prominent Sign In button for submitting the login details. New users can navigate to the Sign Up page (Figure 4.3) to create an account, while the Back to Home option allows users to return to the landing page. The page also maintains the platform's light, network-themed design to

match its AI-based branding. After successful login, the student is redirected to their personalized dashboard, where they can access the available learning activities.

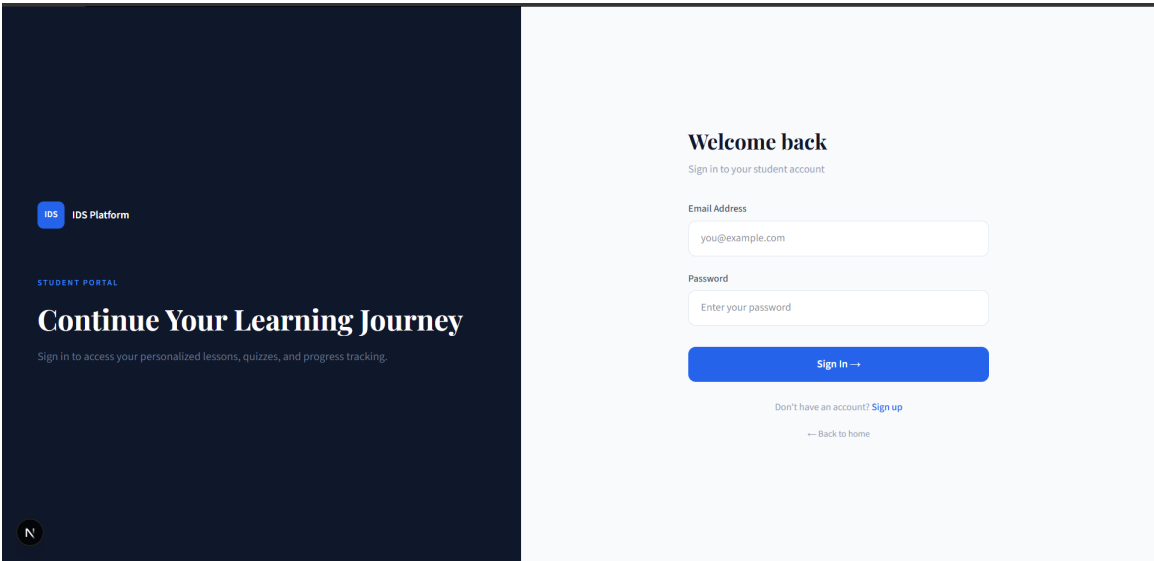


Figure 4.2: Student Login page

As shown in Figure 4.3, the student signup page provides a simple and welcoming interface for new users to register on the platform. To create an account, students just need to enter their basic details—such as their name, email address, and a secure password. The form is kept short and easy to fill out so that students can quickly get started with their learning journey. If a user already has an account, they can easily switch back to the login page using the link at the bottom. Once registered, the system sets up a fresh profile and directs the student to their new dashboard to begin exploring the adaptive lessons.

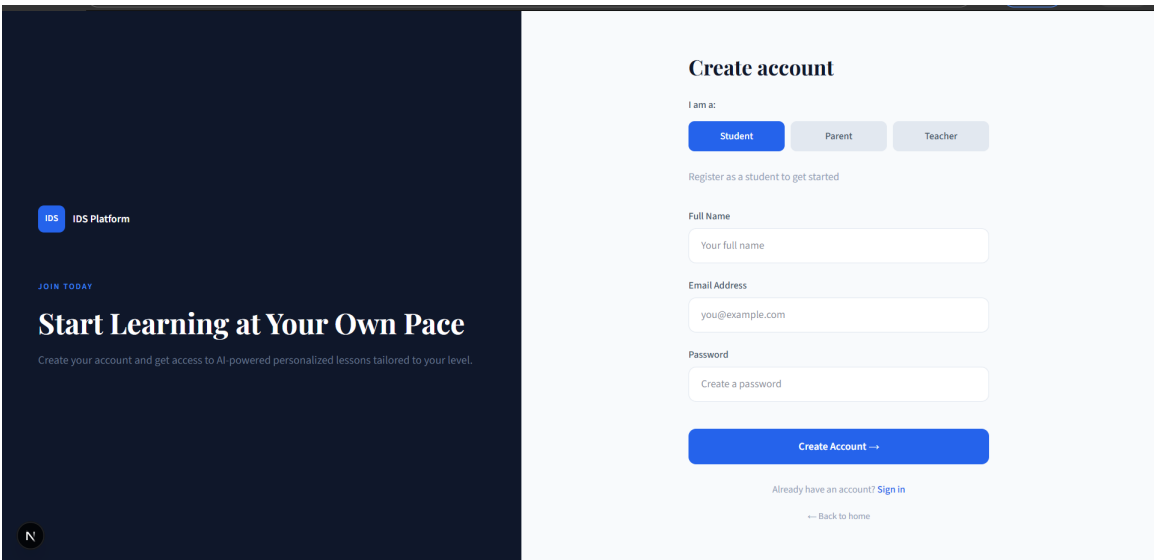


Figure 4.3: Student Signup page

Figure 4.4 shows the admin login page, which provides a dedicated and secure entry point for teachers and system administrators. Unlike the student login, this page is

specifically built for educators to access the management portal of the platform. Once signed in, administrators can manage registered students, track lesson completion progress, and monitor overall quiz performance. The design remains clean and consistent with the rest of the application, ensuring that teachers can quickly log in and get right to managing their classrooms without any hassle.

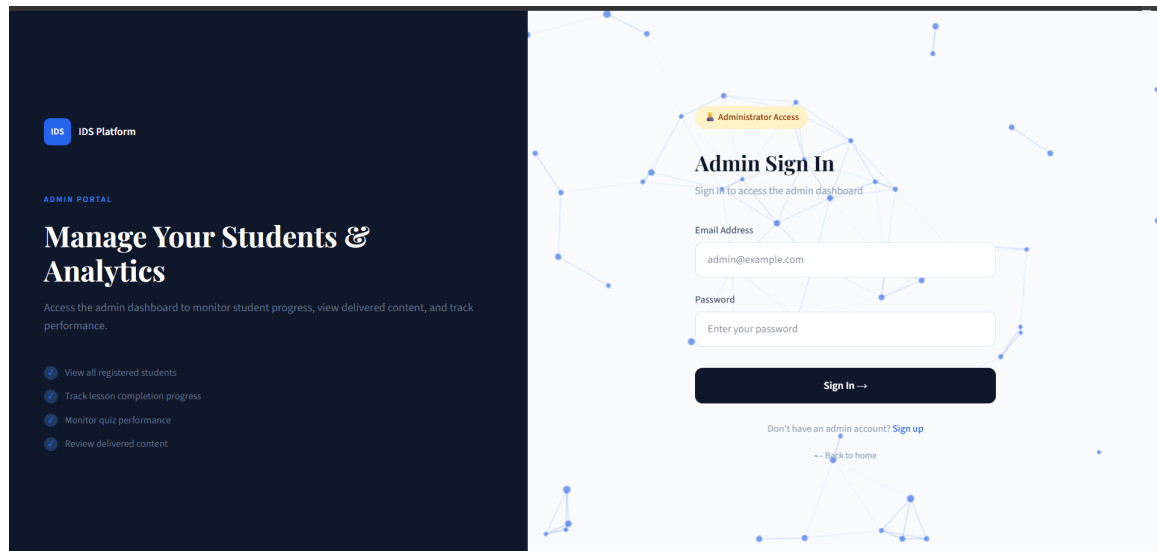


Figure 4.4: Admin Login page

As shown in Figure 4.5, the admin signup page allows new teachers or system administrators to create their own accounts on the platform. It asks for basic information such as a name, an email address, and a password, ensuring that the registration process is quick and easy. Just like the student signup, the design is kept clean and simple so that new staff members can get their accounts set up without any confusion. A link is also provided at the bottom for those who already have an admin account, allowing them to easily jump back to the login page.

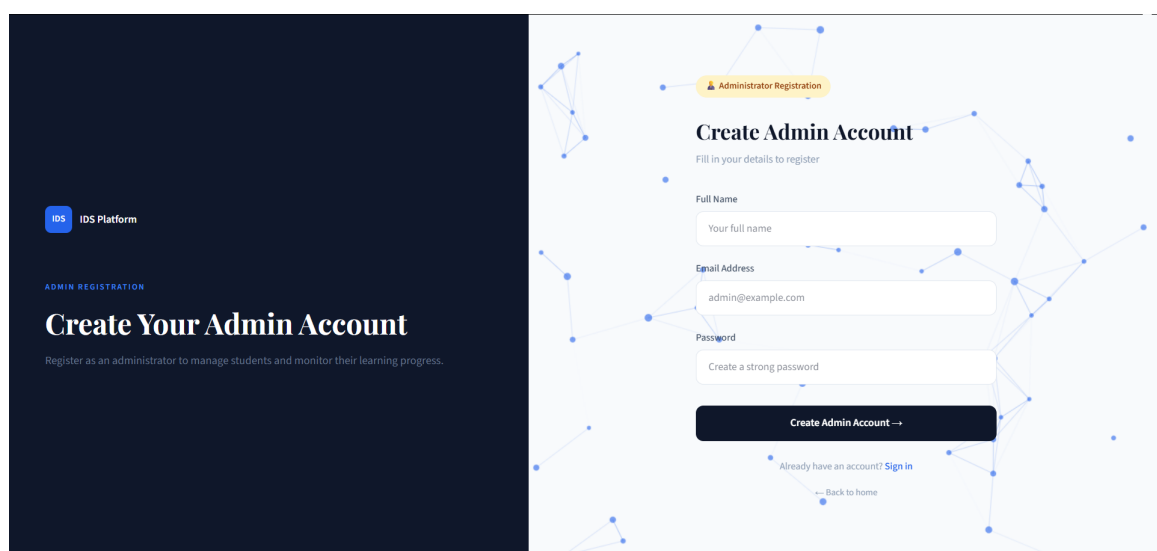


Figure 4.5: Admin Signup page

4.6.3 Student Interface

As shown in Figure 4.6, the Dashboard is the main page students see after logging in. It starts with a personalized welcome message, such as "Hello, Kasun Gunasekara," displayed on a deep-blue banner with a network-style background. The header also shows four summary details: Grades, Subjects, Lessons, and Topics, giving students a quick overview of the available learning content. On the left side, the My Subjects section shows the subjects the student has joined, along with a small progress bar that indicates how many topics have been completed, such as 0/42. The dashboard also includes a "Link to your teacher" section where students can enter a teacher code after registration if they have not already done so. This allows the system to connect the student with the correct teacher and related lessons and progress records. In the main section, Your Curriculum allows students to select a grade, such as Grade 11, 12, or 13, using the available tabs and then view the subjects offered for that grade. Each subject is displayed as a card with its name, icon, and number of lessons, such as "6 Lessons" for Economics. When a student selects a subject, they are taken to the lesson list, where they can continue to the topics and adaptive pre-quiz. A floating chat icon at the bottom-right also gives students quick access to the Q&A assistant whenever they need help.

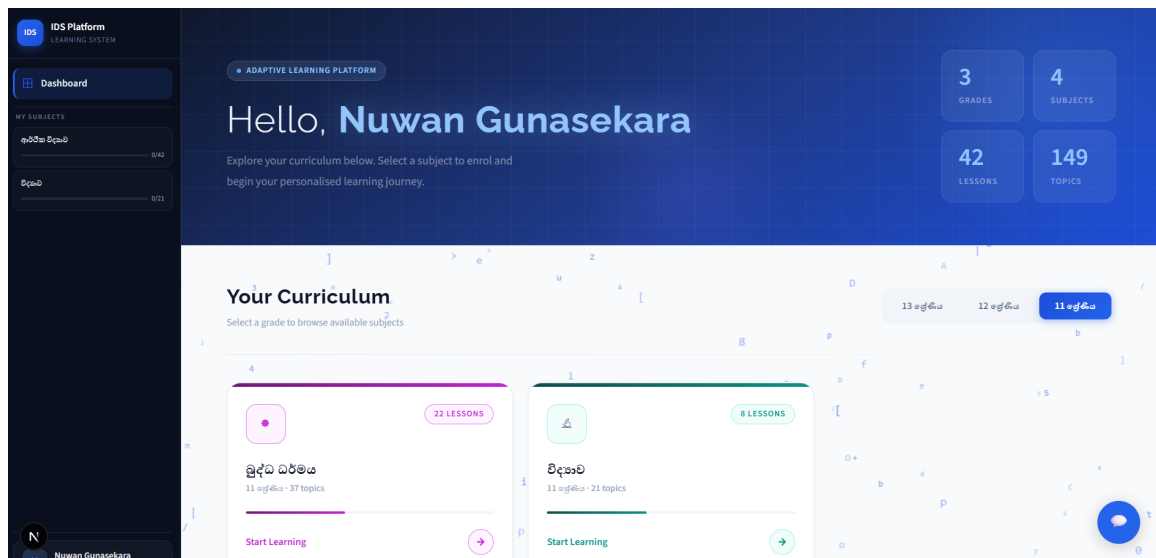


Figure 4.6: Student Dashboard page

As shown in Figure 4.7, after selecting a subject from the Dashboard, the student is taken to the Available Lessons page, which shows the lessons available for the selected subject. For example, for Grade 11 Science, the page displays a breadcrumb such as Grade 11 Science to show the student's current location in the curriculum, along with a Dashboard button to return to the main page. The header also provides a quick overview of the subject by showing the total number of Lessons and Topics, such as 8 lessons and 21 topics. Below this, each lesson is displayed as a numbered card from 01 to 08, showing the lesson title and the number of topics included in it. Small dots are also used to visually represent the lesson structure or progress. Each card has an arrow that allows the student to open the selected lesson. When a lesson is selected, the student can view its individual topics and continue to the adaptive pre-quiz and personalized learning process. Therefore, this page acts as a simple navigation step

between selecting a subject and accessing individual topics.

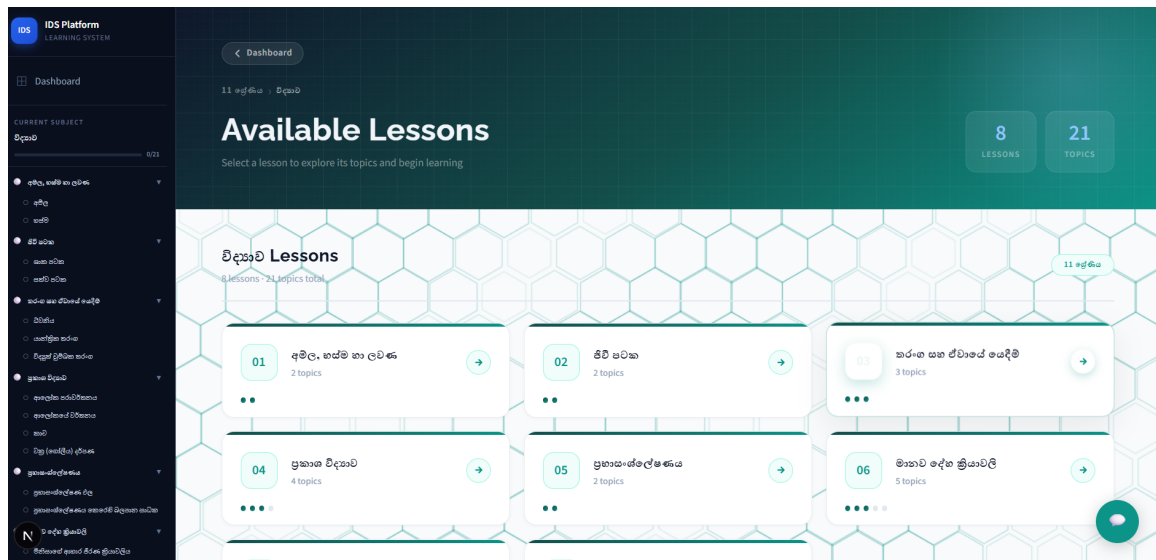


Figure 4.7: Available Lessons page

As shown in Figure 4.8, once a student selects a specific topic, they are directed to the Quiz Preparation page before the learning material is presented. This page acts as a readiness check, informing the student about the upcoming adaptive pre-quiz. It outlines the purpose of the quiz, which is to assess their current understanding of the topic so the system can tailor the subsequent lesson content to their specific level. The interface is simple and encouraging, featuring clear instructions and a prominent button to begin the quiz, ensuring the student feels prepared and understands the adaptive nature of the platform.

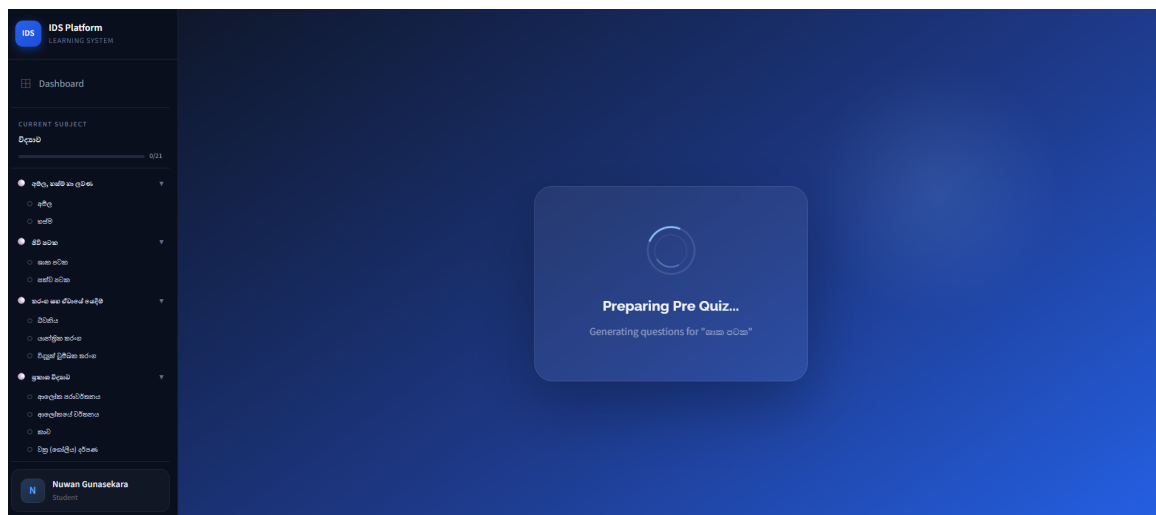


Figure 4.8: Quiz Preparation page

Figure 4.9 shows the pre-lesson quiz page, where the system assesses the student's initial knowledge. The interface presents multiple-choice questions one at a time to keep the student focused. Each question is clearly displayed, and the student can select their answer from the provided options before moving to the next one. This

short quiz helps the AI understand exactly what the student already knows and what they need help with, ensuring the upcoming lesson is perfectly tailored to their level.

IDS Platform
Learner Portal

MAIN MENU
Dashboard

මුද්‍රිත වර්ග 11 ලේඛන - මුද්‍රණ අත්‍යවශ්‍ය

Pre Lesson Quiz

Topic: ගිණුම්පත්පත්‍රය - 3 questions

Q1. මුද්‍රණ අත්‍යවශ්‍ය පාඩමේ ප්‍රධාන මාතෘකාව කුමක්ද?

- A. නොකඩඉම් පාර්ශ්ව
- B. සිල පාර්ශ්ව
- C. ගිණුම්පත්පත්‍රය
- D. දාන පාර්ශ්ව

Q2. ගිණුම්පත්පත්‍රය යටතේ රෝගී වූ මහණ බමුණන්ට උපස්ථාන කිරීම මුද්‍රණ අත්‍යවශ්‍ය පාඩමේ කුමක්ද?

- A. වර්ග ප්‍රස්ථාර
- B. මුද්‍රිත ප්‍රස්ථාර
- C. ආමාස ප්‍රස්ථාර
- D. දාන ප්‍රස්ථාර

Q3. ගිණුම් මහණ බමුණන් උදෙසා කළ යුතු ප්‍රශ්න කීයක්ද?

- A. ඔවුන් අනතුරු යාම
- B. ඔවුන්ගේ දේපළ පාහැර ගැනීම
- C. ඔවුන්ට භික්ෂා කිරීම
- D. උපස්ථාන කිරීම

Submit Quiz →

Nimesh Perera
Logout

Figure 4.9: Pre Lesson Quiz page

Once the quiz is completed, the student is taken to the analysis page, as shown in Figure 4.10. During this brief wait, the system processes their quiz answers in real-time. A friendly message is displayed to let the student know that the AI is actively evaluating their performance. Behind the scenes, the platform is figuring out the best difficulty level and learning path to present to the student next.

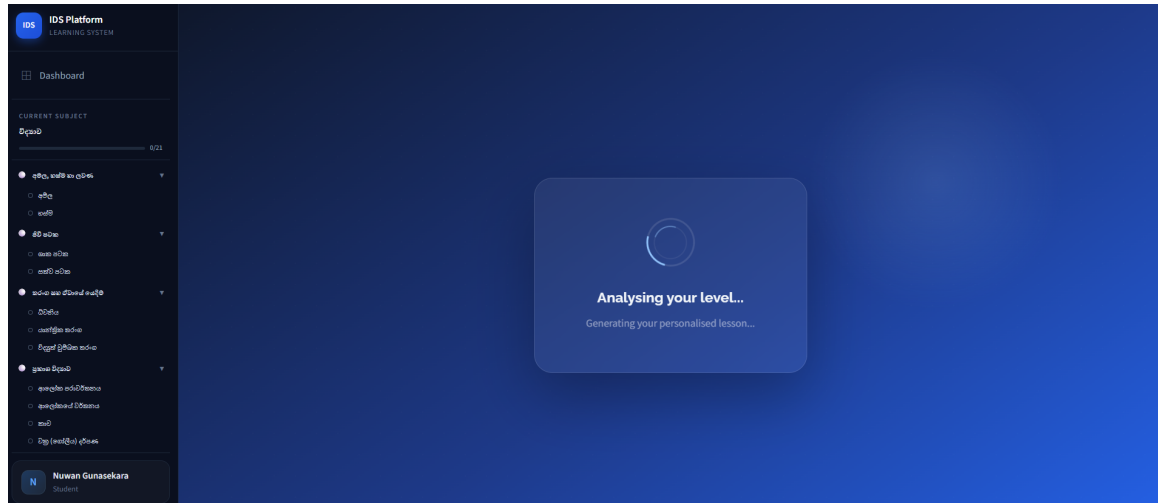


Figure 4.10: Student Level Analyzing page

Finally, Figure 4.11 displays the results of the pre-quiz. This page gives the student immediate feedback on how they performed, highlighting their score and the specific knowledge level the system has assigned them. By clearly showing these results, the student understands why certain lesson materials will be presented to them. From here, they can confidently proceed to start their personalized learning session.

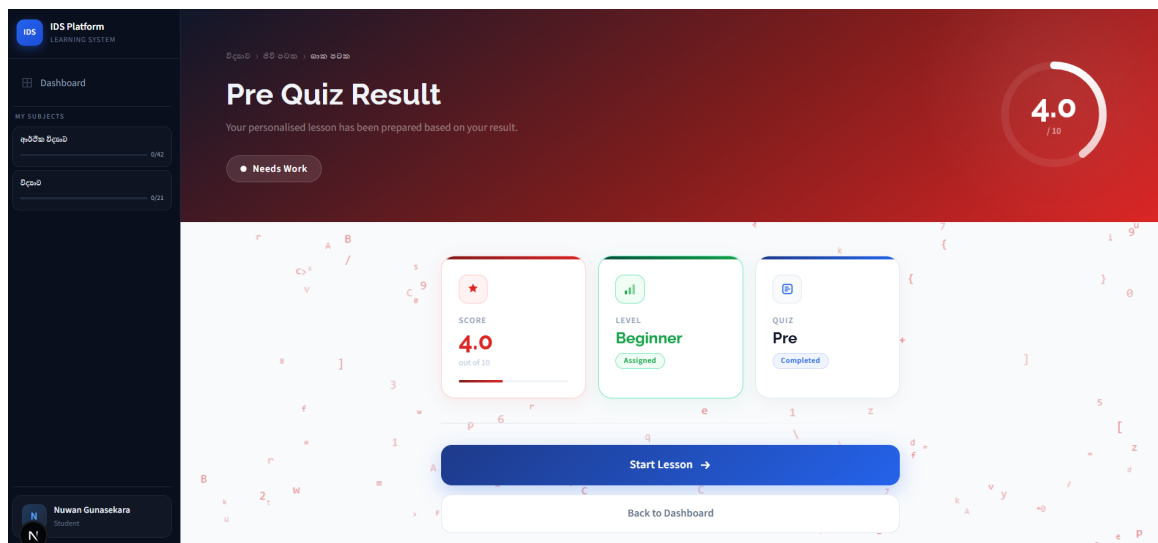


Figure 4.11: Pre Quiz Result page

As shown in Figure 4.12, the Lesson Delivering Page is the main interface where students receive their personalized learning content. It brings together the platform's adaptive content generation, AI teaching avatar, and engagement monitoring features in one learning environment. Students are directed to this page after completing the pre-lesson quiz. The quiz result is used to determine the student's learning level, such as Beginner, Intermediate, or Advanced, and the selected level is displayed as a difficulty badge at the top of the page.

The page header also provides two additional features. The "Watch a Video" option allows students to access relevant YouTube videos related to the current lesson for additional explanations. The "Engagement" indicator shows that the webcam-based

engagement monitoring feature is active. This allows the system to monitor the student's attentiveness during the lesson without requiring the student to perform any additional actions.

The main learning area is divided into two sections. The left section contains the AI teaching avatar, while the right section displays the lesson content. The avatar section provides two different presentation modes. The first is AVTR-1, which provides a real-time video-based avatar streamed through WebRTC from the dedicated AVTR-1 rendering server. This mode provides spoken narration together with lip-synchronized avatar movements. The second option is the Full Body 3D avatar, which is rendered directly in the browser and uses Gemini text-to-speech for narration. Students can also select their preferred avatar character and voice. In addition, the narration speed can be adjusted between $0.75\times$ and $1.5\times$, allowing students to control the pace of the lesson according to their preference.

The right section contains the actual lesson material provided to the student. The content is first retrieved from the educational knowledge base using the Retrieval-Augmented Generation (RAG) pipeline. The retrieved information is then processed by the fine-tuned SinLlama model, which rewrites the content according to the student's assigned learning level. The lesson therefore provides the same underlying educational information in a form that is appropriate for the student's current level. The interface also displays an estimated reading time and the selected difficulty level.

Images and diagrams extracted from the original educational materials are included within the lesson content where they are relevant. This allows students to view the visual information together with the related explanation instead of having to access the images separately. During avatar narration, the corresponding section of the lesson text can also be highlighted to support a read-along learning experience, helping students connect the spoken explanation with the written content.

The page also provides quick access to a Q&A assistant through floating action buttons. Students can use this feature to ask questions or request clarification about the lesson without leaving the current page. This allows additional interaction with the AI tutor while the lesson is in progress.

After completing the lesson, the student can proceed to the post-lesson quiz using the action provided at the bottom of the page. The quiz is used to reassess the student's understanding after receiving the personalized lesson content.

Overall, the Lesson Delivering Page represents an important part of the platform's adaptive learning process. It combines personalized content generation, RAG-based information retrieval, AI avatar narration, interactive question answering, and engagement monitoring within a single interface. This enables the system to provide students with a learning experience that is not only personalized to their current knowledge level but also interactive and continuously connected to the assessment and progress-tracking components of the platform.

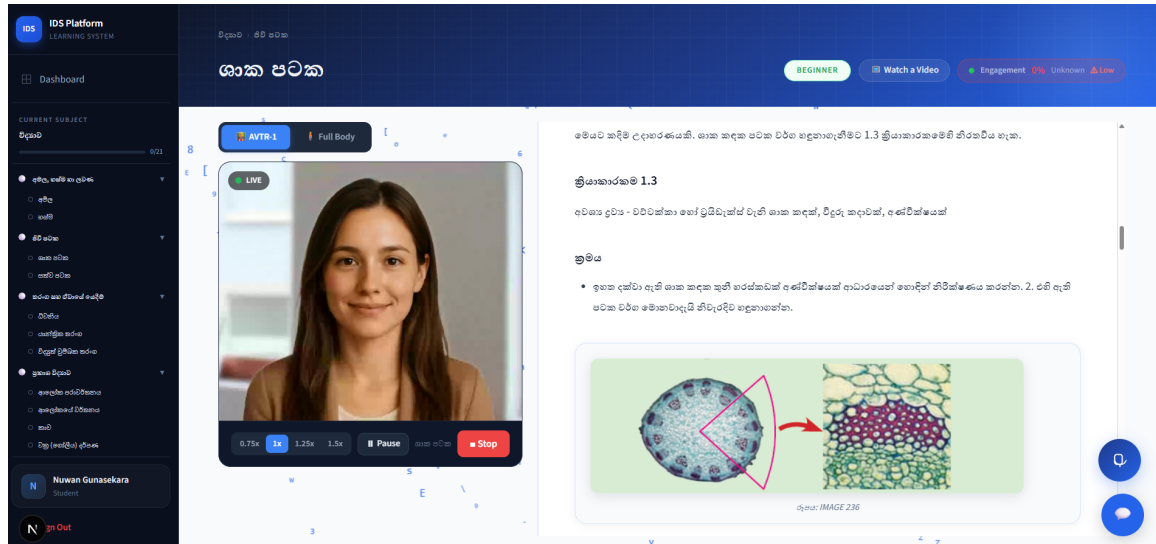


Figure 4.12: Lesson Delivering page

To further enrich the learning experience, the platform includes a YouTube feature page, as shown in Figure 4.13. This section seamlessly integrates educational video content directly into the lesson flow. When text and images aren't enough to grasp a complex topic, students can watch curated video tutorials without ever leaving the application. This multi-media approach keeps students engaged and caters to visual learners, providing an alternative way to understand the material before moving on to the next topic.

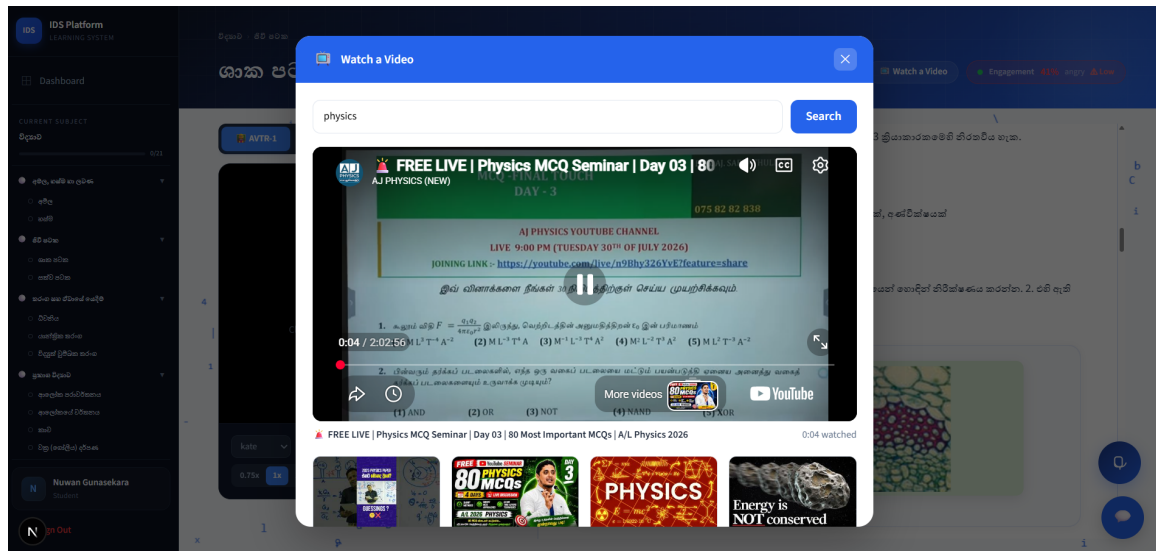


Figure 4.13: Youtube Feature page

Figure 4.14 illustrates the engagement detection interface. While the student learns, the system monitors their attention and focus levels in real-time. By tracking metrics such as eye movement and facial expressions, the platform ensures the student remains attentive. If their focus begins to drop, the system can prompt them or adjust the lesson flow to keep them engaged. This feature adds an extra layer of interactive monitoring to support effective learning.

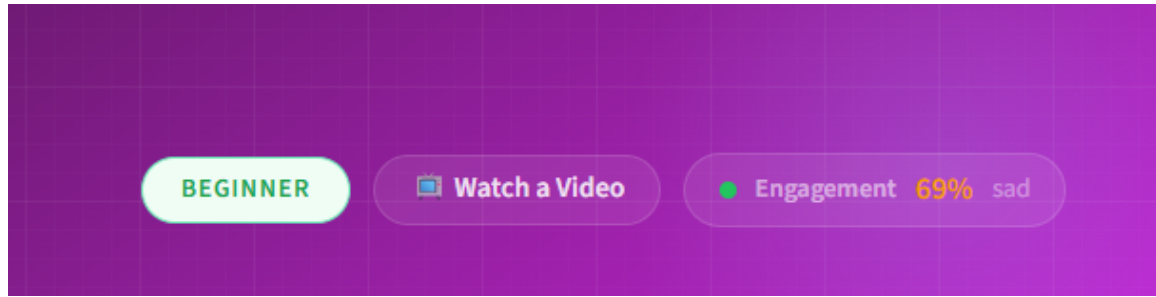


Figure 4.14: Engagement Detection

4.6.4 Teacher Interface

As shown in Figure 4.15, the Teacher Dashboard, shown as “Student Analytics” in the Admin Panel, is designed to help teachers monitor the students connected to their accounts and review their learning progress. Through this dashboard, teachers can view student performance, quiz results, topic progress, and other learning-related information. Unlike the student interface, the teacher dashboard uses a darker and more information-focused design to support monitoring and administrative activities. Teachers access this section through a separate administrator login, which keeps teacher functions and student functions separated within the system.

Navigation and Teacher Account-The dashboard contains a persistent left-side navigation bar that provides access to the main teacher functions. It includes the Dashboard, where student analytics can be viewed, and the Add Content section, which allows teachers or administrators to add educational content through the PDF-to-vector-database ingestion process. The sidebar also displays the currently logged-in administrator’s account information and provides a sign-out option. This gives teachers a consistent way to move between student monitoring and content-management functions.

Teacher Code and Student Linking-A Teacher Code is displayed at the top of the dashboard. Each teacher has a unique code, which can be copied using the Copy button. This code is used to connect student accounts with the appropriate teacher. During registration, or later through the student’s dashboard, a student can enter the teacher code using the “Link to your teacher” option. Once the student is successfully linked, the teacher can access that student’s learning information through the Student Analytics dashboard. This linking mechanism ensures that teachers only view the progress and quiz information of students associated with their account.

Summary Statistics-The dashboard provides several summary cards that give teachers a quick overview of their current student group. These include Total Students, which shows the number of students linked to the teacher, and Active Now, which indicates students who are currently engaged with the learning platform. The dashboard also displays the Selected Student and Active Subject, which change according to the student’s record and subject currently being viewed. These summary statistics allow teachers to understand the current state of their students without having to examine individual records immediately.

Student List and Search-A student list is provided on the left side of the main analytics area. It contains the students who are linked to the teacher and includes a search function to make it easier to locate a particular student. This is especially useful when a teacher is responsible for a larger number of students. When a teacher

selects a student from the list, the student's learning information is loaded into the detail panel on the right side of the dashboard. If no student has been selected, the panel displays a message asking the teacher to select a student before viewing the analytics.

Student Learning Details-After selecting a student, the dashboard displays detailed information about the student's learning progress. The information is obtained from the backend by combining the student's recorded progress with the corresponding educational content. For each topic, the teacher can view information such as the initial quiz mark, final quiz mark, mastery level, and topic unlock status. The dashboard can also display the actual lesson content that was provided to the selected student. Displaying both the student's performance and the content delivered to them is particularly important in this system because the platform uses personalized and adaptive learning. Different students may receive different versions or levels of learning content depending on their learning progress. Therefore, the teacher is able to understand not only how a student performed, but also what learning material the student received before completing the related assessment.

Purpose of the Teacher Dashboard-The Teacher Dashboard provides a human-oversight layer for the adaptive learning system. Most of the student's learning process is automated, including personalized content generation, quiz generation, and progress tracking. However, teachers still need to be able to monitor this process and identify students who may require additional support. By reviewing quiz marks, mastery levels, unlocked topics, and the content provided to each student, teachers can identify students who are having difficulties and monitor their improvement over time. The dashboard also provides an opportunity for teachers to review the personalized learning experience and ensure that the generated educational content is appropriate. Therefore, the Teacher Dashboard connects the automated AI-based learning process with teacher supervision. This allows the platform to provide personalized learning while still maintaining teacher involvement, monitoring, and pedagogical accountability.

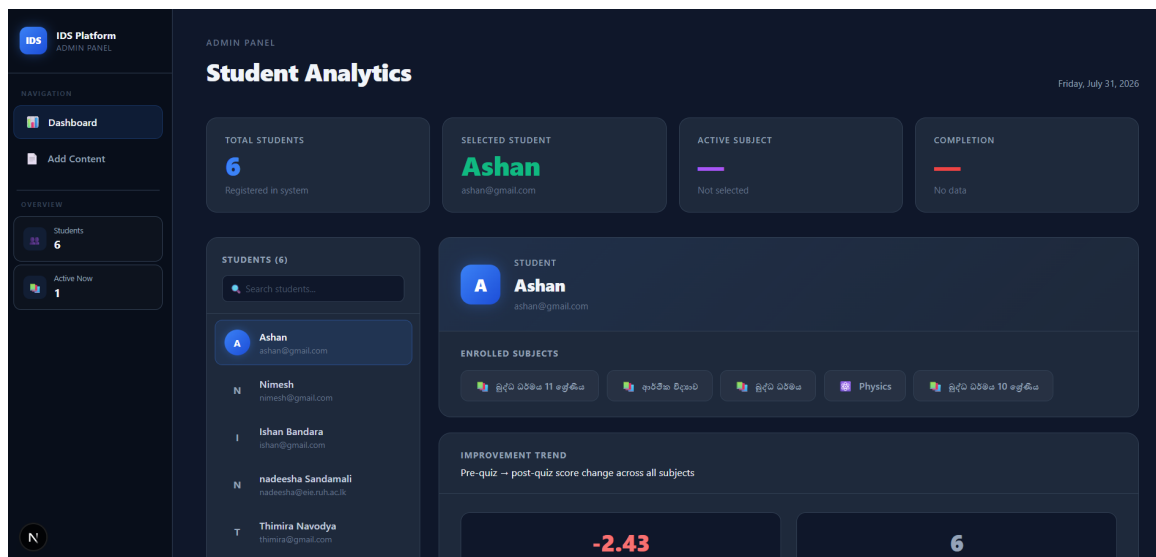


Figure 4.15: Teacher Dashboard page

When a teacher wants to upload new educational material, they navigate to the Lesson Adding page, as shown in Figure 4.16. This interface provides a straightforward form where teachers can structure their content before distributing it. They can enter the main lesson topic, break it down into a specific sub-topic, and add the detailed lesson content in the provided text area. Additionally, teachers can paste a YouTube video link to include multimedia resources for the students. Once the form is filled out, clicking the Submit Lesson button saves the content to the system, making it instantly accessible and ready to be integrated into the students' personalized learning paths.

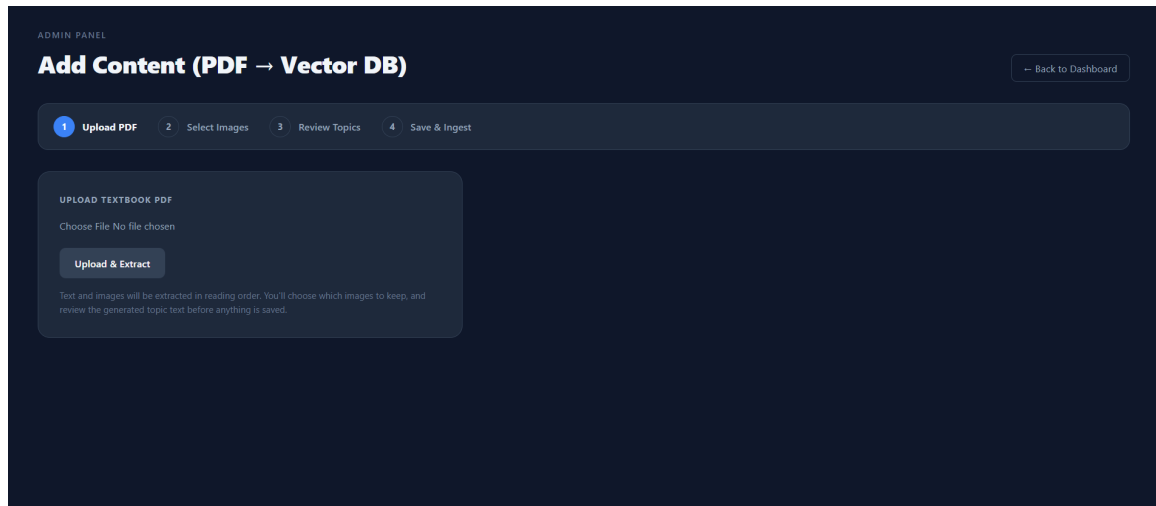


Figure 4.16: Lesson Adding page

When adding content, teachers often upload PDF documents. Figure 4.17 shows the second step of the PDF ingestion process, where the system extracts content from the uploaded file. Here, the teacher can specify the grade, subject, and lesson. The interface also allows the teacher to review any images extracted from the PDF and select which ones to keep for the lesson before generating the text.

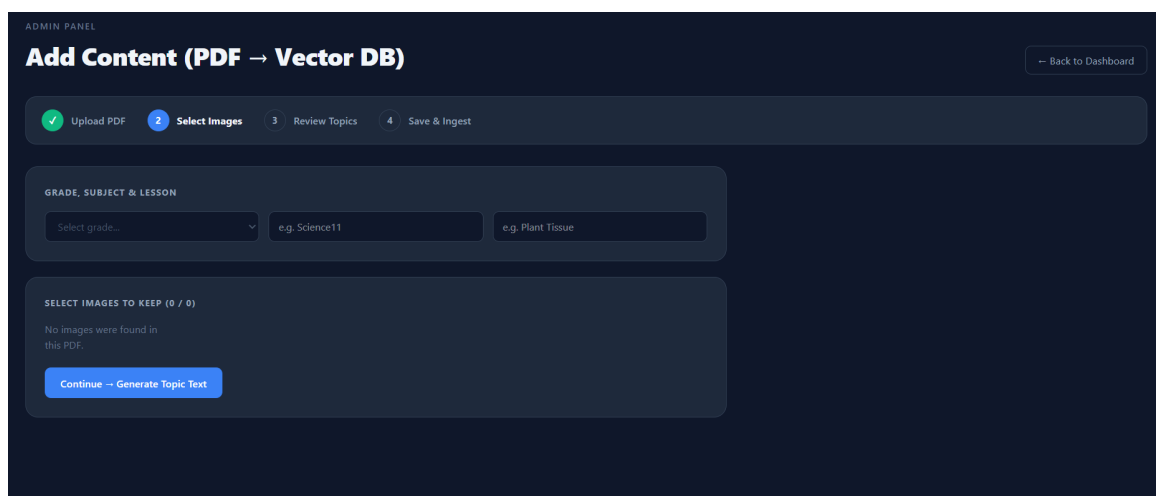


Figure 4.17: PDF Ingestion - Select Images

After the system extracts the text from the PDF, the teacher is taken to the topic review step, as shown in Figure 4.18. This page features a text editor that displays

the extracted Sinhala content. The teacher can easily read through the text, make any necessary corrections, and format it properly before saving it to the database.

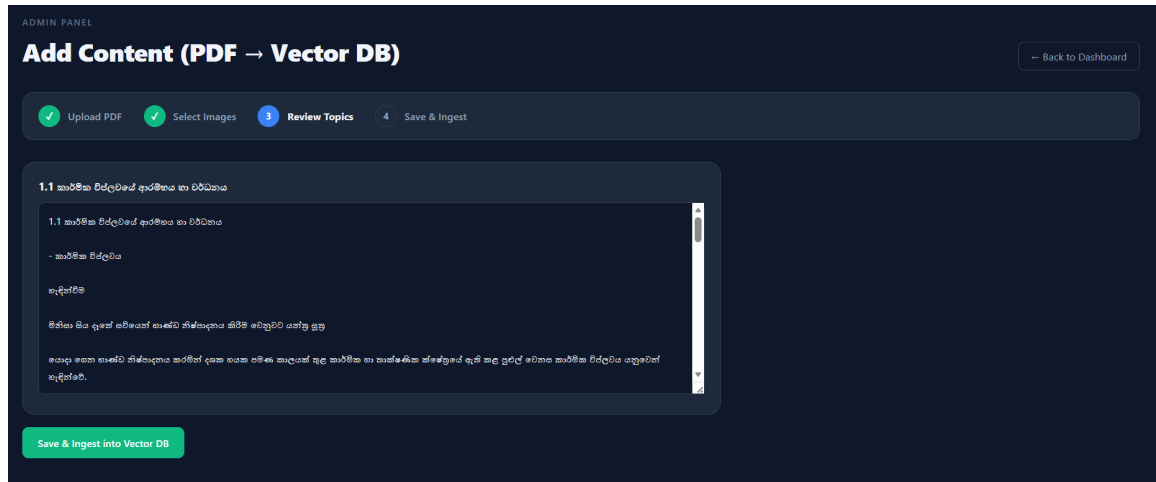


Figure 4.18: PDF Ingestion - Review Topics

Once the teacher approves the content, they proceed to the ingestion step. Figure 4.19 displays a simple loading screen that appears while the system saves the files and processes the text into the vector database. This behind-the-scenes processing ensures the content is properly indexed for the AI to use later.

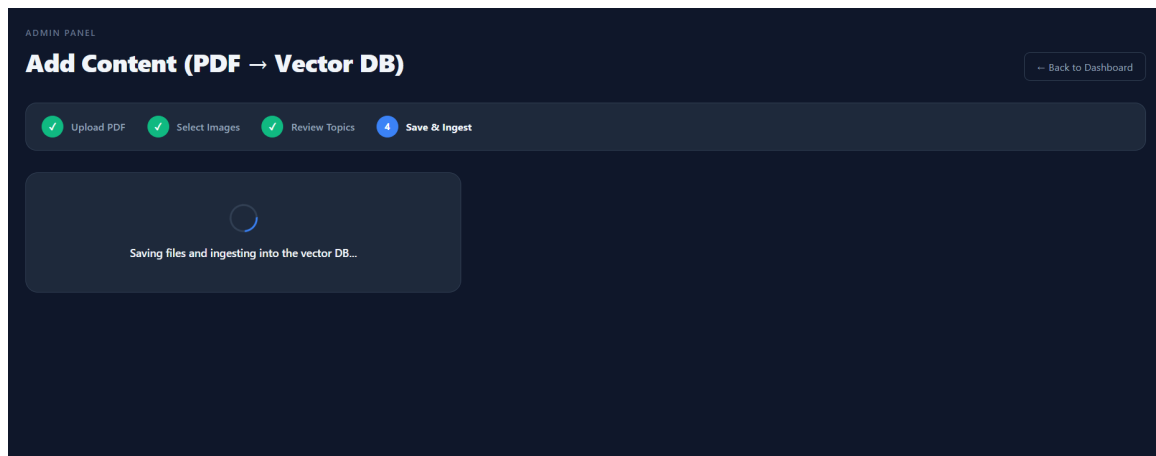


Figure 4.19: PDF Ingestion - Save and Ingest

Finally, Figure 4.20 shows the success screen that appears when the process is complete. It confirms that the topic files and images have been successfully saved into the system. From here, the teacher can choose to upload another PDF or return to the main dashboard.

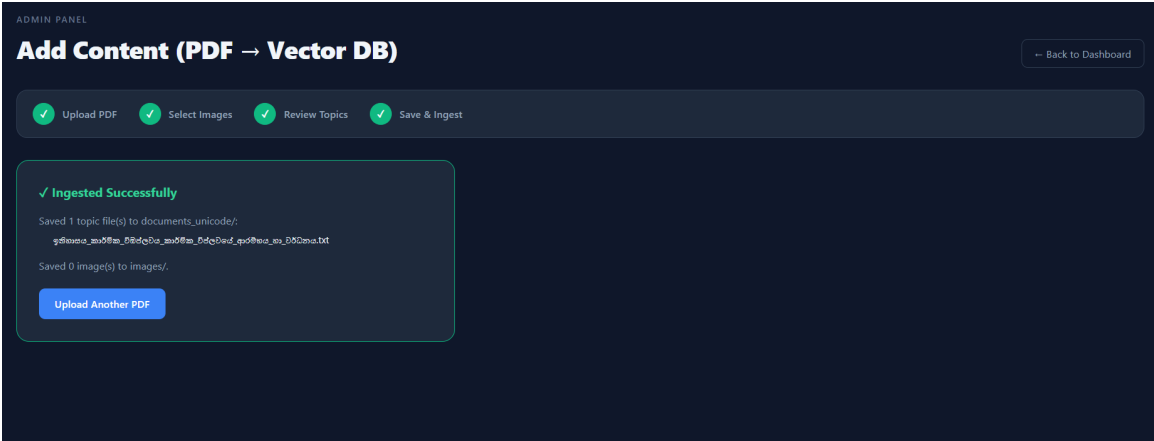


Figure 4.20: PDF Ingestion - Success

4.7 User Interfaces in Mobile App

The mobile application is designed specifically for parents and teachers, offering a convenient way to access the platform on the go. As shown in Figure 4.21, the signup page provides a simple mobile interface where new users can easily register by entering their basic details. For returning users, the login page (Figure 4.22) allows them to quickly sign into their accounts. Whether it is a teacher checking on their classes or a parent monitoring their child’s progress, both screens are designed with a clean, touch-friendly layout to ensure a smooth experience.

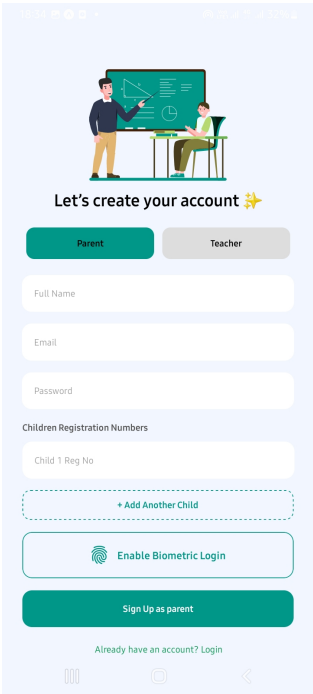


Figure 4.21: Signup page

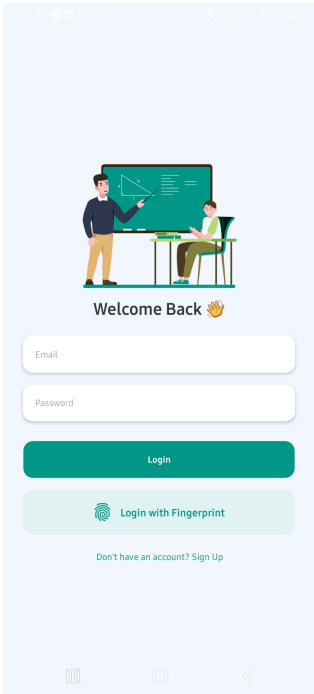


Figure 4.22: Login page

Once logged in, parents and teachers are greeted by the landing page, illustrated in Figure 4.23. This main screen acts as a mobile dashboard, giving quick access to enrolled subjects, upcoming lessons, or teaching materials depending on the user’s

role. Figure 4.24 displays the overall progress page, which is essential for tracking the student’s learning journey. Here, parents can see their child’s completion rates and quiz scores, while teachers can monitor the overall performance of their classes in a clear, easy-to-read format. This ensures that both parents and educators stay informed and engaged with the student’s progress directly from their smartphones.

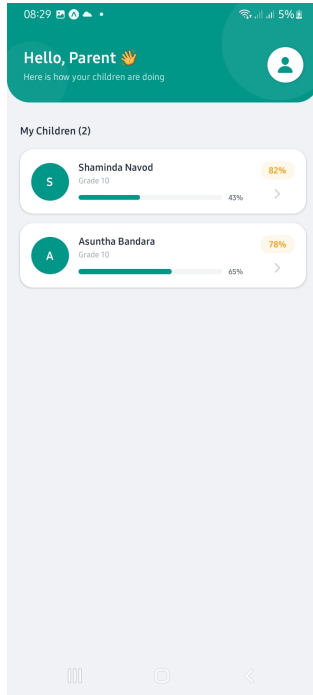


Figure 4.23: Landing page

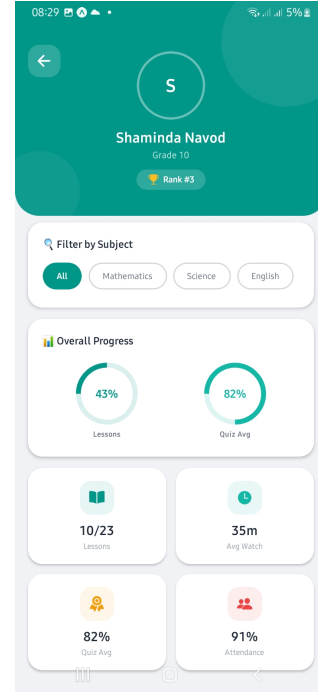


Figure 4.24: Student Overall Progress page

4.8 Achievements

A research paper based on this project, titled “An AI-Driven Personalized Sinhala Teaching Assistant for Secondary Education Using Curriculum Aligned Large Language Models,” has been submitted for the 2026 IEEE International Conference on Electrical Engineering and Emerging Technologies (ICEEET 2026). ICEEET 2026 is the twelfth edition of the International Electrical Engineering Conference series (initiated in 2016), and the first held under its new IEEE-approved title, reflecting an expanded scope that now covers emerging technologies alongside core electrical engineering. The conference is jointly organized by NED University of Engineering & Technology, Pakistan, and the Faculty of Engineering, University of Peradeniya, Sri Lanka, and will be held in a hybrid format in Kandy, Sri Lanka, on 17–18 October 2026, under the theme “International Collaboration for Global Progress: Towards a Sustainable and Connected Future.”

An AI-Driven Personalized Sinhala Teaching Assistant for Secondary Education Using Curriculum Aligned Large Language Models

Abstract—Sri Lanka faces a shortage of qualified teachers, particularly in rural regions, resulting in unequal learning opportunities for General Certificate of Education Ordinary Level (GCE O/L) and Advanced Level (GCE A/L) students. Existing digital platforms provide static, prerecorded content and lack adaptive personalization, while general-purpose Large Language Models (LLMs) lack curriculum alignment. This paper presents an Artificial Intelligence (AI)-driven Sinhala teaching assistant integrating four components: (i) a curriculum-aligned Sinhala LLM (SinLlama-8B) for educational content generation, (ii) a Hybrid Bayesian Knowledge Tracing (Hybrid-BKT) framework combining Personalized Clustered Bayesian Knowledge Tracing with Long Short-Term Memory (LSTM)-based temporal modeling for learner mastery estimation, (iii) a Retrieval-Augmented Generation (RAG) pipeline using ChromaDB-indexed syllabus content, and (iv) a LangGraph-orchestrated multi-agent architecture for instructional support. SinLlama-8B outperforms Qwen-14B, Llama-3.1-8B, DeepSeek-14B, and Mistral-7B in curriculum alignment and semantic fidelity. The Hybrid-BKT continuously estimates learner knowledge states, enabling adaptive content delivery based on proficiency levels. The production system uses a dual-LLM architecture: fine-tuned SinLlama-8B model handles curriculum-grounded generation through RAG, while Llama-3.3-70B powers rapid agent orchestration. Experimental results demonstrate the effectiveness of integrating curriculum-aligned LLMs, knowledge tracing, and RAG for personalized secondary education in Sri Lanka. An interactive AI Avatar powered by Gemini Text-to-Speech delivers Sinhala lesson explanations.

Keywords—Bayesian knowledge tracing, Intelligent tutoring systems, Personalized Learning Systems, Retrieval-augmented generation (RAG), Sinhala natural language processing (NLP)

I. INTRODUCTION

Education is widely recognized as a cornerstone of human development, yet students preparing for the General Certificate of Education Ordinary Level (G.C.E. O/L) and Advanced Level (G.C.E. A/L) examinations in Sri Lanka face severe systemic challenges. The country faces a shortage of more than 30,000 qualified teachers, causing student-to-teacher ratios to exceed 50:1, with the problem disproportionately affecting rural provinces [1]. These conditions perpetuate a stark divide in educational quality between urban and rural learners. Statistically, only 63.3% of O/L candidates qualify for A/L studies, blocking over 130,000 students annually from tertiary education pathways, while a mere 15% of A/L candidates secure university admission [2].

Several digital platforms have attempted to bridge this divide. The government-sponsored e-Thaksalawa portal [3] and private initiatives such as Dammika Perera (DP) Education offer pre-recorded video lectures, downloadable notes, and fixed question banks but lack the adaptive, interactive dimension necessary for effective personalized

tutoring. General purpose Large Language Models (LLMs), while increasingly capable in Sinhala, are not aligned with the Sri Lankan national curriculum and tend to generate generic responses inappropriate for exam preparation [4].

These observations point to a research gap that, to the best of our knowledge, has not been addressed in the literature: The absence of an Artificial Intelligence (AI) based educational system that (a) operates natively in the Sinhala medium, (b) strictly adheres to national syllabus content, (c) dynamically adapts to each student's evolving knowledge state, and (d) provides verifiable, curriculum-grounded responses rather than unchecked generative output, are the observations that point to the research gap.

The work is guided by four research questions investigating: (i) the effectiveness of different LLM architectures in achieving high semantic fidelity for curriculum-aligned Sinhala content generation after fine-tuning on national syllabus data, (ii) whether a hybrid ensemble combining Personalized Clustered BKT and LSTM-based temporal modeling improves student mastery estimation compared to conventional BKT approaches, (iii) the extent to which a RAG pipeline grounded in vectorized syllabus content can constrain model outputs within curriculum boundaries, and (iv) the ability of a multi-agent architecture to effectively orchestrate personalized and adaptive learning workflows for secondary-level education. Finally, a deployed web and mobile prototype is presented to demonstrate practical applicability.

The remainder of this paper is organized as follows. Section II reviews related work across intelligent tutoring systems, adaptive learning, LLMs in education, RAG, and multi-agent architectures. Section III details the system methodology, Section IV reports experimental results. Section V discusses findings and limitations, and Section VI concludes with directions for future work.

II. RELATED WORK

This section reviews five areas of related research: intelligent tutoring systems, adaptive learning and student modelling, generative AI and LLMs in education, retrieval-augmented generation for education, and multi-agent tutoring architectures.

A. Intelligent Tutoring Systems

Intelligent Tutoring Systems (ITS) employ machine learning, educational data mining, and adaptive assessment to personalize instruction [6]. Xu et al. [7] demonstrated that AI-driven adaptive learning frameworks significantly improve engagement, assessment scores, and knowledge retention compared to non-adaptive systems. However, most ITS

Figure 4.25: A Snapshot of the First Page of the Submitted Paper for ICEEET 2026

Chapter 5

User Feedback

Following the deployment of the adaptive learning platform, initial user feedback was collected from Grade 11 students to evaluate the practical impact and user acceptance of the system. The responses, gathered through direct messaging, were overwhelmingly positive and highlighted the effectiveness of the AI-driven personalization and the interactive teaching avatar.

As shown in Figure 5.1, one student noted that the platform significantly accelerated their learning process, making the subject matter much easier to grasp. The student emphasized the immense value of the system, mentioning that they enthusiastically recommended it to their peers, who also found it highly beneficial. A recurring sentiment among the users was the desire to see this personalized learning approach expanded to cover all subjects in the curriculum, underscoring the strong demand for localized, adaptive e-learning solutions.

Figure 5.2 illustrates another student's experience, who initially expressed skepticism about learning from an AI, questioning whether the automated explanations would be comprehensible. However, after using the platform, their perception shifted entirely. They found the system to be highly practical and noted that the AI's explanations were exceptionally simple and clear. Furthermore, the engaging nature of the teaching avatar—both visually and auditorily—was particularly praised, with the student stating that it made the learning experience highly enjoyable to watch and listen to.

Figure 5.3 highlights the psychological benefits of the system's supportive design. One student mentioned that the friendly and non-judgmental nature of the AI avatar allowed them to ask questions without any embarrassment. They particularly appreciated the encouraging feedback provided when making mistakes in quizzes, which motivated them to keep trying without feeling discouraged. The student also noted a significant improvement in understanding complex science topics that were previously difficult to grasp, emphasizing a strong desire to continue using the platform for daily practice.

Similarly, Figure 5.4 demonstrates the effectiveness of the system's adaptive difficulty and human-like interaction. A student who initially felt anxious about taking quizzes found that the questions were appropriately tailored to their level, allowing them to answer confidently. Furthermore, the student expressed that the platform felt like having a real teacher sitting right beside them rather than interacting with a rigid robotic system, ultimately leading to a much deeper understanding of the subject matter.

These testimonials demonstrate that despite initial reservations about AI-driven

education, students quickly recognize its value when presented with a well-designed, curriculum-aligned, and interactive platform. The feedback strongly validates the project's core objective: delivering personalized, engaging, and high-quality education to Sri Lankan students through advanced generative AI.

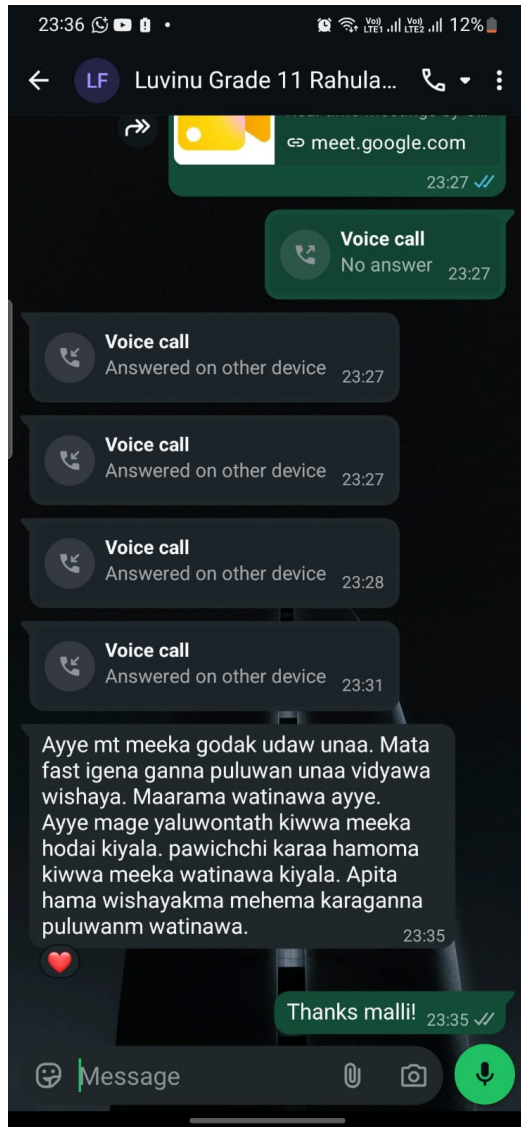


Figure 5.1: Student feedback highlighting accelerated learning and value

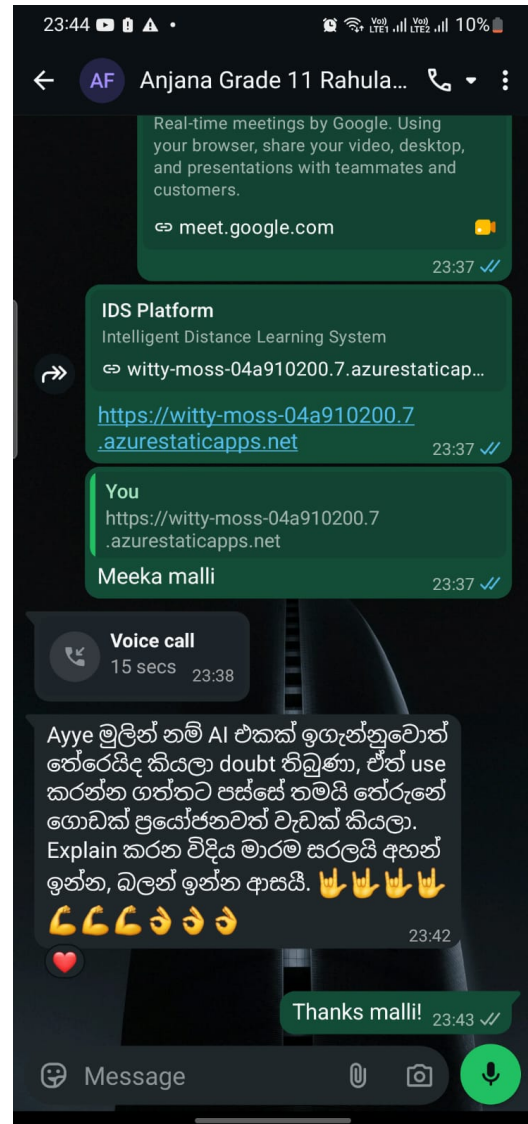


Figure 5.2: Student feedback on the clarity and engaging nature of the AI avatar

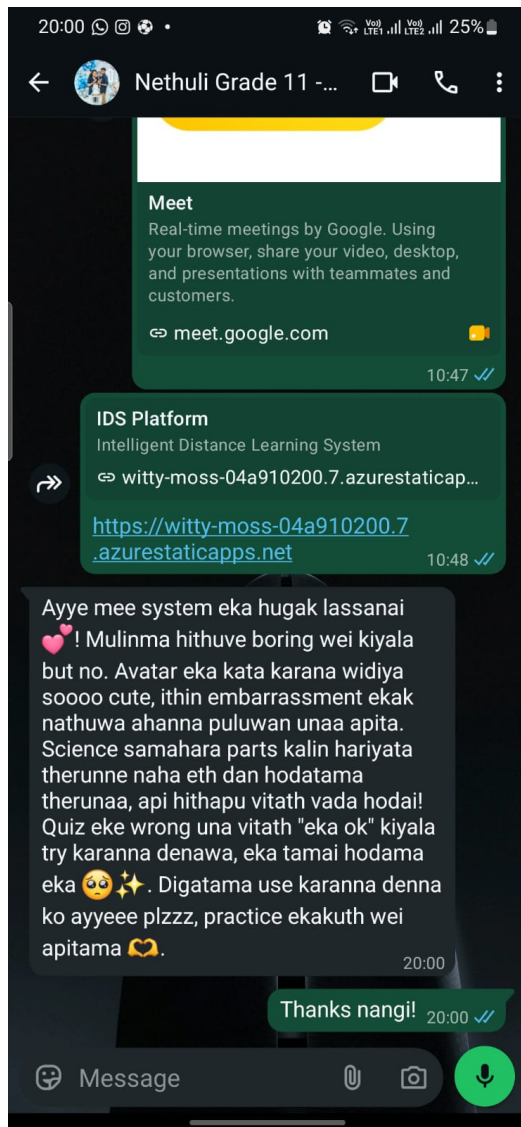


Figure 5.3: Student feedback on the supportive avatar and encouraging quiz experience

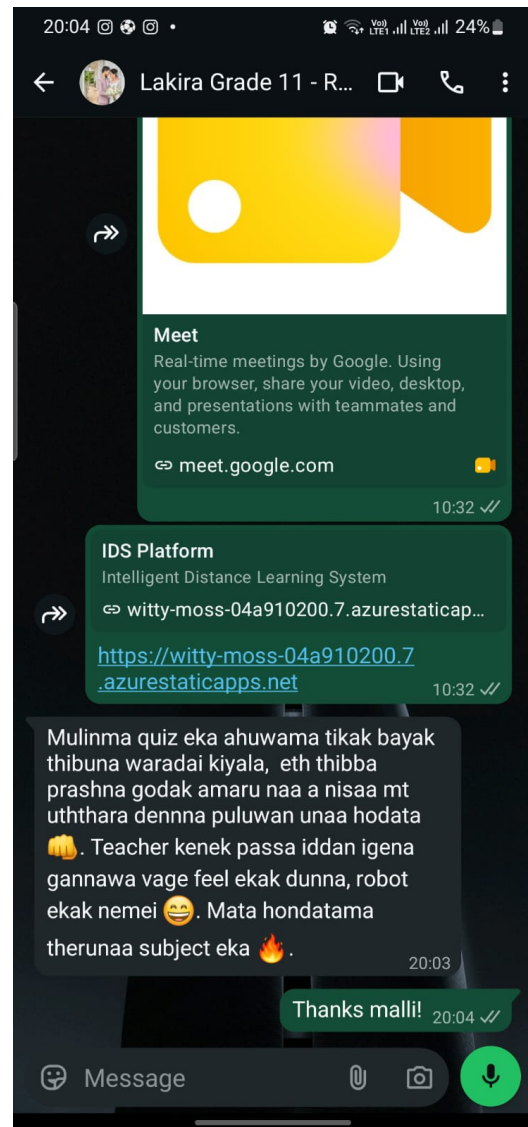


Figure 5.4: Student feedback on personalized difficulty and the human-like teaching experience

Chapter 6

Conclusion

6.1 Summary of the Project

This project successfully designed, implemented, and evaluated a comprehensive AI-based intelligent tutoring system specifically tailored for the Sri Lankan Sinhala-medium education sector. Focusing initially on the Grade 11 Buddhism curriculum, the research addressed the critical lack of localized, curriculum-aligned personalized learning platforms. By orchestrating multiple advanced AI technologies through a sophisticated multi-agent architecture (LangGraph), the system achieved a seamless educational workflow. This workflow seamlessly integrates automated quiz generation, dynamic lesson adaptation based on student mastery, real-time engagement tracking, and conversational delivery via an AI teaching avatar. The successful deployment of this platform as functional web and mobile applications demonstrates the immense potential of integrating state-of-the-art generative AI and adaptive learning algorithms into national educational frameworks.

6.2 Novel and Important Aspects of the Methodology

In contrast to previous AI systems, the proposed solution brings many advantages and introduces several novel components specifically tailored for the Sri Lankan educational context. Firstly, this work incorporates a Hybrid-BKT algorithm that enhances personalized education by integrating PC-BKT with an LSTM predictor model. It should be noted that the standard BKT technique presupposes a uniform learning pattern among all students; however, such an assumption may be unrealistic. The final prediction is calculated through a weighted sum, namely, $P_{\text{Hybrid}} = 0.65 \times P_{\text{PC-BKT}} + 0.35 \times P_{\text{LSTM}}$, thus allowing for a balanced consideration of both interpretability and temporal learning behavior, where the former model focuses on student clustering patterns while the latter highlights learning patterns over time. Notably, such an approach is not widespread in Sinhala-based educational systems.

Secondly, the current research involves a curriculum-aligned Sinhala language AI model. Typically, most existing language models are trained mainly based on English text data, which makes it difficult to incorporate the Sri Lankan school curriculum into their operation. In the current project, five different LLMs were fine-tuned through a domain-specific dataset. The fine-tuning went beyond simple question-answering by introducing task-specific instruction sets that taught the model to dynamically

generate quizzes, explain content, and adapt language difficulty to the student’s mastery level. Among the evaluated models, SinLlama proved to be the most effective, producing a ROUGE-1 score of 0.7714 and a BERTScore of 0.9424.

Thirdly, the system introduces a dedicated Real-Time Avatar Speech Delivery pipeline. Unlike traditional platforms that rely on pre-rendered video files, this system utilizes an AVTR-1 generative model operating over a low-latency WebRTC connection. By generating chunk-based facial motion conditioned on synthesized speech embeddings, the avatar provides live, dynamically generated interactions. Additionally, a secondary client-rendered 3D avatar mode utilizing Gemini TTS offers a lightweight alternative, ensuring the platform remains highly accessible.

Fourthly, an advanced Engagement Detection System was integrated to continuously monitor student attention. By fusing YOLOv8-based physical tracking (head pose, eye aspect ratio, and yawning detection) with EmotionCNN-based facial expression analysis, the system calculates a real-time, weighted engagement score. This continuous feedback loop allows the educational platform to autonomously intervene if student focus drops, a feature rarely found in local e-learning environments.

Fifthly, the system adopts a sophisticated multi-agent architecture supported by LangGraph, whereby various agents-such as the Content Agent, Student Modeling Agent, Evaluator Agent, and Progress Tracker Agent-undertake different activities autonomously. This is combined with a robust RAG pipeline that retrieves relevant knowledge from vectorized Sinhala textbooks, ensuring that generated content remains strictly within syllabus limits.

To support this RAG pipeline, a novel Automated Content Ingestion Pipeline was developed to overcome the challenge of legacy Sinhala font encodings in existing curriculum PDFs. A font-aware, 13-stage text cleaning and extraction process successfully transforms unstructured, legacy-encoded documents into structured, Unicode-compliant knowledge chunks without destroying educational context, drastically reducing the manual effort required to digitize national curriculum resources.

Lastly, the proposed system resolves the cold start problem by adopting an initialization approach based on clusters. The system does not adopt the assumption of prior probability of knowledge, $P(L_0) = 0.3$ for all students, but divides learners into three clusters according to their abilities (High, Medium, and Low). Students’ prior mastery probabilities are assigned based on the nearest cluster, improving prediction accuracy during the early stage by about 12%.

6.3 Results and Final Observations

Five LLMs have been tuned using the prepared Buddhism dataset and tested using ROUGE scores, BERT Score, token generation time, and LLM-as-a-Judge evaluation criteria. Among all the models, SinLlama proved to be superior by attaining maximum ROUGE-1 (0.7714), ROUGE-2 (0.6418), and BERTScore (0.9424). This implies that SinLlama performs better at capturing semantic and context-based information from Sinhala curriculum-based documents than any other evaluated LLM.

As per the qualitative evaluation, Mistral performed the second best among all the models, scoring 14.97/20. The SinLlama scored the highest score (15.88/20) among all models. Although DeepSeek proved to be less accurate in terms of semantics than other models, DeepSeek generated the most number of tokens (444.9 tokens/sec), indicating that this model can be preferred in latency-critical applications where

accuracy is secondary to response time.

To assess the effectiveness of Hybrid-BKT as a personalized learning system, we tested it on data involving more than 6,500 student interactions across 47 knowledge components from Buddhism. Our analysis showed that Hybrid-BKT was able to provide superior performance compared to Standard BKT and BKT-LSTM models with an accuracy rate of 56.2% and RMSE of 0.49. The performance of Standard BKT was lower, with 48.1% accuracy and RMSE of 0.51, while BKT-LSTM model had the accuracy rate of 54.7% and RMSE of 0.53. It can be inferred that hybrid models combining probability and time learning algorithms have greater potential to make more accurate predictions.

Furthermore, the model was able to sustain high-performance levels while being evaluated, with an average inference latency of less than 200 ms for each update. It indicates its feasibility in interactive quiz-based environments that require timely assessments of user progress. Moreover, the loss of LSTM decreased from 0.58 to 0.28 during training over 50 epochs, thus demonstrating successful learning of temporal dependencies in students' performance.

In terms of software implementation, the proposed solution proved fully functional through the creation of a working web-based system. The design included login capabilities, a student dashboard, browsing lessons, and pre-/post-quiz sequence. Beyond the core web platform, a companion React Native mobile application was developed to give parents and teachers real-time access to student learning analytics and mastery trends, extending the system's monitoring capabilities beyond the classroom. The engagement detection system was similarly validated as a fully integrated, real-time component of the deployed platform, continuously informing the personalization engine during live lesson delivery rather than functioning as an isolated experimental module.

6.4 Limitations and Recommendations for Future Work

While the system successfully demonstrates a functional, curriculum-aligned adaptive learning platform, several limitations were identified during development and evaluation, each suggesting a concrete direction for future work.

Deployment dependency on local GPU infrastructure. The fine-tuned SinLlama model and AVTR-1 avatar server currently run on a locally available GPU machine, connected to the Azure-hosted application via ngrok tunnels. Since the free tier of ngrok assigns a new public URL on every restart, this requires manual reconfiguration whenever the tunnel or GPU machine restarts. Future work should migrate these components to dedicated GPU-enabled cloud infrastructure, or adopt a fixed tunnel domain, to make the deployment fully autonomous.

Digitization pipeline scope. The content ingestion pipeline currently handles Unicode and legacy-font-encoded Sinhala PDFs but does not support OCR for scanned image-only pages, and its heading/topic detection relies on formatting heuristics that may require manual correction for inconsistently formatted documents. Extending the pipeline with OCR support and more robust structural detection would further reduce administrator effort when onboarding new curriculum materials.

Bibliography

- [1] Advice.lk, “Complete guide to tuition classes in sri lanka,” *Advice.lk*, 2024.
- [2] Education Online, University of Kansas, “Personalizing learning through adaptive curriculum strategies,” in *Education Online*, Oct. 2025.
- [3] TAO Testing, “Adaptive learning explained: Tailoring education to student needs,” in *TAO Testing Blog*, May 2024.
- [4] M. Chandana, N. Habaragamuwa, and A. Fonseka, “Use of artificial intelligence (ai) to improve foundational literacy and numeracy in sri lanka,” in *LIRNEasia*, Oct. 2025.
- [5] A. Abayasekara, U. Perera, and T. de Silva, “Shadow education in sri lanka during covid-19: Trends, impacts, and role in relation to mainstream education,” health and education research series no. 21, Institute of Policy Studies of Sri Lanka, 2023. May 2023.
- [6] Ministry of Education, Sri Lanka, “e-thaksalawa: National e-learning portal and learning content management system.” Ministry of Education, Sri Lanka, 2025. Official national e-learning platform for Grades 1–13 general education.
- [7] H. W. K. Aravinda, R. Sirajudeen, S. Karunathilake, N. de Silva, S. Ranathunga, and R. Kaur, “Sinllama—a large language model for sinhala,” *arXiv preprint arXiv:2508.09115*, 2025.
- [8] G. Karunarathna, “Revolutionizing sri lanka’s education system with artificial intelligence: A path to future-ready learning.” LinkedIn, 2025. Professional article on AI integration in Sri Lanka’s education system.
- [9] StudyWay.lk, “Tuition classes and tutor search in sri lanka.” StudyWay.lk, 2025. Sri Lanka tuition and tutor information (O/L, A/L levels).
- [10] Reddit user, “I’m building a learning platform for sri lankan students, would love honest feedback.” Reddit, r/SriLanka, Jan. 2026. Online social discussion post.
- [11] BrainUs Team, “Best ai for o/l students sri lanka: Complete guide to brainus ai.” BrainUs Blog, Jan. 2026. Guide on AI tools for Ordinary Level students in Sri Lanka.
- [12] Academync Team, “Ai-powered personalized learning: Adaptive curriculum based on student behavior.” Academync Articles, 2025. Discussion on AI adaptive learning and behavior-based personalization.

- [13] Z. Xu, J. Wang, and Y. Liu, “Adaptive learning systems with AI-driven personalized instruction: Design, implementation, and evaluation,” *Computers*, vol. 13, no. 10, p. 270, 2024.
- [14] H. Liu, X. Zhang, and M. Chen, “Advances in intelligent tutoring frameworks for interactive learning and responsive feedback,” *Smart Learning Environments*, vol. 10, pp. 1–15, 2023.
- [15] S. Wang, A. Qiao, and H. Abduljabbar, “Adaptive learning analytics approaches for personalized education,” *Heliyon*, vol. 11, no. 2, pp. 1–12, 2024.
- [16] A. O. Ayeni, R. E. Ovbiye, A. S. Onayemi, and K. E. Ojedele, “Ai-driven adaptive learning platforms: Enhancing educational outcomes for students with special needs through user-centric tailored digital tools,” 2024.
- [17] J. Kim, Y. Park, and S. Choi, “Reinforcement learning-based student modeling for adaptive learning systems,” in *Proc. IEEE Int. Conf. on Advanced Learning Technologies (ICALT)*, pp. 1–8, 2023.
- [18] R. Alshahrani, K. Almarashdah, and H. Almashaqbeh, “Deep learning models for personalized adaptive learning systems,” *IEEE Access*, vol. 10, pp. 12345–12358, 2024.
- [19] S. Lee and J. Park, “Intelligent feedback generation in adaptive learning algorithms based on student interaction data,” in *2024 IEEE International Conference on Advanced Learning Technologies*, pp. 234–241, 2024.
- [20] M. Nouri, F. Hassanzadeh, and A. Monajemi, “Ai-driven personalized education: Adaptive recommendation and learning path optimization,” *Computers & Education: Artificial Intelligence*, vol. 6, no. 1, pp. 1–15, 2025.
- [21] A. Author, “Educational teaching avatars: Design, interaction, and ai-based personalization in digital learning environments.” Preprints, Aug. 2025.
- [22] L. Nanayakkara *et al.*, “A human quality text-to-speech system for sinhala,” in *Proceedings of the 6th International Workshop on Spoken Language Technologies for Under-Resourced Languages (SLTU)*, 2018.
- [23] R. Weerasinghe, A. Wasala, and K. Gamage, “Festival-si: A sinhala text-to-speech system,” in *Proceedings of the International Conference on Text, Speech and Dialogue (TSD)*, Springer, 2007.
- [24] W. Manamperi *et al.*, “Sinhala speech recognition for interactive voice response systems accessed through mobile phones,” in *Proceedings of the IEEE Moratuwa Engineering Research Conference (MERCon)*, 2018.
- [25] O. Zawacki-Richter, V. I. Marín, M. Bond, and F. Gouverneur, “Systematic review of research on artificial intelligence applications in higher education,” *International Journal of Educational Technology in Higher Education*, vol. 16, no. 1, 2019.

- [26] E. Kasneci *et al.*, “Chatgpt for good? on opportunities and challenges of large language models for education,” *Learning and Individual Differences*, vol. 103, 2023.
- [27] W. Holmes, M. Bialik, and C. Fadel, *Artificial Intelligence in Education: Promises and Implications for Teaching and Learning*. Center for Curriculum Redesign, 2022.
- [28] J. Swacha and M. Gracel, “Retrieval-augmented generation (rag) chatbots for education: A survey of applications,” *Computers & Education: Artificial Intelligence*, 2025.
- [29] Y. Lian, “Machine assistant with reliable knowledge: Enhancing student learning via rag-based retrieval,” *arXiv preprint*, 2023.
- [30] Z. Li, Z. Wang, W. Wang, K. Hung, H. Xie, and F. L. Wang, “Retrieval-augmented generation for educational application: A systematic survey,” *Applied Sciences*, vol. 15, no. 8, p. 4234, 2025.
- [31] Y. Lv, W. Ni, and Y. Tan, “An overview of research on multimodal teaching and learning,” *Journal of Education and Educational Research*, 2024.
- [32] Z. C. Mou, “A study of multimodal intelligent adaptive learning system and its implementation,” *David Publishing*, 2023. Available: David Publisher.
- [33] Y. Yuan, Z. Li, and B. Zhao, “A survey of multimodal learning: Methods, applications, and future,” *ACM Computing Surveys*, 2025.
- [34] A. Author and Others, “Intellicode: A multi-agent llm tutoring system with centralized learner modeling,” *arXiv preprint*, vol. arXiv:2512.18669, 2025.
- [35] B. Author, “Dynamic tutor-tutee pairing system,” in *Proceedings of the International Conference on Information Systems (ICIS)*, 2024.
- [36] C. Glossner, D. R. Smith, and E. Johnson, “Multi-agent mathematics learning,” *Rivier Online Academic Journal*, vol. 18, no. 1, 2024.
- [37] Author/Organization, “Digital transformation in higher education: Learning analytics dashboards,” 2024.
- [38] Author, “Teacher-facing analytics dashboards: A systematic review,” *ERIC Journal of Educational Research*, 2024.
- [39] Author, “Ai in student progress tracking,” 2024. SchoolAI Blog.
- [40] LangChain, “Langgraph supervisor.” LangChain Documentation, 2025. Official documentation for the Supervisor architecture and multi-agent orchestration.
- [41] LangChain, “Langgraph stategraph api reference.” LangChain Documentation, 2025. Official documentation describing shared state management between graph nodes and agents.
- [42] LangChain, “Langgraph documentation.” LangChain Documentation, 2025. Official documentation for building stateful, multi-agent applications with LLMs.

- [43] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [44] R. Smith, “An overview of the tesseract ocr engine,” in *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR)*, pp. 629–633, 2007.
- [45] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, pp. 3982–3992, 2019.
- [46] L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, and F. Wei, “Text embeddings by weakly-supervised contrastive pre-training,” *arXiv preprint arXiv:2212.03533*, 2022.
- [47] Chroma, “Chroma documentation.” Chroma Documentation, 2025. Official documentation for the Chroma vector database.
- [48] WebRTC, “Webrtc: Real-time communication in browsers,” *W3C Recommendation*, 2024.
- [49] Google, “Gemini api: Text-to-speech,” *Google AI Developer Documentation*, 2025.
- [50] D. S. Wolf, “Rhubarb lip sync,” *GitHub Repository*, 2024.
- [51] P. Nedungadi and M. Remya, “Personalized bayesian knowledge tracing model for learning analytics,” *International Journal of Artificial Intelligence in Education*, vol. 24, no. 3, pp. 1–15, 2014.
- [52] S. Minn, “Bkt-lstm: Integrating bayesian knowledge tracing with long short-term memory networks for student performance prediction,” in *Proceedings of the International Conference on Educational Data Mining*, 2021.
- [53] V. Kazemi and J. Sullivan, “One millisecond face alignment with an ensemble of regression trees,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1867–1874, 2014.
- [54] S. Li and W. Deng, “Deep facial expression recognition: A survey,” *IEEE Transactions on Affective Computing*, vol. 13, no. 3, pp. 1195–1215, 2018.
- [55] K. Murphy-Chutorian and M. M. Trivedi, “Head pose estimation in computer vision: A survey,” in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2011.
- [56] R. A. Calvo and D. Peters, *Affective Computing*. MIT Press, 2010.
- [57] C.-Y. Lin, “Rouge: A package for automatic evaluation of summaries,” in *Text summarization branches out*, pp. 74–81, 2004.
- [58] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating text generation with bert,” in *International Conference on Learning Representations*, 2020.

- [59] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging llm-as-a-judge with mt-bench and chatbot arena,” in *Advances in Neural Information Processing Systems*, 2023.
- [60] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022.